



**SERVICIO NACIONAL DE APRENDIZAJE (SENA)
ANÁLISIS Y DESARROLLO DE SOFTWARE**

NOMBRE DEL PROYECTO: ByteQuest

FICHA:2848530B

APRENDICES:

Luis Alejandro Gonzalez

Andres Felipe Miranda

Alejandro Avella

Yeimy Trujillo

BOGOTA, 2025

Tabla de contenido

1. PORTADA
2. Resumen (Informe técnico sobre el proyecto, máximo 300 palabras)
3. Abstract (Informe técnico sobre el proyecto en inglés)
4. INTRODUCCIÓN (Al documento)
5. PLANTEAMIENTO DEL PROBLEMA
 - 5.1. DESCRIPCIÓN DEL PROBLEMA
 - 5.2. OBJETIVO GENERAL
 - 5.3. OBJETIVOS ESPECÍFICOS
 - 5.4. JUSTIFICACIÓN DEL PROYECTO
 - 5.5. ALCANCE DEL PROYECTO
 - 5.6. MATRIZ DE RIESGO
6. ELICITACIÓN DE REQUISITOS
 - 6.1. IDENTIFICACIÓN DE PROCESOS
 - 6.2. RECOLECCIÓN DE LA INFORMACIÓN DEL SOFTWARE A CONSTRUIR DE ACUERDO CON LAS NECESIDADES DEL CLIENTE.
 - 6.2.1. ELECCIÓN DE LA TÉCNICA DE RECOLECCIÓN DE LA INFORMACIÓN.
 - 6.2.2. DISEÑO DE LOS FORMATOS, SEGÚN LA TÉCNICA O TÉCNICAS DE RECOLECCIÓN DE LA INFORMACIÓN SELECCIONADA(S).
 - 6.2.3. APLICACIÓN DE LA TÉCNICA DE RECOLECCIÓN DE LA INFORMACIÓN.
 - 6.2.4. ORGANIZACIÓN DE LA INFORMACIÓN RECOLECTADA
 - 6.3. ESPECIFICACIÓN DE REQUERIMIENTOS, DE ACUERDO CON LA INFORMACIÓN RECOLECTADA, APLICANDO ESTÁNDARES DE REQUERIMIENTOS
 - 6.3.1. REQUERIMIENTOS FUNCIONALES

- 6.3.2. REQUERIMIENTOS NO FUNCIONALES
 - 6.3.3. REQUERIMIENTOS NORMATIVOS
 - 6.3.4. PROPUESTA TÉCNICA
- 7. REGLAS DEL NEGOCIO
- 8. ANÁLISIS DE LA ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE.
 - 8.1. ALTERNATIVAS DE SOLUCIÓN (PROTOTIPO O PROTOTIPOS DEL SISTEMA, MOCKUPS)
 - 8.2. DIAGRAMA DE CASOS DE USO Y EXTENSIBILIDAD DE ESTOS
 - 8.3. DIAGRAMAS DE ACTIVIDADES, Y SECUENCIAS
 - 8.4. CONSTRUIR EL MODELO DE DOMINIO DEL SISTEMA (DIAGRAMA DE CLASES).
 - 8.5. ELABORAR EL MODELO ENTIDAD RELACIÓN.
- 9. DISEÑO DE LA SOLUCIÓN DEL SOFTWARE DE ACUERDO CON LOS PROCEDIMIENTOS Y REQUISITOS TÉCNICOS
 - 9.1. ARQUITECTURA DEL SOFTWARE (DIAGRAMA DE COMPONENTES), Y PATRONES DE DISEÑO DE SOFTWARE.
 - 9.2. DIAGRAMA DE DESPLIEGUE.
 - 9.3. DISEÑO FRONT-END - INTERFACES GRÁFICAS DE USUARIO (HTML.CSS Y JS)
 - 9.4. INTERFACES GRÁFICAS DE USUARIO MÓVILES.
 - 9.5. MAPA DE NAVEGACIÓN DE LA APLICACIÓN.
 - 9.6. DETERMINAR TIPOS DE BASES DE DATOS.
 - 9.7. MODELO DE DATOS DIAGRAMA ER (NORMALIZACIÓN BASE DE LA DATOS (Mínimo tercera forma normal)
 - 9.8. DICCIONARIO DE DATOS.
 - 9.9. POLÍTICAS DE SEGURIDAD DE LOS DATOS.
- 10. CONSTRUCCIÓN DEL SOFTWARE
 - 10.1. BASE DE DATOS PARA EL SOFTWARE A PARTIR DEL MODELO DE DATOS.

10.1.1. OBJETOS DE LA BASE DE DATOS (PROCEDIMIENTOS ALMACENADOS, VISTAS, DISPARADORES)

10.1.2. ESQUEMAS DE SEGURIDAD DE LOS DATOS.

10.2. CODIFICACIÓN DEL SOFTWARE DE ACUERDO CON EL DISEÑO ESTABLECIDO.

10.2.1. FRONT END (HTML, CSS, JAVASCRIPT, RESPONSIVE)

10.2.2. ESTÁNDAR DE CODIFICACIÓN (DOCUMENTACIÓN DEL CÓDIGO, APLICABILIDAD DEL ESTÁNDAR DE CODIFICACIÓN).

10.2.3. CÓDIGO FUENTE DE LOS MÓDULOS DEL SOFTWARE WEB Y MÓVIL.

10.2.4. SERVICIOS WEB.

10.2.5. CONTROL DE VERSIONES

10.2.6. PRUEBAS DEL SOFTWARE

10.2.6.1. PLANEACIÓN Y DISEÑO DE LAS PRUEBAS A NIVEL UNITARIO(MÓDULOS), CON BASE EN LOS REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES.

10.2.6.2. EJECUCIÓN DE LAS PRUEBAS, INFORME DE HALLAZGOS

10.2.6.3. CORRECCIÓN DE ERRORES, Y DOCUMENTACIÓN DE LOS MISMOS.

10.2.6.4. MANEJO DE ALERTAS / EXCEPCIONES

10.2.6.5. PLANEACIÓN Y DISEÑO DE LAS PRUEBAS A NIVEL SISTEMA, CON BASE EN LOS REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES.

10.2.6.6. EJECUCIÓN DE LAS PRUEBAS, INFORME DE HALLAZGOS

10.2.6.7. CORRECCIÓN DE ERRORES, Y DOCUMENTACIÓN DE LOS MISMOS.

11. PLAN DE DESPLIEGUE (HOSTING, DOMINIO, SUBDOMINIO, COSTOS, HERRAMIENTAS, PLATAFORMA, SEGURIDAD)

12. IMPLANTACIÓN DEL SOFTWARE.

12.1. PLAN DE IMPLANTACIÓN (PLATAFORMAS TECNOLÓGICAS)

- 12.2. PLAN DE CAPACITACIÓN DE USUARIOS DEL SISTEMA.
 - 12.3. MANUAL DEL USUARIO (PDF, MÓDULO AYUDA)
 - 12.4. COPIAS DE SEGURIDAD DE DATOS Y RESPALDOS
 - 12.5. GARANTÍA (CONTRATOS, ACUERDOS DE NIVEL DE USUARIO)
- 13. ADOPCIÓN DE BUENAS PRÁCTICAS EN EL PROCESO DE DESARROLLO DE SOFTWARE.
 - 14. CONCLUSIÓN
 - 15. BIBLIOGRAFÍA y CIBERGRAFÍA
 - 16. GLOSARIO

2. Resumen (Informe técnico sobre el proyecto)

ByteQuest es una plataforma educativa en desarrollo, cuyo objetivo es enseñar lenguajes de programación de forma gamificada e interactiva, tomando como referencia la metodología de Duolingo. Esta diseñada bajo una arquitectura cliente-servidor, con tecnologías web modernas.

El **backend** está desarrollado en PHP con programación orientada a objetos (POO), lo que facilita la modularidad y el mantenimiento del sistema. Para el almacenamiento de datos se utiliza una base de datos MySQL, que gestiona usuarios, progreso, contenidos, resultados y estadísticas.

En el **frontend**, se utilizan HTML5, CSS3 y JavaScript puro para la interfaz, considerando una futura integración con frameworks como Bootstrap para mejorar la experiencia responsive y el diseño visual.

El sistema cuenta con los siguientes módulos principales:

- **Autenticación de usuarios:** Registro, inicio de sesión y recuperación de contraseña.
- **Gestión de progreso:** Seguimiento de lecciones completadas, puntos, medallas, vidas y niveles.
- **Módulo de ejercicios:** Banco de preguntas interactivas, ejercicios de código y validación automática.
- **Gestión de contenido:** Panel para administradores donde se crean y organizan cursos, temas y retos.
- **Entorno de práctica:** Permite a los usuarios escribir y ejecutar fragmentos de código en un entorno controlado.
- **Gamificación:** Implementación de recompensas, logros para incentivar el aprendizaje continuo.

Inicialmente, se enfocará en los lenguajes **Python, HTML/CSS y SQL**, por su importancia introductoria en el desarrollo web y backend debido a su importancia introductoria.

Este sistema está orientado a estudiantes y busca ofrecer una experiencia educativa estructurada, divertida y accesible, que combine teoría, práctica y juego en una sola plataforma. Su diseño modular permite escalar el proyecto a futuro con nuevas funciones y lenguajes.

3. Abstract

ByteQuest is an educational platform under development, whose objective is to teach programming languages in a gamified and interactive way, based on the Duolingo methodology. It is designed using a client-server architecture, using modern web technologies.

The backend is developed in PHP with Object-Oriented Programming (OOP), which facilitates the modularity and maintenance of the system. A MySQL database is used for data storage, which manages users, progress, content, results, and statistics.

The frontend uses HTML5, CSS3, and pure JavaScript for the interface, with future integration with frameworks such as Bootstrap planned to improve the responsive experience and visual design.

The system has the following main modules:

User authentication: registration, login, and password recovery.

Progress management: tracking completed lessons, points, badges, lives, and levels.

Exercise module: interactive question bank, code exercises, and automatic validation.

Content Management: A dashboard for administrators where courses, topics, and challenges are created and organized.

Practice Environment: Allows users to write and run code snippets in a controlled environment.

Gamification: Implementation of rewards and achievements to encourage continuous learning.

Initially, the focus will be on Python, HTML/CSS, and SQL, due to their introductory importance in web and backend development.

This system is geared toward students and seeks to offer a structured, fun, and accessible educational experience that combines theory, practice, and gameplay in a single platform. Its modular design allows the project to scale in the future with new features and languages.

4. INTRODUCCIÓN

El aprendizaje en las personas es fundamental; sin embargo, suele enseñarse mediante métodos muy técnicos que pueden resultar abrumadores debido a la gran cantidad de información que se debe asimilar. Esto provoca que el proceso de aprendizaje no sea disfrutable. Por ello, se ha decidido desarrollar una aplicación web que incorpore gamificación, haciendo el proceso más entretenido y agradable, generando resultados de aprendizaje sin la sensación de frustración por no avanzar o aburrimiento por falta de retos.

5. PLANTEAMIENTO DEL PROBLEMA

En la actualidad, aprender a programar representa un desafío significativo para muchas personas, especialmente aquellas que se inician sin una base sólida en lógica computacional o experiencia previa en tecnologías digitales. La falta de motivación, métodos de enseñanza tradicionales poco atractivos y barreras técnicas hacen que muchos estudiantes abandonen sus intentos tempranos de adquirir habilidades en programación. A pesar del crecimiento de plataformas educativas, aún existe una necesidad latente de soluciones que combinen accesibilidad, interactividad y una metodología eficaz adaptada al ritmo del usuario.

ByteQuest surge como una aplicación educativa que busca gamificar el aprendizaje de programación mediante un enfoque interactivo basado en microlearning, similar a plataformas como Duolingo. Su objetivo es ofrecer una experiencia atractiva y accesible tanto para principiantes como para personas con conocimientos básicos, mediante funcionalidades como logros por usuario, una interfaz interactiva, registro personalizado y contenido sobre cinco lenguajes de programación.

Sin embargo, el desarrollo e implementación de ByteQuest enfrenta ciertos problemas clave: asegurar la compatibilidad con el hardware requerido, garantizar el cumplimiento de requisitos funcionales y no funcionales, y adaptar la plataforma a las necesidades reales del usuario. Estos desafíos técnicos y pedagógicos requieren una planificación cuidadosa y continua evolución para que la aplicación pueda satisfacer eficazmente su objetivo educativo y motivacional.

Por ello, se plantea la necesidad de una solución que permita a ByteQuest superar estas limitaciones técnicas y mejorar la experiencia de aprendizaje, asegurando que la enseñanza de la programación sea efectiva, dinámica y sostenible a largo plazo.

5.2. OBJETIVO GENERAL

Diseñar y desarrollar una aplicación web interactiva que facilite el aprendizaje de lenguajes de programación a través de lecciones estructuradas en formato de texto, complementadas con actividades evaluativas y dinámicas similares a juegos, fomentando una experiencia educativa motivadora y progresiva.

5.3. OBJETIVOS ESPECÍFICOS

Diseñar la arquitectura de la aplicación: según los principios de programación orientada a objetos (POO), cumpliendo con los estándares del desarrollo web moderno.

Crear módulos de lecciones: que presenten contenidos educativos en formato texto o video, enfocados en diversos lenguajes de programación.

Implementar un sistema de evaluación: que permita a los usuarios validar los conocimientos adquiridos mediante ejercicios interactivos.

Desarrollar funcionalidades de gamificación: como el uso de vidas, niveles o recompensas, para incentivar el progreso continuo del usuario.

Construir e integrar una base de datos relacional: en MySQL que permita gestionar usuarios, cursos, lecciones, y resultados de evaluación.

Diseñar una interfaz amigable y adaptable: utilizando tecnologías como Bootstrap, que facilite la navegación desde distintos dispositivos.

Evaluar el desempeño y funcionalidad de la aplicación: aplicando pruebas y recopilando retroalimentación de usuarios en contexto formativo.

5.4. JUSTIFICACIÓN DEL PROYECTO

En la actualidad, el aprendizaje de lenguajes de programación es una competencia esencial para acceder a oportunidades en el mundo tecnológico y digital. Sin embargo, muchas personas enfrentan dificultades al iniciar este proceso debido a la falta de recursos interactivos, métodos tradicionales poco motivadores y barreras técnicas que dificultan la comprensión de los conceptos.

Ante esta situación, surge la necesidad de diseñar herramientas innovadoras que promuevan el aprendizaje autodirigido de forma práctica, accesible y atractiva. *ByteQuest* es una propuesta que responde a esta necesidad, al ofrecer una plataforma web interactiva que permite a los usuarios aprender lenguajes de programación a través de lecciones en formato texto o video, complementadas con actividades evaluativas.

La aplicación incorpora dinámicas inspiradas en juegos, como el uso de vidas, niveles y retroalimentación inmediata, con el objetivo de aumentar la motivación y el compromiso de los usuarios. Esta metodología favorece la retención del conocimiento y la constancia en el proceso de formación.

Desde el punto de vista técnico, el desarrollo de esta plataforma también implica la aplicación de buenas prácticas en programación orientada a objetos, gestión de bases de datos, diseño de interfaces y desarrollo web moderno, lo cual aporta valor tanto al usuario final como al proceso de desarrollo.

En consecuencia, este proyecto busca facilitar el acceso al aprendizaje de la programación de manera didáctica y entretenida, aportando una herramienta útil tanto para principiantes como para personas que deseen reforzar sus conocimientos de forma progresiva y autónoma.

5.5. ALCANCE DEL PROYECTO

El presente proyecto comprende el desarrollo de una aplicación web denominada *ByteQuest*, orientada a la enseñanza de lenguajes de programación a través de un enfoque gamificado. La plataforma permitirá que los usuarios avancen en su proceso de aprendizaje mediante lecciones teóricas y prácticas, presentadas en formatos de texto y video, seguidas de actividades evaluativas que fomenten la retención de conocimientos.

Las funcionalidades incluidas en el alcance son:

- Registro e inicio de sesión de usuarios.
- Gestión de perfiles y seguimiento del progreso individual.
- Presentación de lecciones en formato multimedia (texto y video).
- Ejercicios y cuestionarios con retroalimentación automática.
- Sistema de puntos y logros como incentivo al aprendizaje.
- Almacenamiento y consulta de resultados para análisis de rendimiento.

Quedan fuera del alcance de esta versión:

- Desarrollo de una aplicación móvil nativa.
- Implementación de soporte multilenguaje.
- Integración con plataformas externas de certificación o acreditación.

5.6. MATRIZ DE RIESGO

Tabla 2

Nº	Riesgo	Probabilidad	Impacto	Nivel de riesgo	Plan de prevención/mitigación
1	Retrasos en el desarrollo por dificultades técnicas	Media	Alto	Alto	Realizar una planificación detallada, establecer hitos y asignar tareas de forma clara.
2	Fallos en la base de datos que provoquen pérdida de información	Baja	Alto	Medio	Implementar copias de seguridad periódicas y pruebas de recuperación.
3	Falta de conocimientos específicos en alguna tecnología utilizada	Media	Medio	Medio	Capacitación previa y consulta de documentación oficial.
4	Baja participación de usuarios en las pruebas	Alta	Medio	Alto	Incentivar la participación mediante recompensas o reconocimiento.
5	Problemas de compatibilidad en distintos navegadores o dispositivos	Media	Alto	Alto	Realizar pruebas de usabilidad y compatibilidad en diferentes entornos.
6	Ciberataques o accesos no autorizados a la aplicación	Baja	Alto	Medio	Implementar medidas de seguridad como cifrado y autenticación segura.

6. ELICITACIÓN DE REQUISITOS

En esta sección realizamos la elicitación de requisitos, que consiste en recopilar, analizar y documentar la información necesaria para definir las funcionalidades y características que debe tener el software, asegurándonos de que se ajusten a las necesidades y expectativas de los usuarios finales.

6.1. IDENTIFICACIÓN DE PROCESOS

El sistema *Byte Quest* se centrará en el proceso de enseñanza-aprendizaje de lenguajes de programación mediante una plataforma web gamificada. Los procesos identificados son:

- Gestión de usuarios (registro, inicio de sesión, edición de perfil).
- Gestión de contenido educativo (creación, edición y visualización de lecciones).
- Evaluación de conocimientos (cuestionarios y ejercicios interactivos).
- Seguimiento del progreso del usuario (historial de avances, logros y puntajes).
- Administración de la plataforma (gestión de cursos, usuarios y reportes).

6.2. RECOLECCIÓN DE LA INFORMACIÓN DEL SOFTWARE A CONSTRUIR DE ACUERDO CON LAS NECESIDADES DEL CLIENTE.

Para comprender las necesidades de los usuarios, se realizó un levantamiento de información con el objetivo de definir claramente las funcionalidades y restricciones del sistema.

6.2.1. ELECCIÓN DE LA TÉCNICA DE RECOLECCIÓN DE LA INFORMACIÓN.

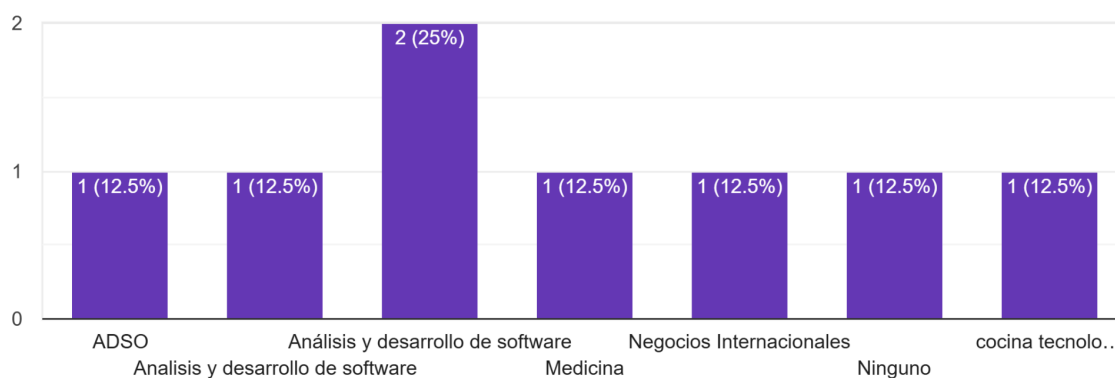
Se seleccionó la encuesta estructurada como técnica principal, combinando preguntas cerradas para obtener datos cuantitativos y preguntas abiertas para captar opiniones y sugerencias cualitativas. Además, se complementó con la observación directa de plataformas similares para identificar buenas prácticas y funcionalidades relevantes.

6.2.2. DISEÑO DE LOS FORMATOS, SEGÚN LA TÉCNICA O TÉCNICAS DE RECOLECCIÓN DE LA INFORMACIÓN SELECCIONADA(S).

El formato de la encuesta fue diseñado para cubrir aspectos clave como la experiencia previa en programación, preferencias de aprendizaje, funcionalidades deseadas en la plataforma y expectativas sobre la gamificación. Se incluyeron preguntas de opción múltiple, escala Likert y espacios para comentarios libres, buscando obtener una visión integral de las necesidades del usuario.

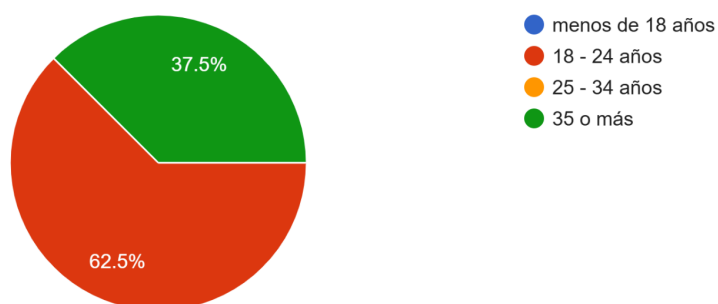
¿Que programa de formacion esta estudiando?

8 respuestas



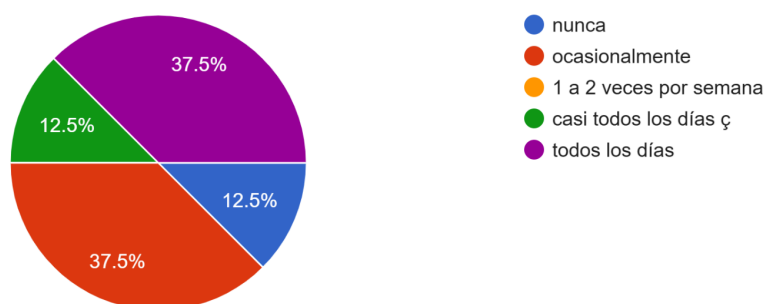
¿Cuál es su rango de edad?

8 respuestas



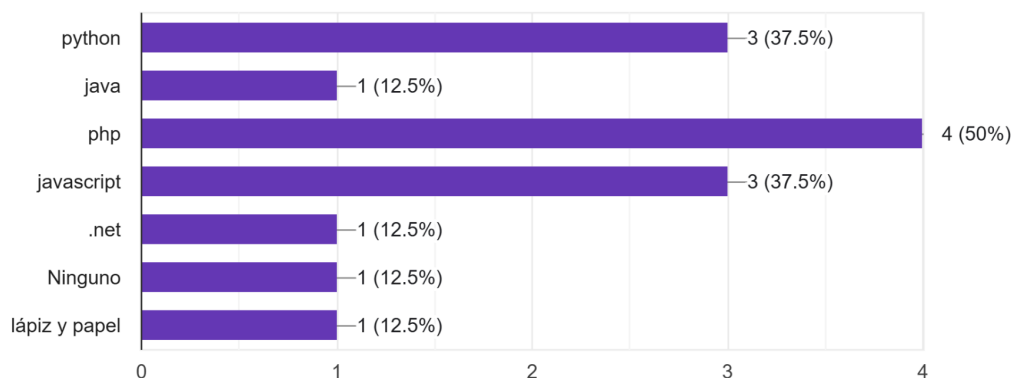
con que frecuencia utiliza aplicaciones o plataformas educativas en linea

8 respuestas



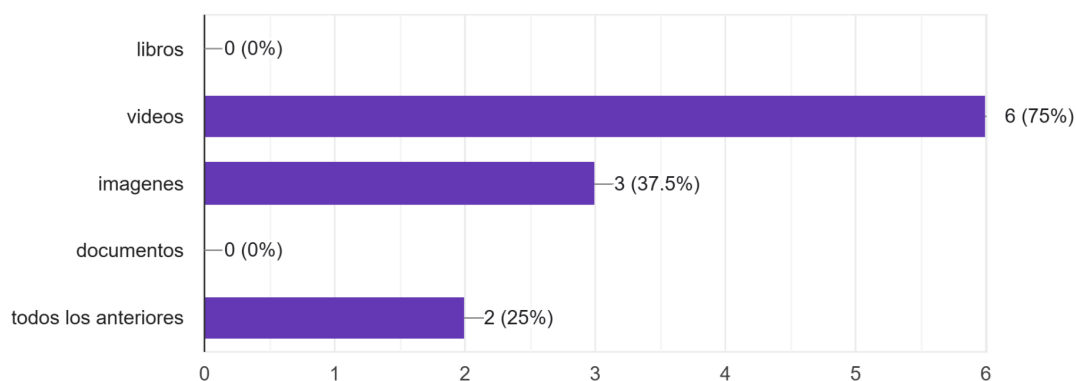
que lenguajes de programacion sabe

8 respuestas



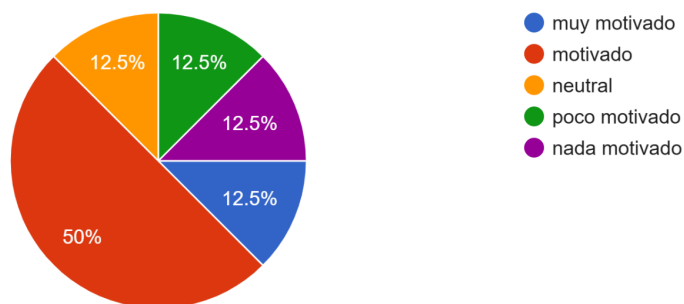
¿Qué tipo de contenidos le resultan mas útiles para aprender programación?

8 respuestas



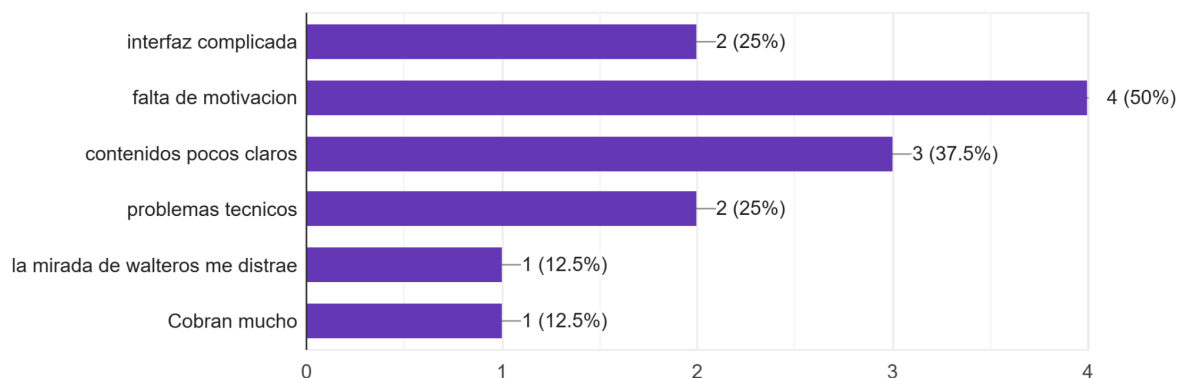
¿Qué tan motivado se siente al aprender cuando una plataforma incluye elementos de juegos?

8 respuestas



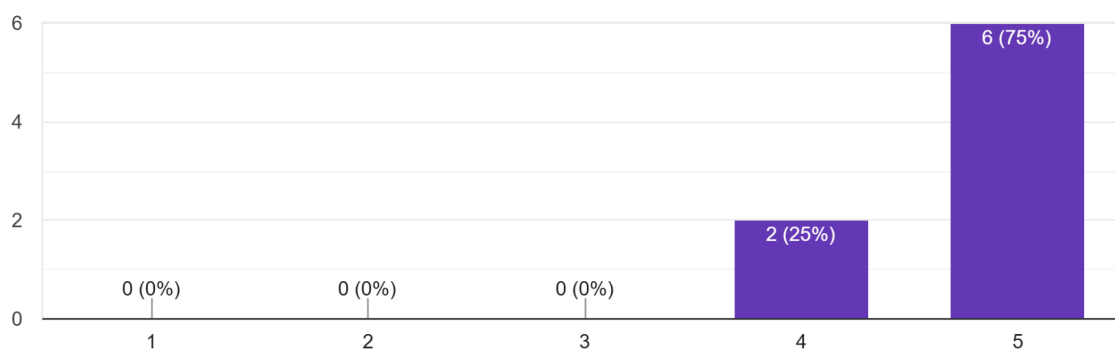
que dificultades encuentra al usar plataformas de aprendizaje actuales

8 respuestas



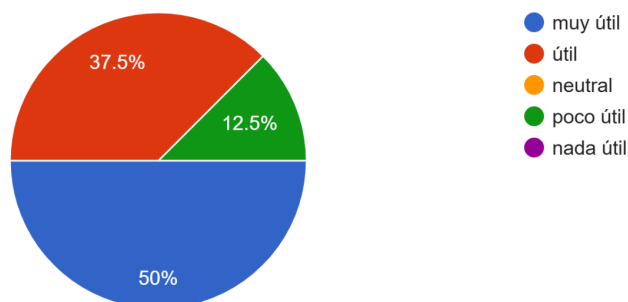
en una escala de 1 a 5 ¿que tan importante es que la plataforma tenga una interfaz facil de usar?

8 respuestas



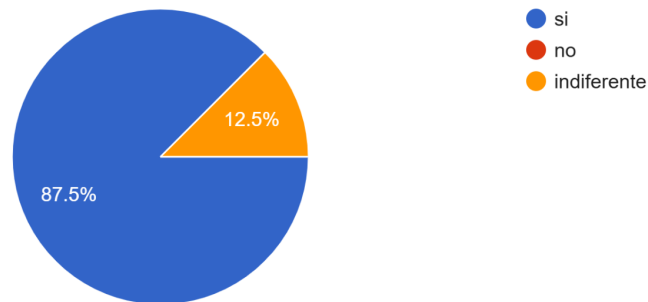
que tan útil le resultaria contar con un sistema de retroalimentacion inmediata al responder preguntas o realizar ejercicios

8 respuestas



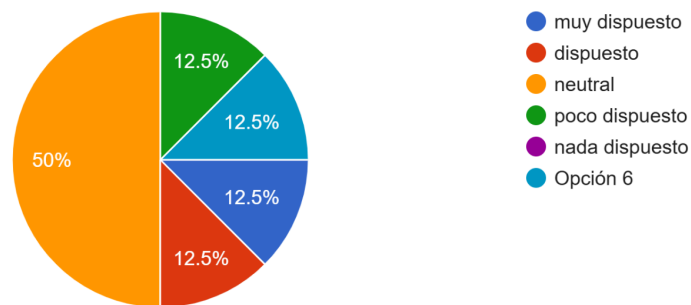
¿Prefiere que la plataforma tenga un sistema de progreso visual?

8 respuestas



¿Que tan dispuesta estaría a utilizar una tienda interna para obtener vidas o recursos adicionales dentro de la plataforma?

8 respuestas



6.2.3. APLICACIÓN DE LA TÉCNICA DE RECOLECCIÓN DE LA INFORMACIÓN.

La encuesta fue aplicada de forma virtual a un grupo representativo de estudiantes de programación. La recolección se realizó garantizando la confidencialidad y anonimato de los participantes. Los resultados obtenidos permitieron identificar las funcionalidades prioritarias y las restricciones técnicas que deben considerarse en el desarrollo de ByteQuest.

6.2.4. ORGANIZACIÓN DE LA INFORMACIÓN RECOLECTADA

La información obtenida a través de la encuesta fue clasificada y organizada en tres categorías principales para facilitar su análisis y aplicación en el desarrollo del software:

- Funcionalidades esenciales: Requisitos indispensables para el correcto funcionamiento de la plataforma, identificados como prioritarios por la mayoría de los encuestados.
- Funcionalidades complementarias: Características que mejoran la experiencia del usuario pero que no son críticas para la operación básica del sistema.
- Restricciones técnicas y normativas: Limitaciones relacionadas con aspectos técnicos, de seguridad y cumplimiento legal que deben ser consideradas durante el desarrollo.

Esta organización permitió priorizar las necesidades del cliente y usuarios, asegurando que el diseño y desarrollo de ByteQuest respondan efectivamente a las expectativas y requerimientos identificados.

6.3. ESPECIFICACIÓN DE REQUERIMIENTOS, DE ACUERDO CON LA INFORMACIÓN RECOLECTADA, APLICANDO ESTÁNDARES DE REQUERIMIENTOS

Con base en la información recolectada, se especifican los requerimientos del sistema aplicando estándares reconocidos, con el fin de garantizar que las funcionalidades y características definidas respondan adecuadamente a las necesidades del cliente y usuarios, facilitando un desarrollo ordenado y eficiente del software.

6.3.1. REQUERIMIENTOS FUNCIONALES

Número de requisito	RF 001
Nombre de requisito	Login
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	usuario ingresa al aplicativo
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 002
Nombre de requisito	Registro
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	usuario se registra en el aplicativo
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 003
Nombre de requisito	Gestión de perfiles de usuario
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	usuario gestiona su propio perfil
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 004
Nombre de requisito	Catalogo de cursos
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	regulación de cursos mostrando cuales hay disponibles
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 005
Nombre de requisito	Pruebas y evaluaciones
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	regulación de pruebas y evaluaciones por parte del usuario administrador
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 006
Nombre de requisito	Sistema de niveles y progreso
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	regulación de sistema de niveles y progreso por medio de un porcentaje
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 007
Nombre de requisito	Administración de contenidos
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	usuario administrador puede gestionar los contenidos de cada curso
Prioridad del requisito	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/ Opcional

Número de requisito	RF 008
Nombre de requisito	Historial de actividades
Tipo	☒ Requisito ☐ Restricción
Fuente del requisito	usuario puede ver las pruebas y evaluaciones por medio de un historial
Prioridad del requisito	☐ Alta/Esencial ☒ Media/Deseado ☐ Baja/ Opcional

6.3.2. REQUERIMIENTOS NO FUNCIONALES

- RNF 1: La interfaz debe ser intuitiva y fácil de usar.
- RNF 2: El tiempo de respuesta de las acciones no debe superar los 2 segundos.
- RNF 3: La plataforma debe ser accesible desde los navegadores más comunes (Chrome, Firefox, Edge).
- RNF 4: La base de datos debe garantizar integridad y disponibilidad de la información.
- RNF 5: El sistema debe garantizar la protección de los datos personales y la privacidad de los usuarios.
- RNF 6: El sistema debe ser capaz de crecer y manejar un incremento en número de usuarios y volumen de datos sin degradar su rendimiento.
-

6.3.3. REQUERIMIENTOS NORMATIVOS

- Cumplir con la Ley 1581 de 2012 sobre protección de datos personales (Colombia).
- Cumplir con estándares de accesibilidad web WCAG 2.1 en nivel AA.

6.3.4. PROPUESTA TÉCNICA

La solución se desarrollará como una aplicación web utilizando:

- **Backend:** PHP con framework **Laravel**.
- **Frontend:** HTML5, CSS3, JavaScript, Bootstrap.
- **Base de Datos:** MySQL.
- **Servidor de Desarrollo:** XAMPP.
- **Alojamiento en Producción:** Hosting en **Railway**.
- **Gestión de versiones:** GitHub.
- **Metodología de Desarrollo:** Ágil (Scrum) para iteraciones y entregas parciales.

7. REGLAS DEL NEGOCIO

1. Registro único por usuario:

Cada usuario debe registrarse con un correo electrónico único y válido. No se permiten cuentas duplicadas.

2. Progreso individual:

El avance y los puntos acumulados se registran de forma individual y no son transferibles entre cuentas.

3. Desbloqueo de lecciones:

Una lección solo se desbloquea cuando el usuario ha completado y aprobado la anterior con un puntaje mínimo del **70%**.

4. Retroalimentación inmediata:

Al finalizar cada cuestionario o actividad, el sistema debe mostrar al usuario su calificación y las respuestas correctas/incorrectas.

5. Sistema de recompensas:

Los usuarios acumularán puntos y logros según su desempeño; estos no pueden ser canjeados por dinero u otros bienes fuera de la plataforma.

6. Protección de datos:

El tratamiento de la información personal debe cumplir con la Ley 1581 de 2012 y su Decreto Reglamentario 1377 de 2013 en Colombia.

7. Roles y permisos:

Solo los administradores podrán crear, modificar o eliminar cursos, lecciones y preguntas.

8. Disponibilidad del servicio:

El sistema debe estar disponible las 24 horas del día, salvo interrupciones por mantenimiento programado.

9. Sistema de vidas:

Cada usuario dispondrá de **5 vidas** para realizar actividades o cuestionarios.

- Por cada respuesta incorrecta, perderá una vida.
- Cuando las vidas lleguen a cero, el usuario deberá esperar **24 horas** o completar una tarea especial para recuperarlas.

10. Tienda virtual:

Los usuarios podrán utilizar los puntos o monedas acumuladas para adquirir beneficios dentro de la plataforma, tales como:

- Recuperar vidas.
- Desbloquear contenido adicional.
- Personalizar su avatar o perfil.

Las compras realizadas en la tienda son definitivas y no se admiten reembolsos.

8. ANÁLISIS DE LA ESPECIFICACIÓN DE REQUISITOS DEL SOFTWARE.

En esta etapa se realiza un análisis de la especificación de requisitos para verificar su coherencia, viabilidad y completitud, con el objetivo de asegurar que los requisitos están claros y sean adecuados para guiar el desarrollo del software.

REQUISITOS FUNCIONALES

Caso de Prueba	Descripción	Esperado	Obtenido
RF1-01	Registro con datos válidos	Registro exitoso	
RF1-02	Registro con correo inválido	Mensaje de error	
RF1-03	Inicio de sesión correcto	Acceso al sistema	
RF1-04	Inicio con credenciales erróneas	Error de autenticación	
RF2-01	Visualizar lección en texto	Texto legible sin errores	
RF2-02	Visualizar lección en video	Video se reproduce correctamente	
RF3-01	Completar cuestionario válido	Retroalimentación inmediata	
RF3-02	Intentar enviar sin responder	Mensaje de advertencia	
RF4-01	Mostrar puntaje final	Puntaje correcto en pantalla	
RF4-02	Mostrar retroalimentación	Comentarios correctos por pregunta	
RF5-01	Ver progreso acumulado	Barra de progreso actualizada	
RF5-02	Historial de actividades	Lista de actividades completadas	
RF6-01	Crear nueva lección (admin)	Lección almacenada en BD	
RF6-02	Eliminar cuestionario (admin)	Eliminación exitosa	

RF7-01	Consultar historial detallado	Información completa mostrada	
RF7-02	Exportar historial	Archivo descargable	

REQUISITOS NO FUNCIONALES

Interoperabilidad:

API Interna: Considerar el desarrollo de una API interna para facilitar la integración futura con otros servicios internos o externos.

Seguridad: Autenticación Multifactor (MFA): Implementar autenticación multifactor para mejorar la seguridad del inicio de sesión.

Rendimiento: Optimización de Recursos: Considerar la optimización de recursos del servidor y la implementación de técnicas de balanceo de carga para manejar altos volúmenes de tráfico.

Usabilidad: Pruebas de Usabilidad: Realizar pruebas de usabilidad con usuarios reales para identificar y corregir problemas de navegación o interacción.

Disponibilidad: Plan de Recuperación de Desastres: Establecer un plan para garantizar la recuperación rápida en caso de fallos graves del sistema.

Escalabilidad: Arquitectura en la Nube: Explorar la posibilidad de usar servicios en la nube que escalan automáticamente según la demanda.

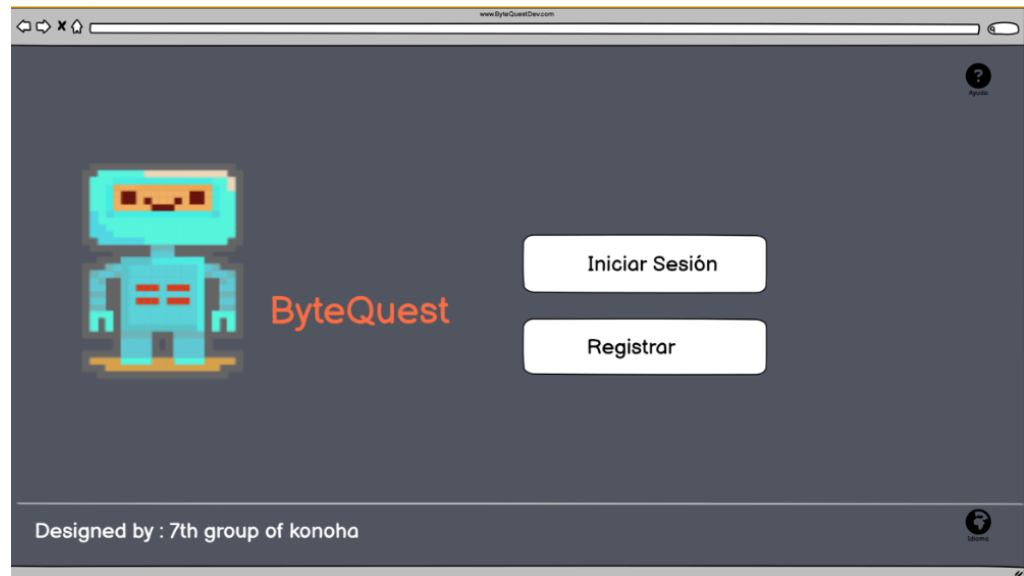
Mantenibilidad: Documentación y Comentarios: Asegurarse de que el código esté bien documentado y comentado para facilitar la mantenibilidad.

Compatibilidad: Pruebas en Navegadores y Dispositivos: Realizar pruebas exhaustivas en diferentes navegadores y dispositivos para garantizar una experiencia de usuario consistente.

8.1. ALTERNATIVAS DE SOLUCIÓN (PROTOTIPO O PROTOTIPOS DEL

SISTEMA, MOCKUPS)

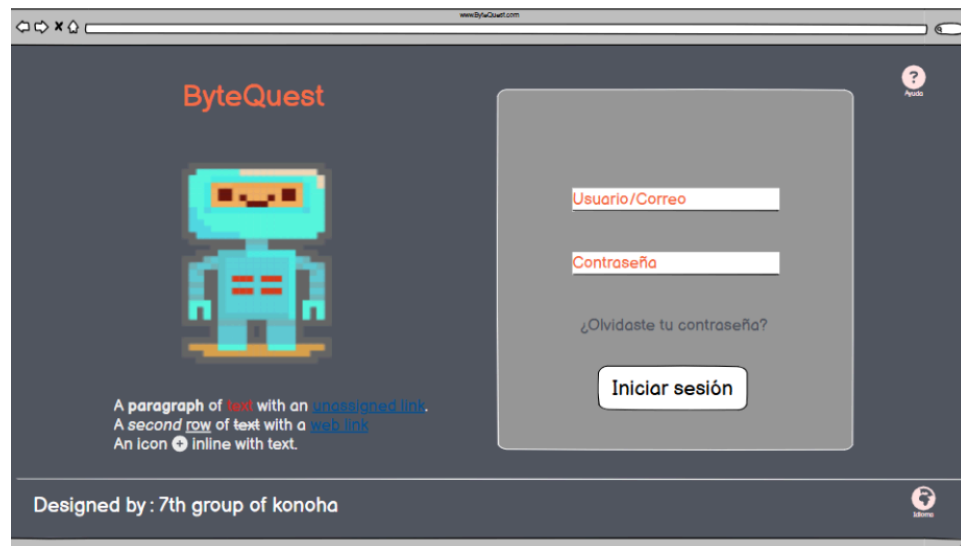
Landing



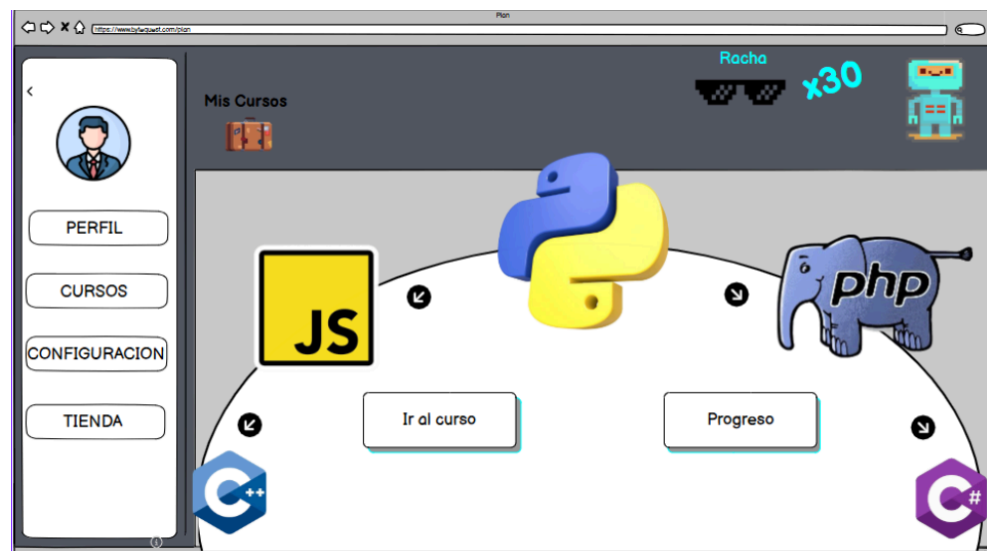
Registro

A screenshot of the ByteQuest registration form. The browser's address bar shows the URL "www.ByteQuestDev.com". The page has a dark gray background. On the left, there is a pixel art robot character with a blue body and orange head. To its right, the text "ByteQuest" is displayed in a stylized orange font. In the center, there is a light gray rectangular box with the title "Crea tu perfil" (Create your profile). Inside this box, there are three input fields: "Correo" (Email), "Nombre" (Name), and "Contraseña" (Password). The "Contraseña" field has a small eye icon to its right. Below the input fields, there is a white button with black text labeled "Registrar" (Register). In the top right corner, there is a small circular icon with a question mark and the word "Ayuda" (Help) below it. In the bottom right corner, there is a small circular icon with a globe and the word "Idioma" (Language) below it.

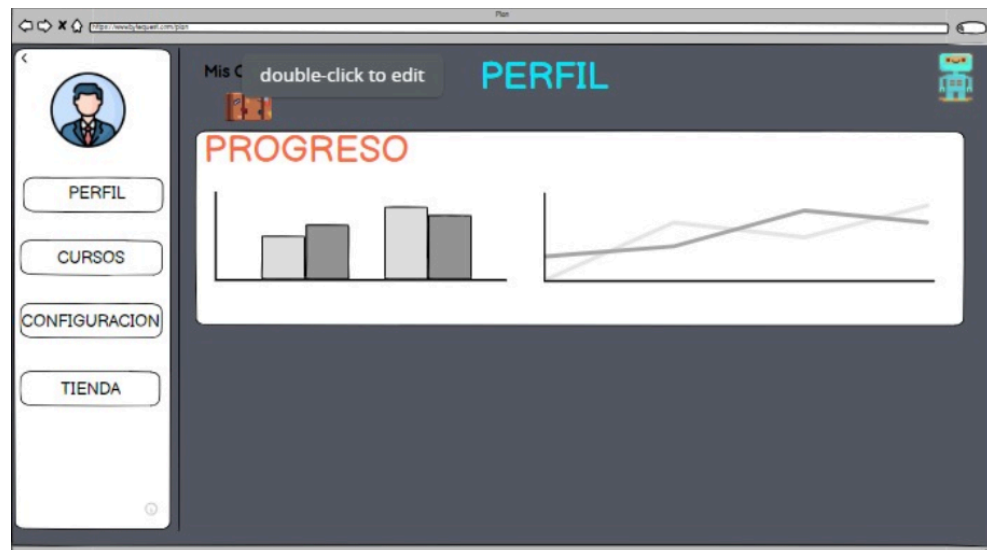
Login



Home



Perfil



Aprendizaje



Lección



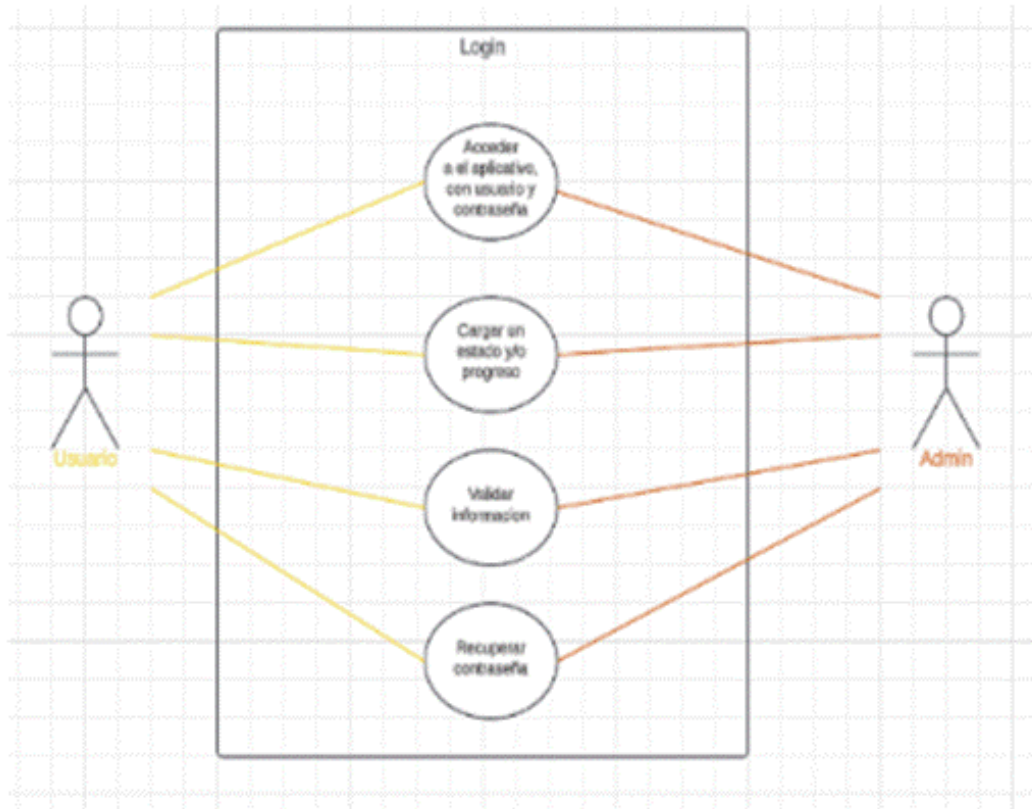
8.2. DIAGRAMA DE CASOS DE USO Y EXTENSIBILIDAD DE ESTOS

1. Login

Historia de Usuario: Como usuario registrado, quiero iniciar sesión en la plataforma usando mi nombre de usuario y contraseña, para que pueda acceder a mi cuenta y comenzar a interactuar con los cursos y contenidos disponibles.

Criterios de Aceptación:

- El sistema debe permitir al usuario ingresar un nombre de usuario y una contraseña.
- El sistema debe validar las credenciales y proporcionar acceso si son correctas.
- En caso de error en las credenciales, el sistema debe mostrar un mensaje adecuado.
- El sistema debe ofrecer una opción para recuperar la contraseña en caso de olvido.

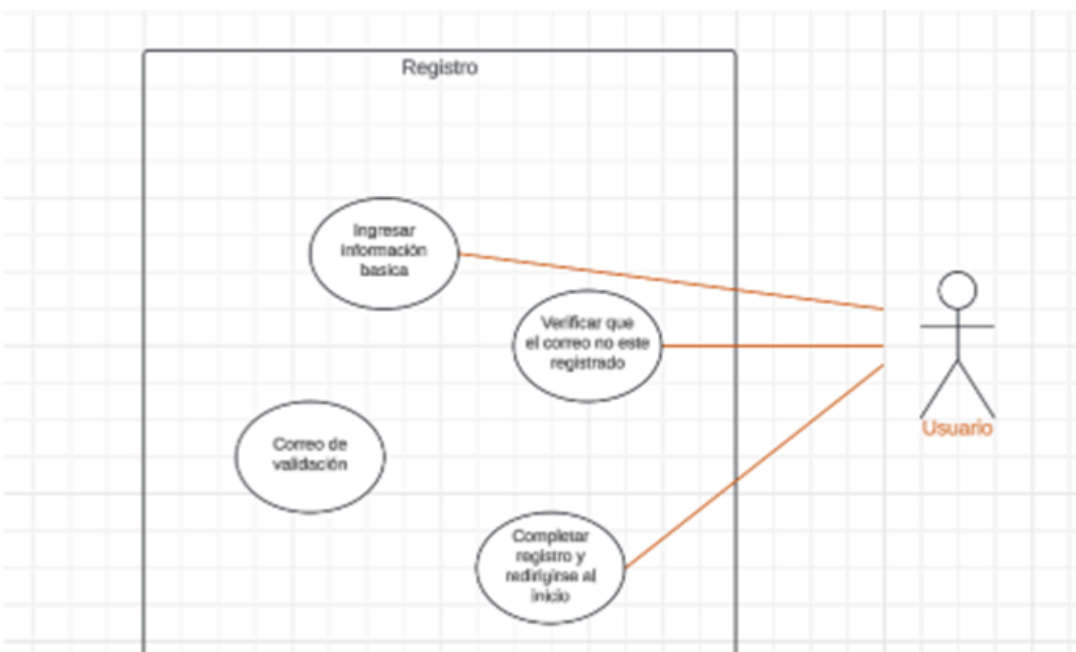


2. Registro

Historia de Usuario: Como nuevo usuario, quiero registrarme en la plataforma proporcionando mi información personal y creando una cuenta, para que pueda empezar a explorar y matricularse en los cursos disponibles.

Criterios de Aceptación:

- El sistema debe permitir al usuario ingresar información personal (nombre, correo electrónico, etc.) y crear una contraseña.
- El sistema debe verificar que la dirección de correo electrónico no esté ya registrada.
- El sistema debe enviar un correo electrónico de confirmación para verificar la cuenta.
- El usuario debe poder completar el registro y ser redirigido a su perfil.



3. Gestión de Perfiles de Usuario

Historia de Usuario: Como usuario, quiero editar mi perfil para actualizar mi información personal y mis preferencias, para que mi cuenta esté actualizada y refleje mis datos correctos y mis intereses.

Criterios de Aceptación:

- El usuario debe poder acceder a su perfil desde un menú de configuración.
- El sistema debe permitir al usuario actualizar información personal como nombre, foto de perfil, y preferencias de notificación.
- Los cambios realizados en el perfil deben guardarse y reflejarse de inmediato.
- El usuario debe recibir una confirmación de que los cambios se han guardado correctamente.

4. Catálogo de Cursos

Historia de Usuario: Como usuario, quiero explorar el catálogo de cursos disponibles en la plataforma, para que pueda elegir los cursos que más me interesan y matricularme en ellos.

Criterios de Aceptación:

- El sistema debe mostrar una lista de cursos disponibles con detalles básicos (título, descripción, instructor).
- El usuario debe poder filtrar y buscar cursos por categorías, nivel de dificultad, o palabra clave.
- El usuario debe poder ver información detallada de cada curso al hacer clic en él.
- El usuario debe poder matricularse en un curso desde la vista del catálogo.

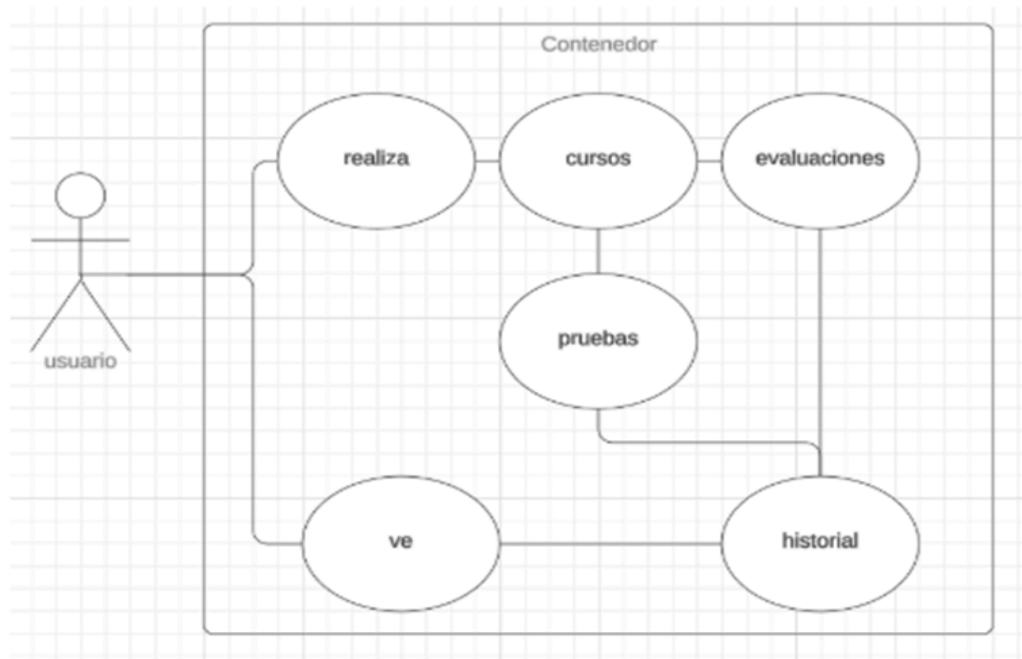
5. Pruebas y Evaluaciones

Historia de Usuario: Como estudiante, quiero realizar pruebas y evaluaciones dentro de un curso, para poder demostrar mi comprensión del material y recibir retroalimentación sobre mi rendimiento.

Criterios de Aceptación:

- El sistema debe permitir al usuario acceder a exámenes y evaluaciones desde el curso correspondiente.
- Las pruebas deben presentar preguntas en diferentes formatos (opción múltiple, verdadero/falso, respuestas cortas).

- El sistema debe permitir al usuario enviar sus respuestas y recibir retroalimentación inmediata o después de completar el examen.
- El sistema debe registrar los resultados de las pruebas y mostrar el desempeño del usuario.

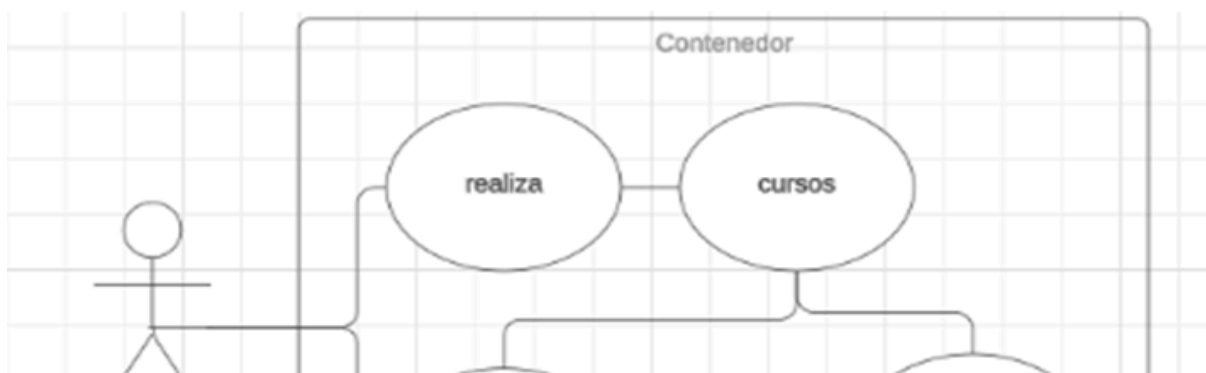


6. Sistema de Niveles y Progresión

Historia de Usuario: Como estudiante, quiero ver mi progreso y los niveles alcanzados en los cursos en los que estoy inscrito, para que pueda monitorear mi avance y motivarme a completar el curso.

Criterios de Aceptación:

- El sistema debe mostrar una visualización del progreso en cada curso (porcentaje completado, niveles alcanzados).
- El usuario debe poder ver un resumen de su avance y las tareas completadas.
- El sistema debe actualizar el progreso en tiempo real a medida que el usuario completa actividades y evaluaciones.
- El usuario debe recibir notificaciones sobre su progreso y logros alcanzados.



7. Administración de Contenidos

Historia de Usuario: Como administrador, quiero gestionar el contenido de los cursos, incluyendo la creación, edición y eliminación de materiales, para que pueda mantener la plataforma actualizada y ofrecer contenido relevante y de calidad.

Criterios de Aceptación:

- El sistema debe permitir al administrador crear y editar contenido del curso (módulos, lecciones, materiales de lectura).
- El sistema debe permitir la carga y gestión de archivos y recursos multimedia.
- El administrador debe poder eliminar contenido obsoleto o incorrecto.
- Los cambios en el contenido deben ser reflejados inmediatamente en la plataforma para los usuarios.



8. Historial de Actividades

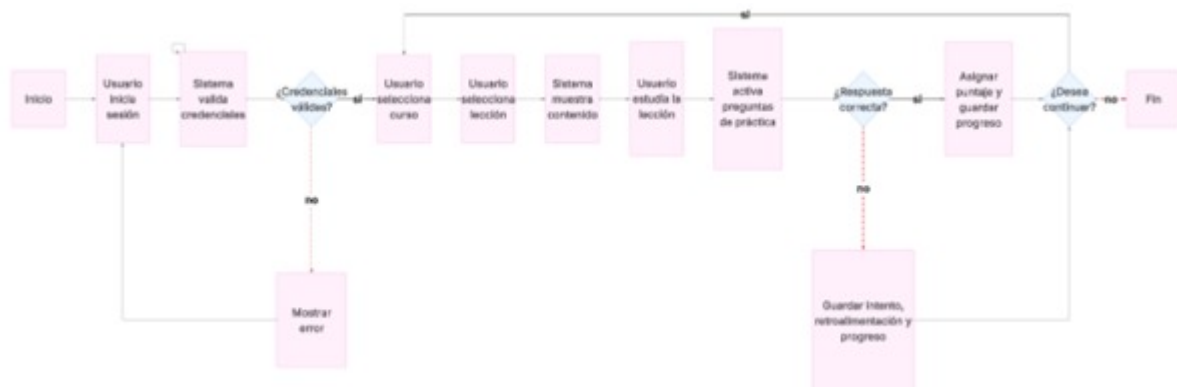
Historia de Usuario: Como usuario, quiero ver un historial de mis actividades en la plataforma, incluyendo los cursos que he tomado, las evaluaciones realizadas, y los progresos alcanzados, para que pueda revisar y reflexionar sobre mi desempeño y participación.

Criterios de Aceptación:

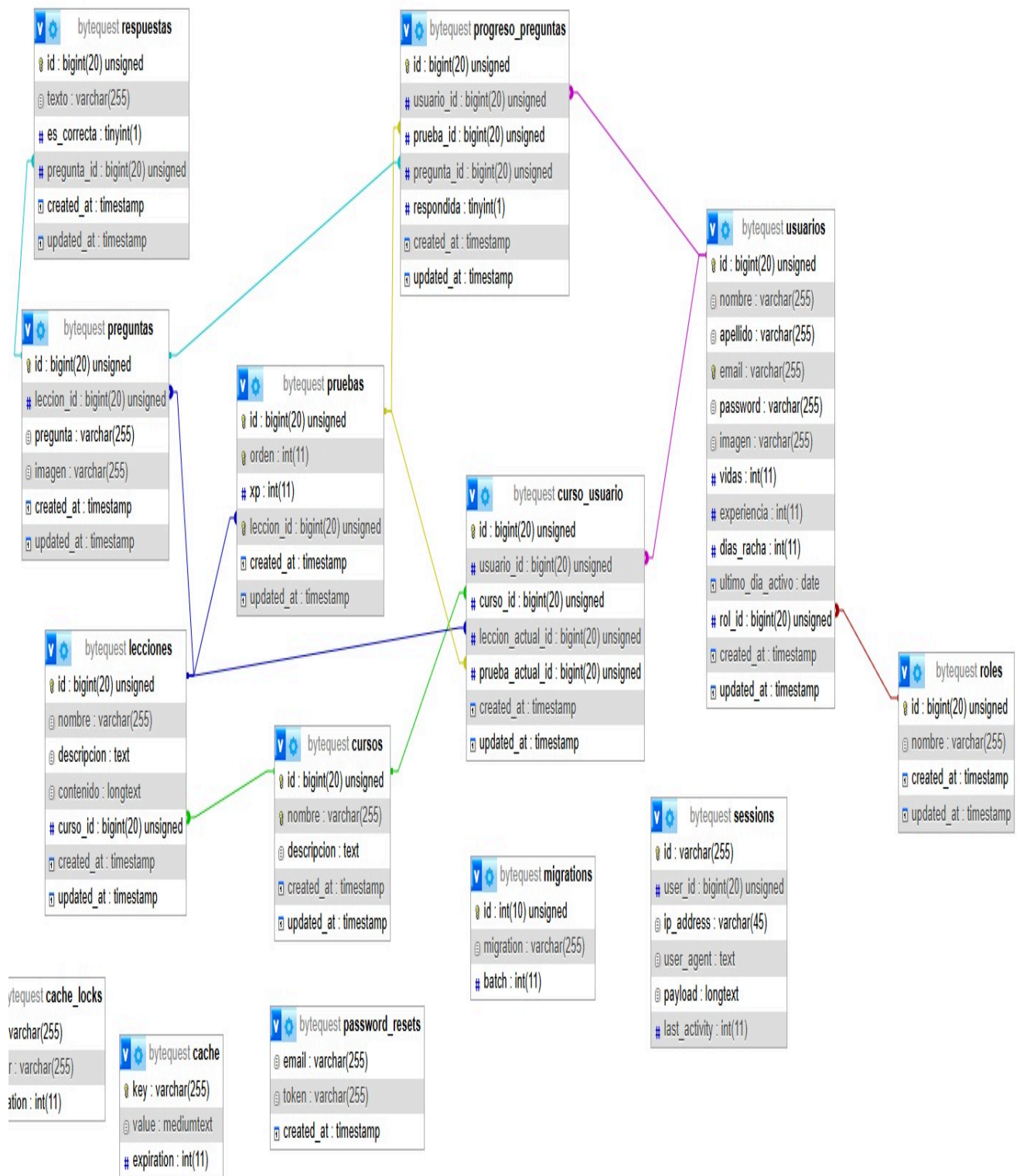
- El sistema debe mostrar un registro detallado de todas las actividades del usuario, como cursos inscritos, exámenes completados, y logros obtenidos.
- El historial debe ser accesible desde el perfil del usuario.
- El usuario debe poder filtrar y buscar actividades por tipo y fecha.
- El sistema debe actualizar el historial en tiempo real a medida que el usuario realiza nuevas actividades.

8.3. DIAGRAMAS DE ACTIVIDADES, Y SECUENCIAS

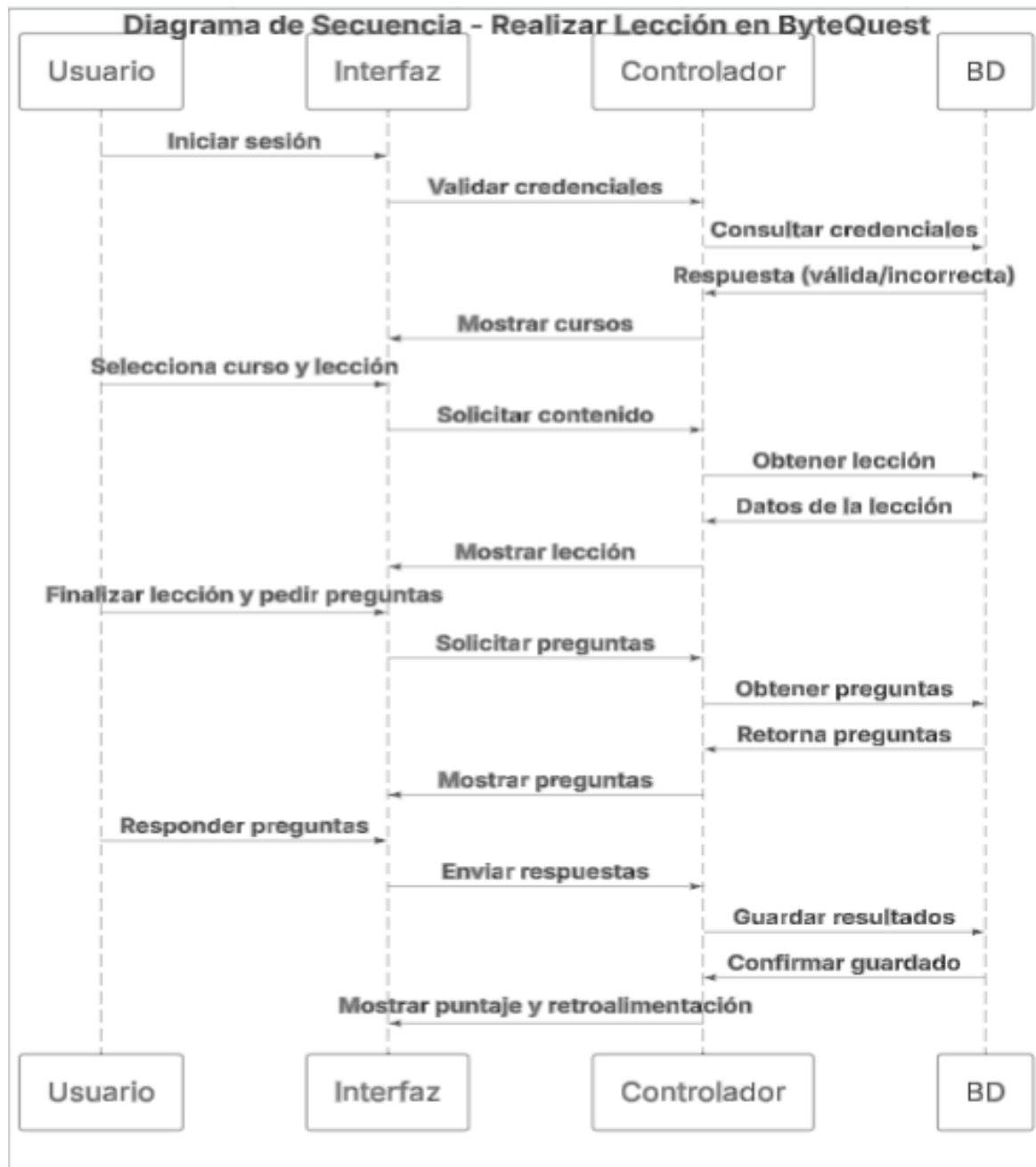
figura:Diagrama de actividades



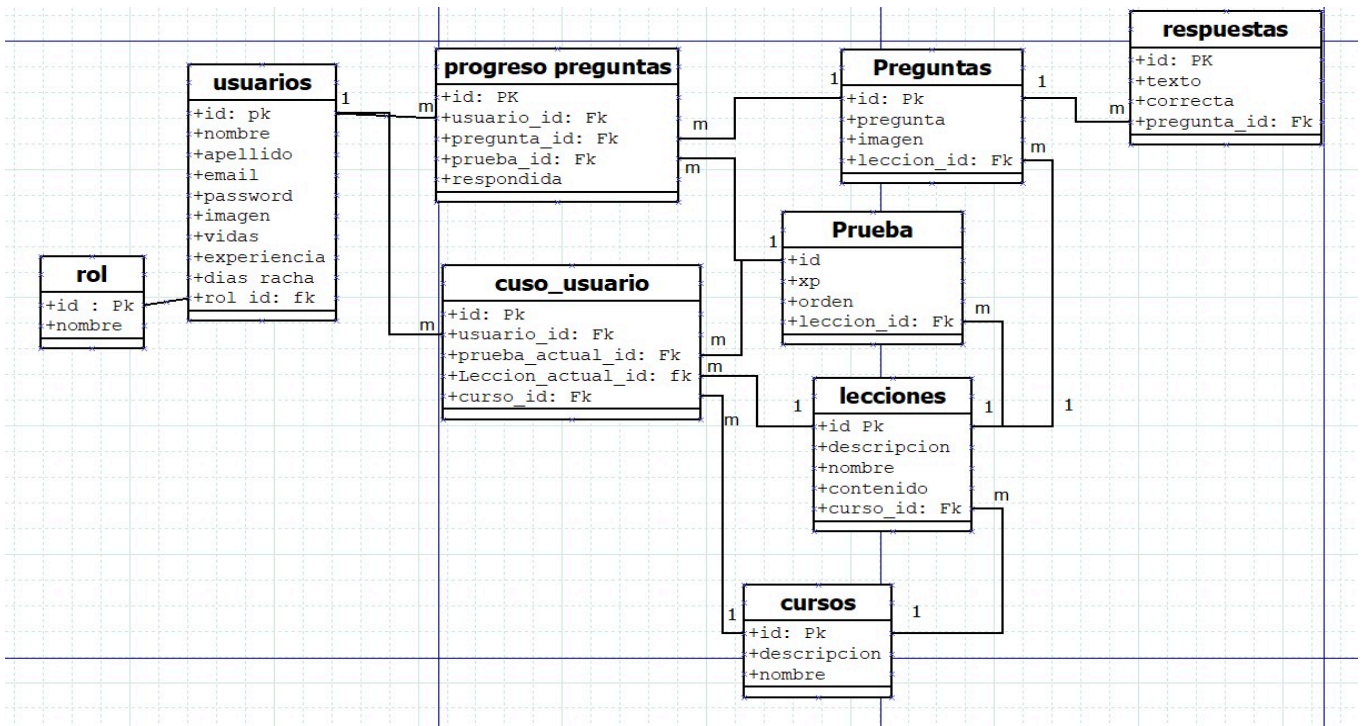
8.4. CONSTRUIR EL MODELO DE DOMINIO DEL SISTEMA (DIAGRAMA DE CLASES).



8.4 DIAGRAMA DE SECUENCIA



8.5. ELABORAR EL MODELO ENTIDAD RELACIÓN.



9. DISEÑO DE LA SOLUCIÓN DEL SOFTWARE DE ACUERDO CON LOS PROCEDIMIENTOS Y REQUISITOS TÉCNICOS

El diseño **de la solución** para la plataforma **ByteQuest**, alineado con los procedimientos y requisitos técnicos. Contiene la arquitectura propuesta, componentes funcionales, modelo de datos, especificaciones de API, requisitos no funcionales, procedimientos de despliegue y pruebas, y recomendaciones de seguridad y monitoreo.

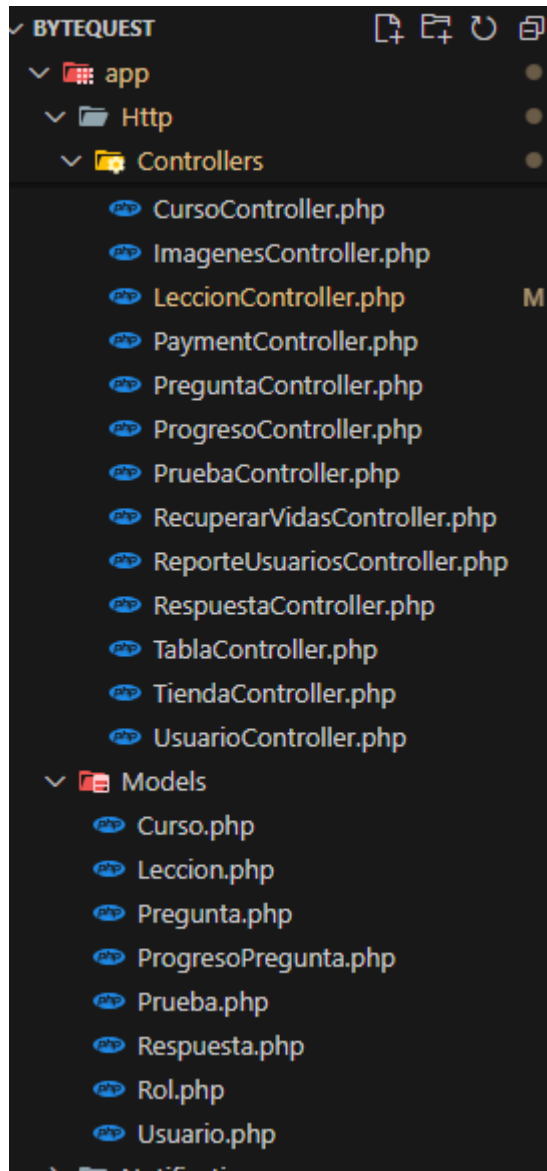
Resumen ejecutivo

BitQuest es una plataforma web para aprender lenguajes de programación con lecciones en texto, evaluaciones por lección y elementos de gamificación (vidas, puntos, logros). El objetivo del diseño es garantizar escalabilidad, mantenibilidad y experiencia de usuario responsiva.

1.Requisitos técnicos

1.1Requisitos funcionales principales

- Gestión de usuarios (registro, login, perfiles).
- Gestión de cursos, módulos y lecciones (texto/video).
- Evaluaciones: preguntas con respuestas y almacenamiento de resultados.
- Sistema de vidas y puntaje.
- Panel administrativo para CRUD de contenidos.
- Reportes básicos (avance, puntuaciones).



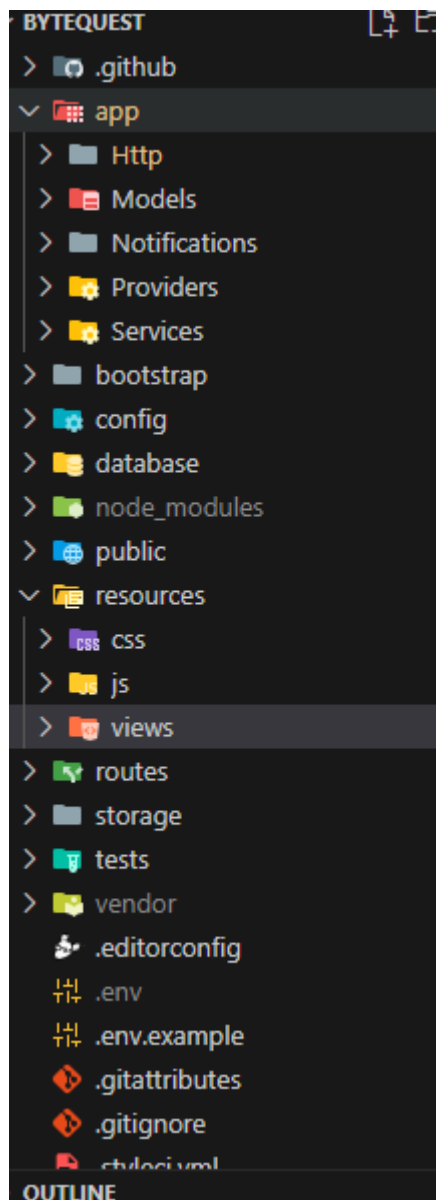
1.2 Requisitos no funcionales

- Rendimiento: tiempos de respuesta < 300–500 ms para operaciones comunes.
- Escalabilidad: permitir crecimiento horizontal del backend y almacenamiento de activos en CDN.
- Seguridad: autenticación segura, cifrado de contraseñas, protección contra XSS/CSRF.
- Disponibilidad: SLA objetivo 99.5%.
- Compatibilidad: responsive (Bootstrap) para móviles y desktop.

2. Arquitectura general

2.1 Diagrama de capas

- **Cliente (Frontend):** HTML5, Bootstrap, JavaScript (vanilla o librería ligera).
- **Servidor Web / Backend:** PHP con Laravel (MVC, API RESTful y vistas Blade).
- **Base de datos:** MySQL (relacional).
- **Almacenamiento de archivos:** Storage local en desarrollo; S3 / MinIO en producción + CDN opcional.
- **Servicios auxiliares:** Redis para cache y sesiones, queue worker (Laravel Queue) para tareas asíncronas.
- **CI/CD:** GitHub Actions para pruebas y despliegue.
- **Monitoreo:** Prometheus / Grafana o servicios SaaS (Plausible, Sentry para errores).



3. Patrón arquitectónico

- **Monolito modular** (Laravel) con separación por módulos: Auth, Cursos, Lecciones, Evaluaciones, Gamificación, Admin.
- Posible migración a microservicios si crece la complejidad (por ejemplo, servicio de medios separado).

4. Diseño detallado de componentes

4.1. Frontend

- **Estructura:** Layout principal con Navbar, Sidebar (panel admin) y content area.
- **Framework UI:** Bootstrap 5 para componentes y grid.
- **Páginas clave:** Landing, Registro/Login, Dashboard usuario, Página curso, Lección, Evaluación, Perfil, Panel admin.
- **Interacciones JS:** Validaciones client-side, consumo de API para datos dinámicos (fetch/axios), manejo de notificaciones y animaciones ligeras.

```

resources > css > login.css > ...
1  @import url(https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap);
2  html {
3      height: 100%;
4  }
5
6  body {
7      font-family: Montserrat, Arial, sans-serif !important;
8      margin: 0;
9  }
10
11  .bg {
12      animation: slide 3s ease-in-out infinite alternate;
13      background-image: linear-gradient(-60deg, #00b2c3 50%, #252746 50%);
14      bottom: 0;
15      left: -50%;
16      opacity: 0.5;
17      position: fixed;
18      right: -50%;
19      top: 0;
20      z-index: -1;
21  }
22
23  .bg2 {
24      animation-direction: alternate-reverse;
25      animation-duration: 4s;
26  }
27
28  .bg3 {
29      animation-duration: 5s;
30  }
31
32  .content {

```

4.2 Backend (Laravel)

- **Controladores:** AuthController, CursoController, LeccionController, EvaluacionController, GamificationController, AdminController.
- **Modelos y relaciones:**
 - User (hasMany Progreso, hasMany Resultados)
 - Curso (hasMany Leccion)
 - Leccion (belongsTo Curso, hasMany Pregunta)
 - Pregunta (belongsTo Leccion, hasMany Respuesta)

- Resultado (belongsTo User, belongsTo Leccion)
- Progreso / Vida (modelo para vida por usuario o campo en User)
- **Servicios:** capa de servicios para lógica de gamificación (calcular puntos, restar vidas), manejo de subida de videos.
- **Queues:** procesamiento de transcodificación (si aplica), envío de emails, generación de reportes.

```

9
10 class PreguntaController extends Controller
11 {
12     public function index(Request $request)
13     {
14         $preguntas = \App\Models\Pregunta::with(['leccion.curso']);
15
16         if ($request->filled('search')) {
17             $search = $request->search;
18             $preguntas->where(function ($query) use ($search) {
19                 $query->where('pregunta', 'like', '%' . $search . '%')
20                     ->orWhereHas('leccion', function ($q) use ($search) {
21                         $q->where('nombre', 'like', '%' . $search . '%')
22                             ->orWhereHas('curso', function ($qc) use ($search) {
23                                 $qc->where('nombre', 'like', '%' . $search . '%');
24                             });
25                     });
26             });
27         }
28
29         $preguntas = $preguntas->get();
30
31         return view('CrudPreguntas.GestionarPregunta', compact('preguntas'));
32     }
33
34     public function create()
35     {
36         // Muestra todas las lecciones disponibles para asignar preguntas
37         $lecciones = Leccion::all();
38     }

```

4.3 Base de datos (MySQL)

- **Tablas clave:** users, cursos, lecciones, preguntas, respuestas, resultados, progresos, vidas, roles.
- **Índices:** índices en columnas de búsqueda (**nombre** de curso, **user_id**, **leccion_id**) y claves foráneas.
- **Migrations:** usar migraciones de Laravel para versionar esquema.

```
-- Estructura de tabla para la tabla `roles`
```

```
--
```

```
CREATE TABLE `roles` (
  `id` bigint(20) UNSIGNED NOT NULL,
  `nombre` varchar(255) NOT NULL,
```

```
`created_at` timestamp NULL DEFAULT NULL,
`updated_at` timestamp NULL DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Volcado de datos para la tabla `roles`
--
```

```
INSERT INTO `roles` (`id`, `nombre`, `created_at`, `updated_at`) VALUES
(1, 'Estudiante', '2025-09-03 02:54:18', '2025-09-03 02:54:18'),
(2, 'Administrador', '2025-09-03 02:54:18', '2025-09-03 02:54:18');
```

```
-----
```

```
--
-- Estructura de tabla para la tabla `sessions`
--
```

```
CREATE TABLE `sessions` (
  `id` varchar(255) NOT NULL,
  `user_id` bigint(20) UNSIGNED DEFAULT NULL,
  `ip_address` varchar(45) DEFAULT NULL,
  `user_agent` text DEFAULT NULL,
  `payload` longtext NOT NULL,
  `last_activity` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```
--
-- Volcado de datos para la tabla `sessions`
--
```

```
INSERT INTO `sessions` (`id`, `user_id`, `ip_address`, `user_agent`, `payload`,
`last_activity`) VALUES
('IbZFeeJQllo4fi0QDPKaA3H0okq6DmIalMEopbaH', NULL, '127.0.0.1', 'Mozilla/5.0
(Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/139.0.0.0 Safari/537.36 Edg/139.0.0.0',
'YTo0OntzOjY6Il90b2tlbiI7czo0MDoiZlVEZ2MwcmJRTG9nUjJYVTdXd3BVNHpiQUF
ETlVlaVR6Rlo3WEZTQiI7czo0OiJ1cmwiO2E6MTp7czo4OiJpbnRlbmRlZCI7czo1MDoi
aHR0cDovLzg1NGJhNmMxY2JhNy5uZ3Jvay1mcmVlLmFwcC92aWV3cy9taXNjdXJzb
3MiO31zOjk6Il9wcmV2aW91cyI7YToxOntzOjM6InVyYbCI7czo1MDoiHR0cDovLzg1N
GJhNmMxY2JhNy5uZ3Jvay1mcmVlLmFwcC92aWV3cy9taXNjdXJzb3MiO31zOjY6Il9
mbGFzaCI7YToyOntzOjM6Im9sZCI7YTowOnt9czo0OiJuZXciO2E6MDp7fX19',
1756850727);
```

```
-----
```

```
--
-- Estructura de tabla para la tabla `usuarios`
--
```

```

CREATE TABLE `usuarios` (
  `id` bigint(20) UNSIGNED NOT NULL,
  `nombre` varchar(255) NOT NULL,
  `apellido` varchar(255) NOT NULL,
  `email` varchar(255) NOT NULL,
  `password` varchar(255) NOT NULL,
  `imagen` varchar(255) DEFAULT NULL,
  `vidas` int(11) NOT NULL DEFAULT 5,
  `experiencia` int(11) NOT NULL DEFAULT 0,
  `dias_racha` int(11) NOT NULL DEFAULT 0,
  `ultimo_dia_activo` date DEFAULT NULL,
  `rol_id` bigint(20) UNSIGNED NOT NULL DEFAULT 1,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Volcado de datos para la tabla `usuarios`
--

INSERT INTO `usuarios` (`id`, `nombre`, `apellido`, `email`, `password`, `imagen`,
`vidas`, `experiencia`, `dias_racha`, `ultimo_dia_activo`, `rol_id`, `created_at`,
`updated_at`) VALUES
(1, 'Admin', 'ByteQuest', 'admin@bytequest.com',
'$2y$12$Wcxus2de0.bN4FcB9Mo5meWduBjSrj5ih0ljeSFIVQw1AT.T7yXJy', NULL, 5,
0, 0, NULL, 2, '2025-09-03 02:54:20', '2025-09-03 02:54:20'),
(2, 'Admin', 'ByteQuest', 'admin@gmail.com',
'$2y$12$X0ItZMXvSSfaMvCiGhQVHOiJ5rJuMJJlnNvQIE46Bbf1U2dljucP.', NULL, 5,
0, 0, NULL, 2, '2025-09-03 02:54:28', '2025-09-03 02:54:28'),
(3, 'usuarioprueba', 'prueba', 'usuarioprueba@gmail.com',
'$2y$12$P7EurrDIWs7HS7B7mvHPuepgagYB4VMh.aaoVDt4VcztFQppKtxMG', NULL,
5, 0, 0, NULL, 1, '2025-09-03 02:54:29', '2025-09-03 02:54:29'),
(4, 'John', 'Doe', 'john@example.com',
'$2y$12$YDi4ws/IY8pUA/dCdzFo.u/wAkSzjVnLGv.RsjW3Jk0ySNKO36Jmy', NULL, 5,
100, 0, NULL, 1, '2025-09-03 02:54:39', '2025-09-03 02:54:39');

--

```

5. API / Endpoints (sugeridos)

- POST /api/auth/register — registrar usuario.
- POST /api/auth/login — login (retorna token JWT o session cookie).

- GET /api/cursos — listar cursos.
- GET /api/cursos/{slug} — detalle curso.
- GET /api/cursos/{curso_id}/lecciones — listar lecciones.
- GET /api/lecciones/{id} — obtener contenido de lección.
- POST /api/lecciones/{id}/resultados — enviar resultados de evaluación.
- POST /api/gamificacion/usar-vida — consumir vida.
- GET /api/usuario/progreso — progreso y estadísticas.

Autenticación: Laravel Sanctum o JWT. Preferible Sanctum para SPA + cookies seguras.

6. Seguridad

- **Autenticación y autorización:** Laravel Sanctum; roles y permisos en panel admin.
 - **Encriptación de contraseñas:** Bcrypt (Hash::make).
 - **Protección CSRF/XSS:** middleware CSRF de Laravel y escape en vistas Blade.
 - **Validaciones server-side y client-side.**
 - **Rate limiting:** limitar endpoints sensibles (login, registro).
 - **Backups:** copias diarias de BD y almacenamiento de archivos críticos.
-

7. Procedimientos operativos

7.1 Despliegue (procedimiento básico)

1. Merge a main tras aprobación y pasar CI.
2. GitHub Actions ejecuta pruebas y build.

3. Deploy a servidor (DigitalOcean / Heroku / AWS EC2) o contenedor (Docker).
4. Ejecutar migraciones: `php artisan migrate --force`.
5. Cache config: `php artisan config:cache`.

7.2. Backup y recuperación

- Programar backups automáticos de MySQL (dump) y del storage (S3 snapshot).
- Probar restauración trimestralmente.

7.3. Monitorización y alertas

- Integrar Sentry (errores) y Prometheus/Grafana (métricas) o servicio SaaS.
 - Alertas por caída de servicio, error 5xx o cola acumulada.
-

8. CI/CD (recomendado)

- **Repos:** GitHub.
 - **GitFlow:** ramas feature/*, develop, main.
 - **GitHub Actions:** ejecutar tests, linters, build assets y deploy automático a staging/prod.
-

9. Requerimientos de hardware / hosting

- **Desarrollo:** máquina local con PHP, Composer, MySQL y Laravel.
- **Producción (mínimo para inicio):**
 - 1 vCPU, 2 GB RAM (small droplet) + almacenamiento (20–40 GB).
 - Escalar a 2–4 vCPU y 4–8 GB RAM según uso.

- RDS o managed MySQL para producción recomendado.

9.1. ARQUITECTURA DEL SOFTWARE (DIAGRAMA DE COMPONENTES), Y PATRONES DE DISEÑO DE SOFTWARE.

La aplicación *Byte Quest* se diseña como una aplicación web en capas con la siguiente separación lógica de responsabilidades:

- **Capa de Presentación (Frontend)**
 - Componentes: vistas, páginas, componentes UI (login, dashboard, vista de lección, cuestionario, tienda, perfil).
 - Tecnologías: HTML5, CSS3, JavaScript, Bootstrap (y bibliotecas JS si se requiere).
 - Responsabilidad: interacción con el usuario, validaciones de UI, consumo de las APIs del backend.
- **Capa de Aplicación / Servicio (Backend - Laravel)**
 - Componentes: Controladores HTTP, Servicios (Services), Casos de uso / Interactors, DTOs.
 - Responsabilidad: orquestar lógica de negocio, coordinar repositorios y validaciones, exponer endpoints REST/JSON.
- **Capa de Dominio**
 - Componentes: Entidades/Modelos (Usuario, Curso, Lección, Pregunta, Respuesta, Progreso, Vidas, Compra), reglas de negocio puros.
 - Responsabilidad: contener la lógica de dominio y las invariantes del negocio (por ejemplo, reglas de vidas y desbloqueo de lecciones).
- **Capa de Persistencia / Infraestructura**
 - Componentes: Repositorios, Gateways, Adaptadores a base de datos (MySQL), almacenamiento de ficheros/medios, servicios externos (pasarelas de pago, servicio de emails, CDN, servicio de alojamiento de videos).
 - Infraestructura: Hosting en **Railway** (producción), base de datos MySQL (gestión en Railway o servicio administrado), almacenamiento para vídeos,

backups y logs.

- **Capa de Integración / Operaciones**

- Componentes: CI/CD (GitHub Actions), gestión de versiones (GitHub), monitorización (logs, Sentry), backups automáticos, configuración de despliegue en Railway.

Topología de despliegue (resumen):

Cliente (navegador) $\rightleftharpoons$ Frontend (estático + JS) $\rightarrow$ Backend (Laravel en Railway) $\rightarrow$ MySQL (DB en Railway o externo)

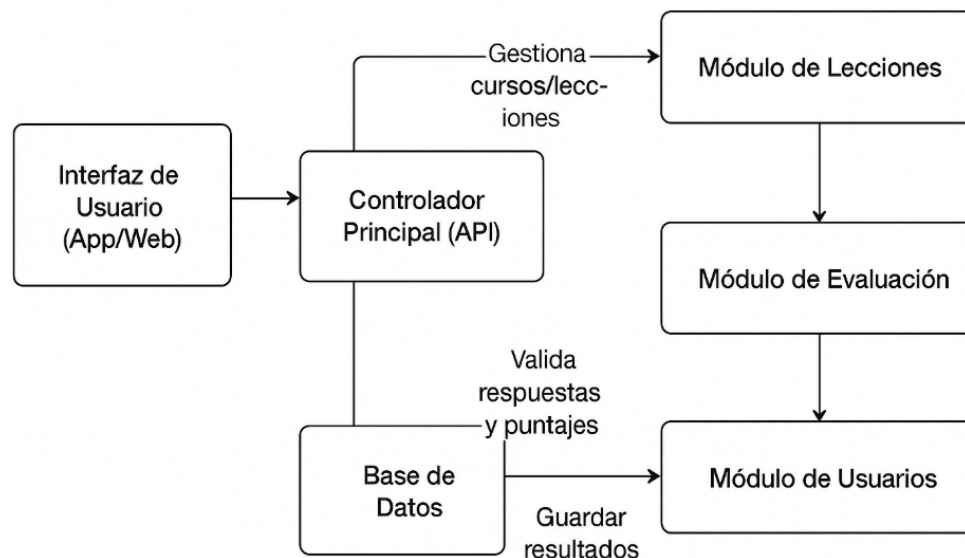
Recursos estáticos/medios $\rightarrow$ CDN/Storage

CI/CD $\rightarrow$ GitHub Actions $\rightarrow$ Deploy a Railway

9.1.2. Diagrama de componentes

Imagen

Diagrama de Componentes - ByteQuest



9.1.3. Requisitos no funcionales asociados a la arquitectura

- **Escalabilidad:** separar media y lógica; Railway permite instancias escalables.
- **Disponibilidad:** backups diarios de la BD, health checks y monitorización.
- **Seguridad:** HTTPS obligatorio, sanitización de entradas, protección contra CSRF/SQL injection, hashing de contraseñas, políticas de roles.

- **Rendimiento:** cacheo de recursos estáticos, paginación en consultas pesadas, uso de índices en MySQL.
- **Mantenibilidad:** separación por capas, tests automatizados

9.1.4. Patrones de diseño recomendados y justificación

1. **MVC (Model-View-Controller)** — *Usado por Laravel*
 - Justificación: separación clara entre presentación, lógica de negocio y acceso a datos. Facilita mantenimiento y comprensión.
2. **Repository Pattern**
 - Uso: Repositorios para encapsular acceso a la BD
 - Justificación: desacopla la capa de dominio de la tecnología de persistencia; facilita pruebas unitarias (mocking).
 -
3. **Service / Use Case Layer (Application Service)**
 - Uso: Servicios que contienen la lógica de las operaciones
 - Justificación: mantiene controladores delgados y agrupa reglas de negocio complejas.
4. **Factory Pattern**
 - Uso: creación de objetos complejos, por ejemplo generación de preguntas/ejercicios dinámicos o entidades de prueba.
 - Justificación: centraliza la lógica de creación y favorece la extensibilidad.
5. **Strategy Pattern**
 - Uso: para diferentes estrategias de evaluación o cálculo de puntajes (p. ej. evaluación por tiempo vs. por precisión).
 - Justificación: permite intercambiar comportamientos de evaluación sin cambiar el cliente.
6. **Observer / Event-Driven (Eventos y Listeners de Laravel)**
 - Uso: emitir eventos al completar una lección (e.g. **Lesson Completed**) y listeners para otorgar puntos, actualizar logros o enviar notificaciones.
 - Justificación: desacopla acciones secundarias del flujo principal y facilita agregar nuevas reacciones (ej: badges, emails).
7. **Adapter Pattern**

- Uso: para integrar servicios externos (p. ej. un servicio de almacenamiento distinto o una pasarela de pagos).
- Justificación: encapsula diferencias de APIs externas y facilita cambios futuros.

8. Facade Pattern (facades de Laravel o propia)

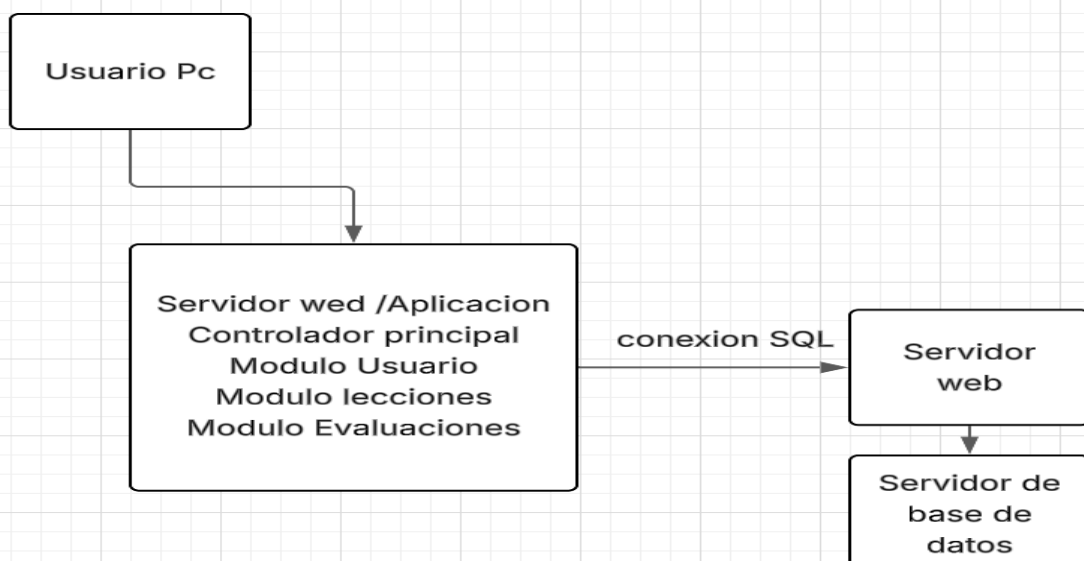
- Uso: simplificar acceso a subsistemas complejos (por ejemplo: **Payment Facade** o **Media Facade**).
- Justificación: ofrece una interfaz simple para operaciones compuestas.

9. Dependency Injection (DI)

- Uso: inyectar repositorios y servicios en controladores y otros servicios (Laravel IoC container).
- Justificación: facilita testing, reemplazo de implementaciones y reduce acoplamiento.

10. Singleton (con precaución)

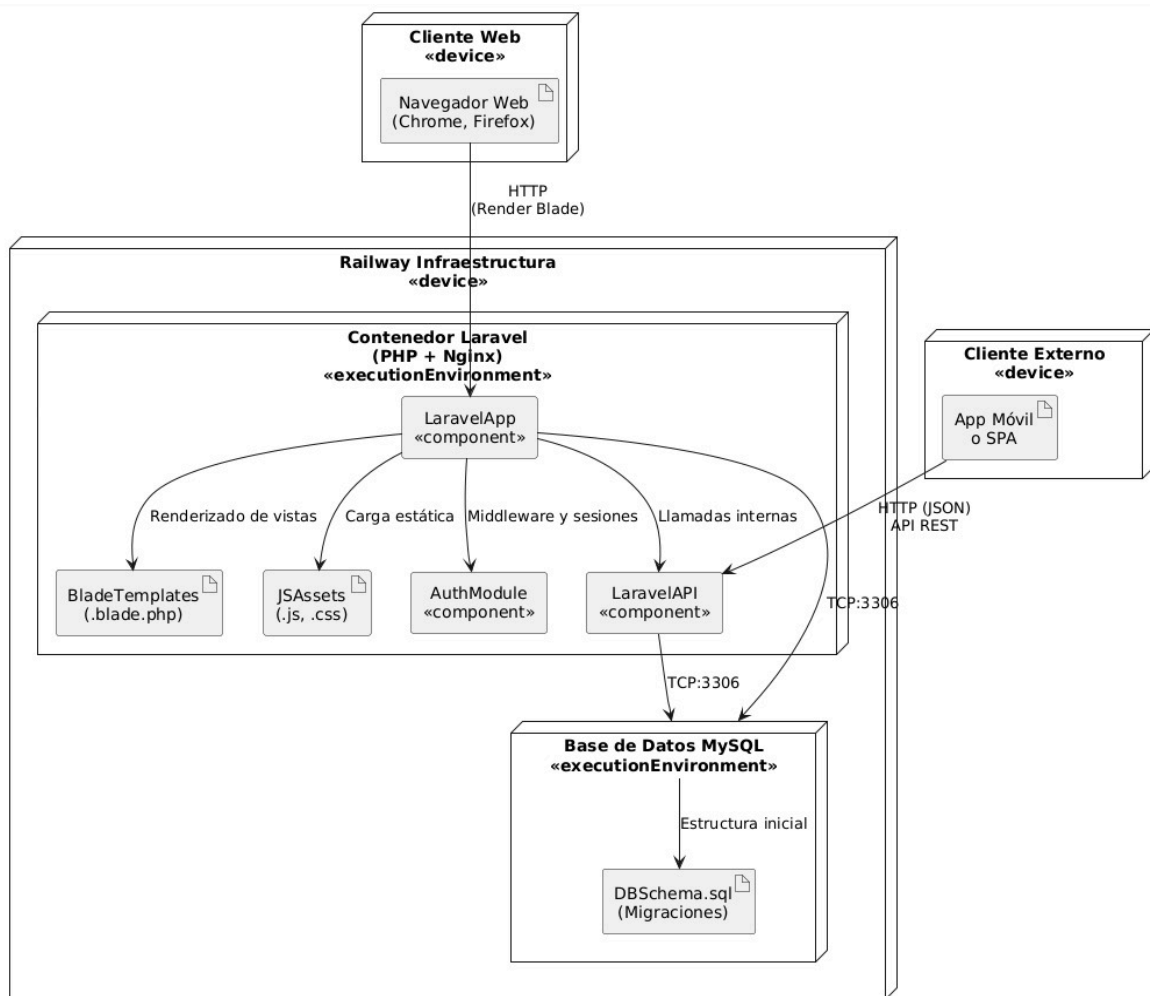
- Uso: recursos únicos de configuración o servicios que deben tener una única instancia (p. ej. logger).
- Justificación: controlar recursos compartidos; usar con moderación para no dificultar tests.



9.1.5. Consideraciones de despliegue y operaciones

- Configurar variables de entorno en Railway (APP_KEY, DB credentials, STORAGE keys).
- Habilitar backups automáticos de la base de datos y políticas de retención.
- Usar GitHub Actions para ejecutar tests, compilar assets (npm run build) y desplegar automáticamente a Railway.
- Habilitar monitorización (Sentry) y alertas ante errores críticos.
- Plan de rollback en despliegues (versionado y migraciones controladas).

9.2. DIAGRAMA DE DESPLIEGUE.

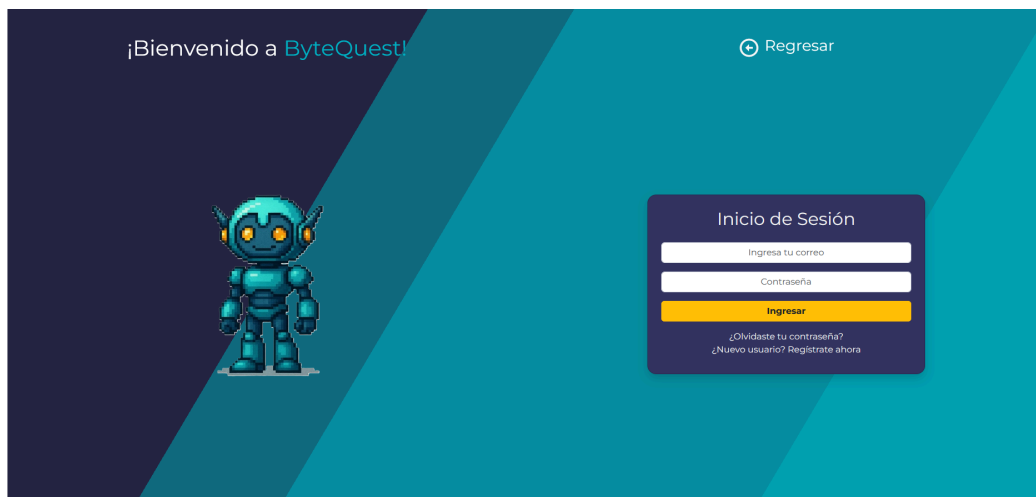


9.3. DISEÑO FRONT-END - INTERFACES GRÁFICAS DE USUARIO (HTML.CSS Y JS)

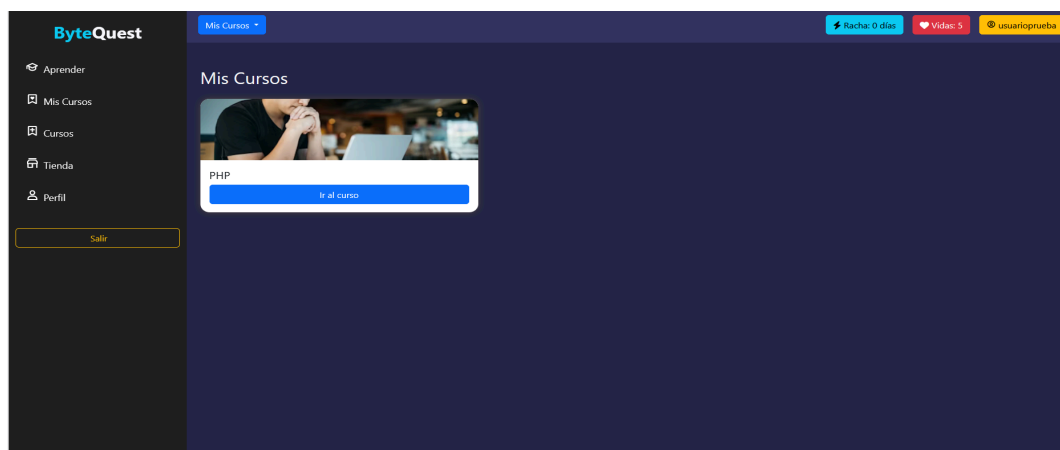
Landing



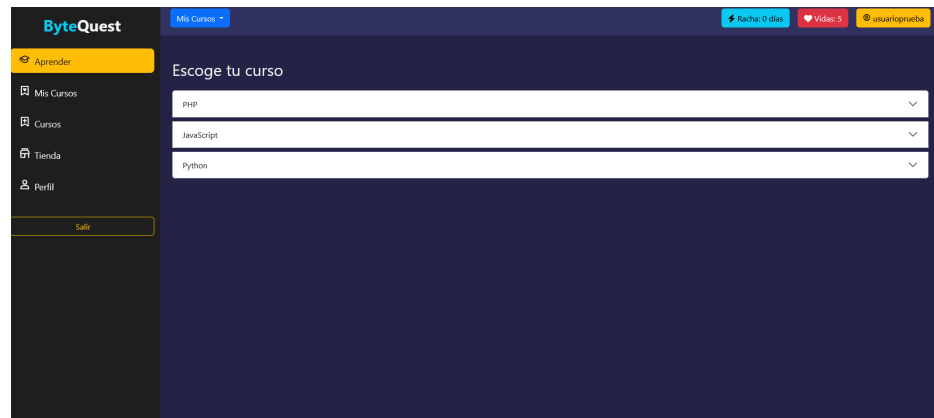
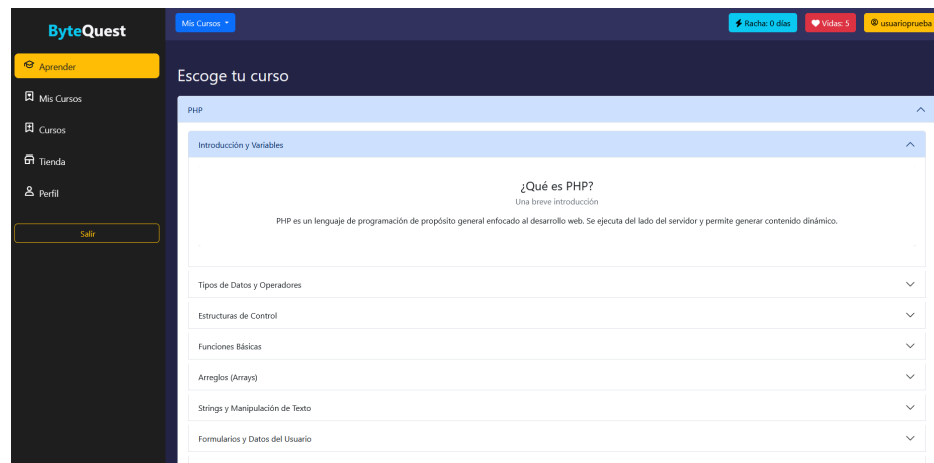
Login



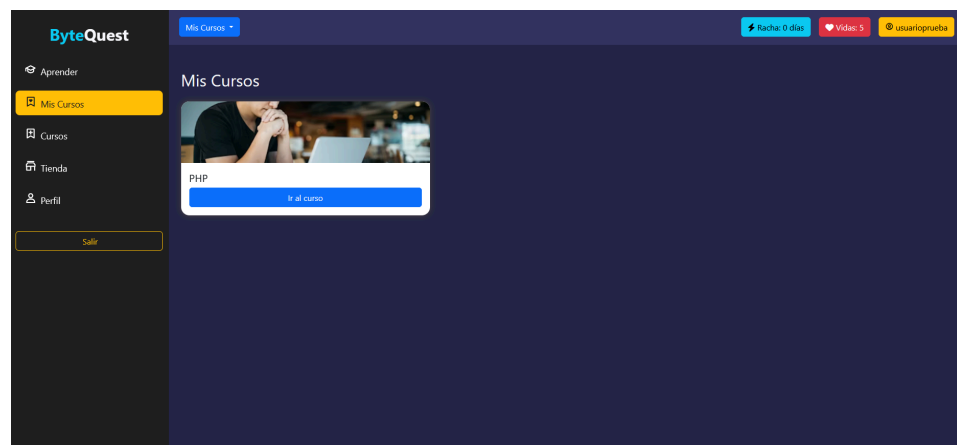
Home aprendiz



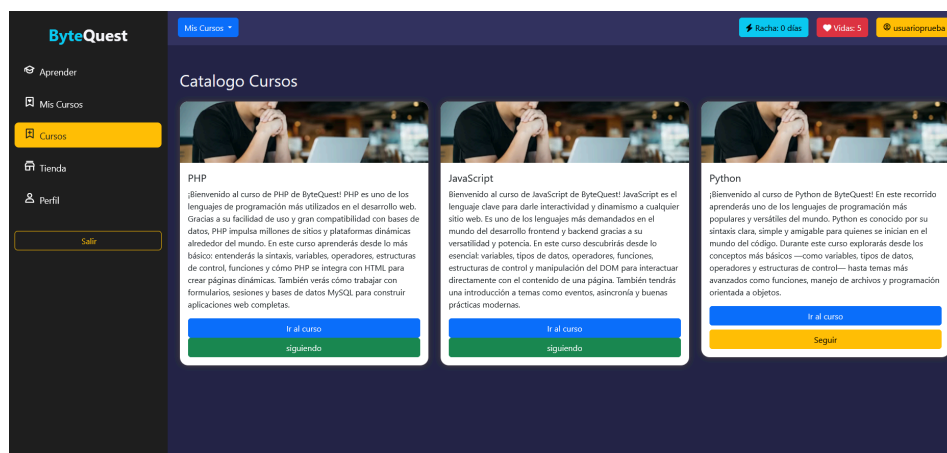
Aprender



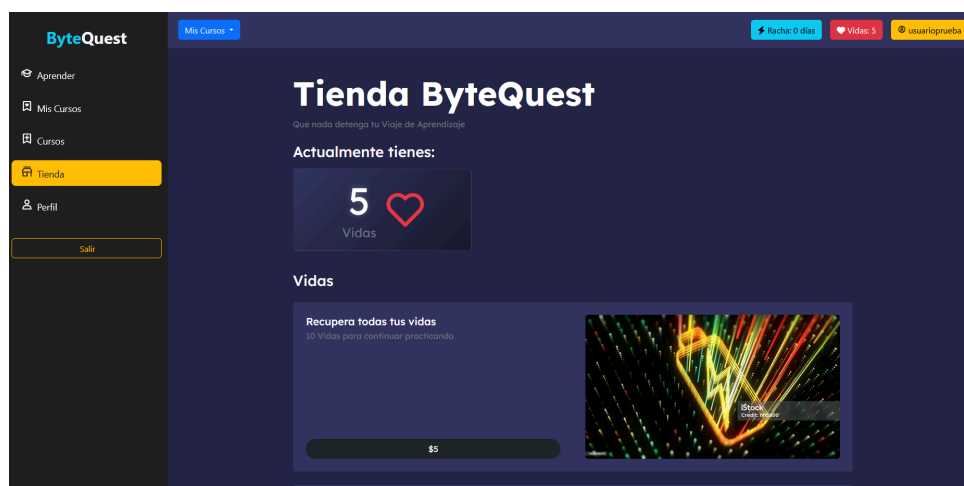
Mis Cursos



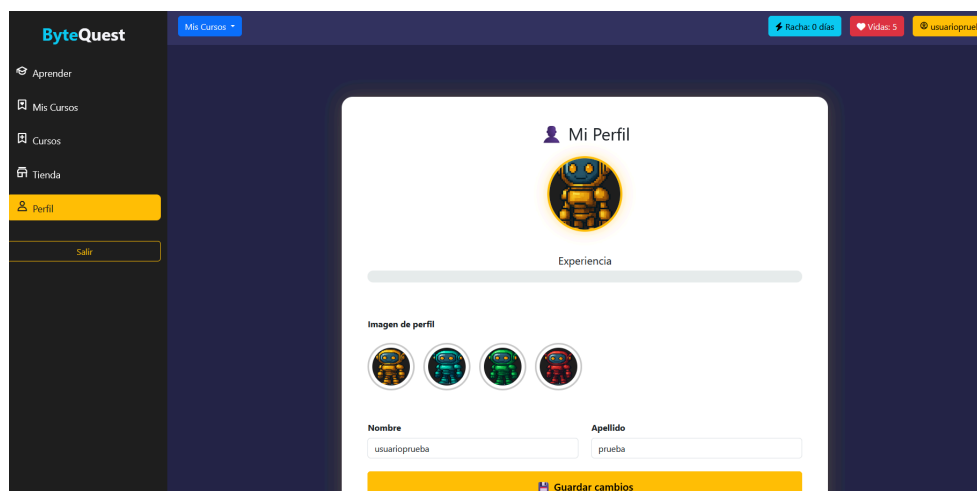
Cursos



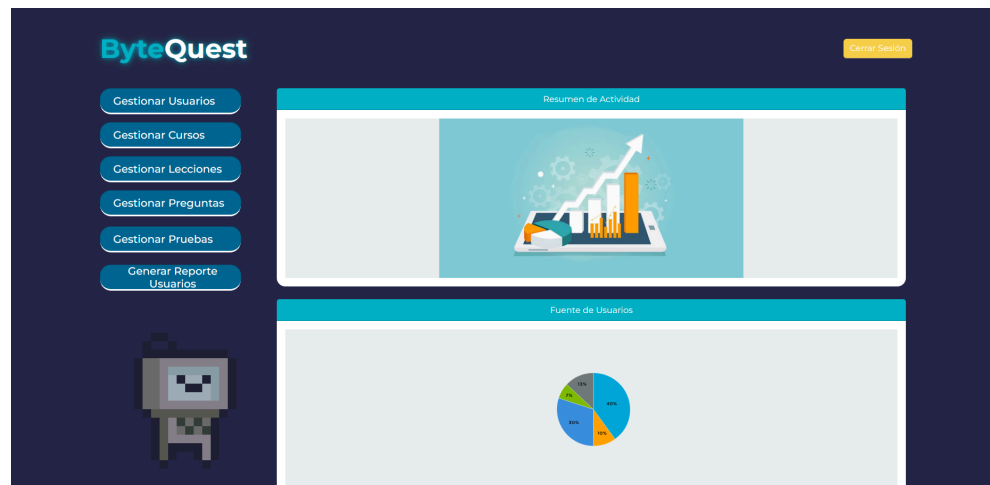
Tienda



Perfil



Home administrador



Gestión de usuarios

ByteQuest Usuarios Cursos Lecciones Pruebas Preguntas [Regresar a Dashboard](#) [Cerrar Sesión](#)

Gestión de Usuarios [Añadir Usuario](#)

Buscar Usuario [Buscar](#)

#	Nombre	Apellido	Correo	Rol	Editar	Eliminar
1	Admin	ByteQuest	admin@bytequest.com	2		
2	Admin	ByteQuest	admin@gmail.com	2		
3	usuarioprueba	prueba	usuarioprueba@gmail.com	1		
4	John	Doe	john@example.com	1		

Gestión de cursos

ByteQuest Usuarios Cursos Lecciones Pruebas Preguntas [Regresar a Dashboard](#) [Cerrar Sesión](#)

Gestión de Cursos [Añadir Curso](#)

ID	Nombre	Descripción	Editar	Eliminar
1	PHP	¡Bienvenido al curso de PHP de ByteQuest! PHP es uno de los lenguajes de programación más utilizados en el desarrollo web. Gracias a su facilidad de uso y gran compatibilidad con bases de datos, PHP impulsa millones de sitios y plataformas dinámicas alrededor del mundo. En este curso aprenderás desde lo más básico: entenderás la sintaxis, variables, operadores, estructuras de control, funciones y cómo PHP se integra con HTML para crear páginas dinámicas. También verás cómo trabajar con formularios, sesiones y bases de datos MySQL para construir aplicaciones web completas.		
2	JavaScript	Bienvenido al curso de JavaScript de ByteQuest! JavaScript es el lenguaje clave para darle interactividad y dinamismo a cualquier sitio web. Es uno de los lenguajes más demandados en el mundo del desarrollo frontend y backend gracias a su versatilidad y potencia. En este curso descubrirás desde lo esencial: variables, tipos de datos, operadores, funciones, estructuras de control y manipulación del DOM para interactuar directamente con el contenido de una página. También tendrás una introducción a temas como eventos, asincronía y buenas prácticas modernas.		
3	Python	¡Bienvenido al curso de Python de ByteQuest! En este recorrido aprenderás uno de los lenguajes de programación más populares y versátiles del mundo. Python es conocido por su sintaxis clara, simple y amigable para quienes se inician en el mundo del código. Durante este curso explorarás desde los conceptos más básicos —como variables, tipos de datos, operadores y estructuras de control— hasta temas más avanzados como funciones, manejo de archivos y programación orientada a objetos.		

Gestión de lecciones


Usuarios Cursos Lecciones Pruebas Preguntas

[Regresar a Dashboard](#)
[Cerrar Sesión](#)

Gestión de lecciones

PHP		
#	Nombre	Descripción
1	Introducción y Variables	Aprende qué es PHP, cómo funciona, y domina la declaración y uso de variables para almacenar y manipular datos básicos.
2	Tipos de Datos y Operadores	Explora los tipos de datos en PHP: strings, números, booleanos y cómo combinarlos con operadores aritméticos y lógicos.
3	Estructuras de Control	Aprende a usar condicionales (if, else) y bucles (for, while) para controlar el flujo de tus programas PHP.
4	Funciones Básicas	Define y usa funciones para organizar tu código y reutilizar bloques lógicos de forma eficiente.
5	Arreglos (Arrays)	Trabaja con arrays para almacenar múltiples valores y manipular listas de datos en PHP.
6	Strings y Manipulación de Texto	Aprende a manipular cadenas de texto con funciones integradas de PHP para crear mensajes dinámicos.
7	Formularios y Datos del Usuario	Envía y recibe datos de formularios HTML, procesa entradas del usuario de forma segura.
8	Manejo de Errores	Identifica y corrige errores comunes, implementa manejo básico de excepciones.
9	Buenas Prácticas	Aprende prácticas recomendadas para escribir código limpio, organizado y fácil de mantener.
10	Repaso y Nivel de Dominio	Repasa todos los conceptos clave a través de preguntas mezcladas y mini retos para afianzar tus conocimientos.

Gestión de preguntas

[ByeQuest](#)
[Usuarios](#)
[Cursos](#)
[Lecciones](#)
[Pruebas](#)
[Preguntas](#)

[Regresar a Dashboard](#)
[Cerrar Sesión](#)

Gestión de Preguntas

PHP

Introducción y Variables

#	Pregunta	Imagen	Acciones
1	¿Cómo se declara una variable en PHP?	No	Editar Eliminar
2	¿Cuál es el prefijo obligatorio para variables en...	No	Editar Eliminar
3	¿Qué tipo de dato puede tener una variable en PHP?	No	Editar Eliminar
4	¿Cuál es la forma correcta de asignar un valor?	No	Editar Eliminar
5	¿Qué símbolo se usa para concatenar en PHP?	No	Editar Eliminar
6	¿Cómo se imprime texto en PHP?	No	Editar Eliminar
7	¿Qué extensión tienen los archivos PHP?	No	Editar Eliminar
8	¿Qué función devuelve la longitud de una string?	No	Editar Eliminar
9	¿Cómo se crea un array?	No	Editar Eliminar
10	¿Qué estructura de control existe en PHP?	No	Editar Eliminar

Gestión de pruebas

ByteQuest

[Usuarios](#)[Cursos](#)[Lecciones](#)[Pruebas](#)[Preguntas](#)

[Regresar a Dashboard](#)[Cerrar Sesión](#)

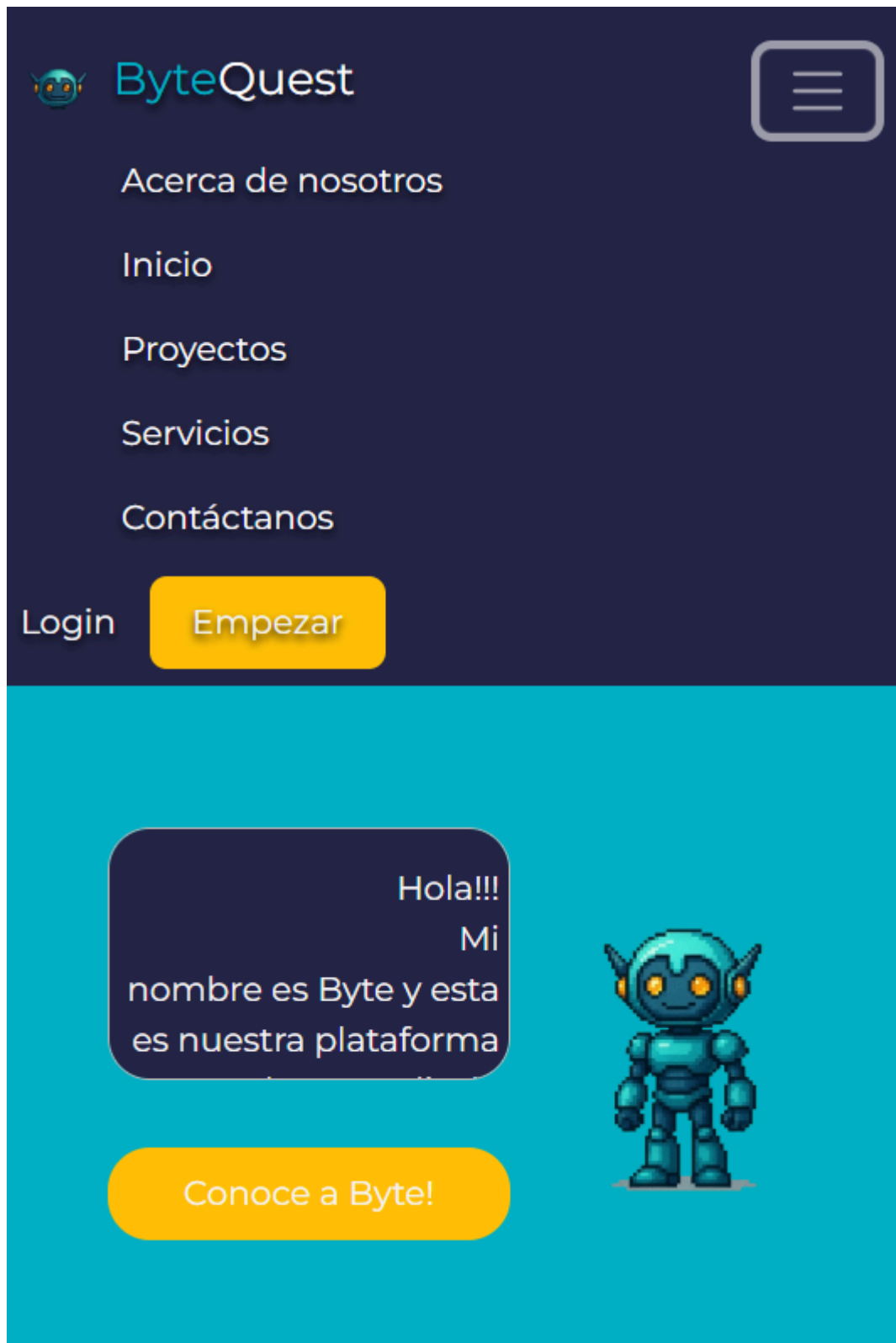
Gestión de Pruebas

Añadir Prueba

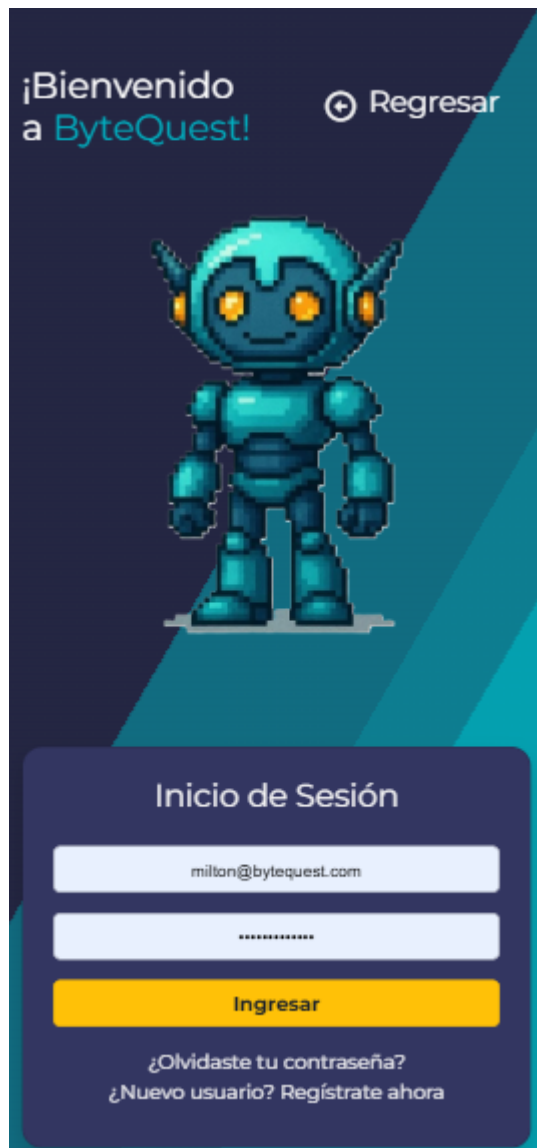
PHP	▼
JavaScript	▼
Python	▼

9.4. INTERFACES GRÁFICAS DE USUARIO MÓVILES.

landing



login



register



A mobile application registration screen with a dark blue background and a teal header and footer. At the top right, there is a teal button with a white plus icon and the text "Regresar". Below this, the title "Registrarse" is centered in white. The form consists of five white input fields, each preceded by a label in white: "Nombre:", "Apellido:", "Correo:", "Contraseña:", and "Confirmar Contraseña:". Below the last input field is a teal button with the text "Registrarse" in white. At the bottom, there is a link in white text that says "¿Ya tienes cuenta? Inicia sesión".

[+ Regresar](#)

Registrarse

Nombre:

Apellido:

Correo:

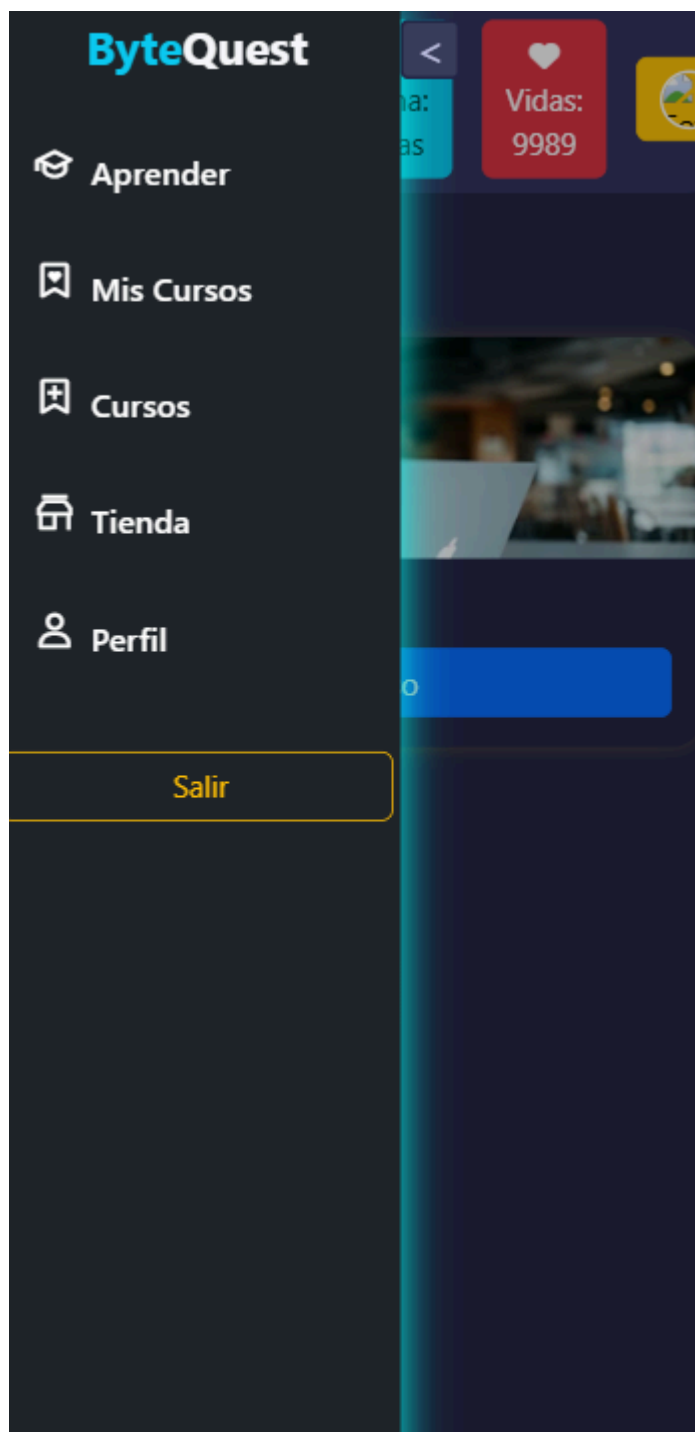
Contraseña:

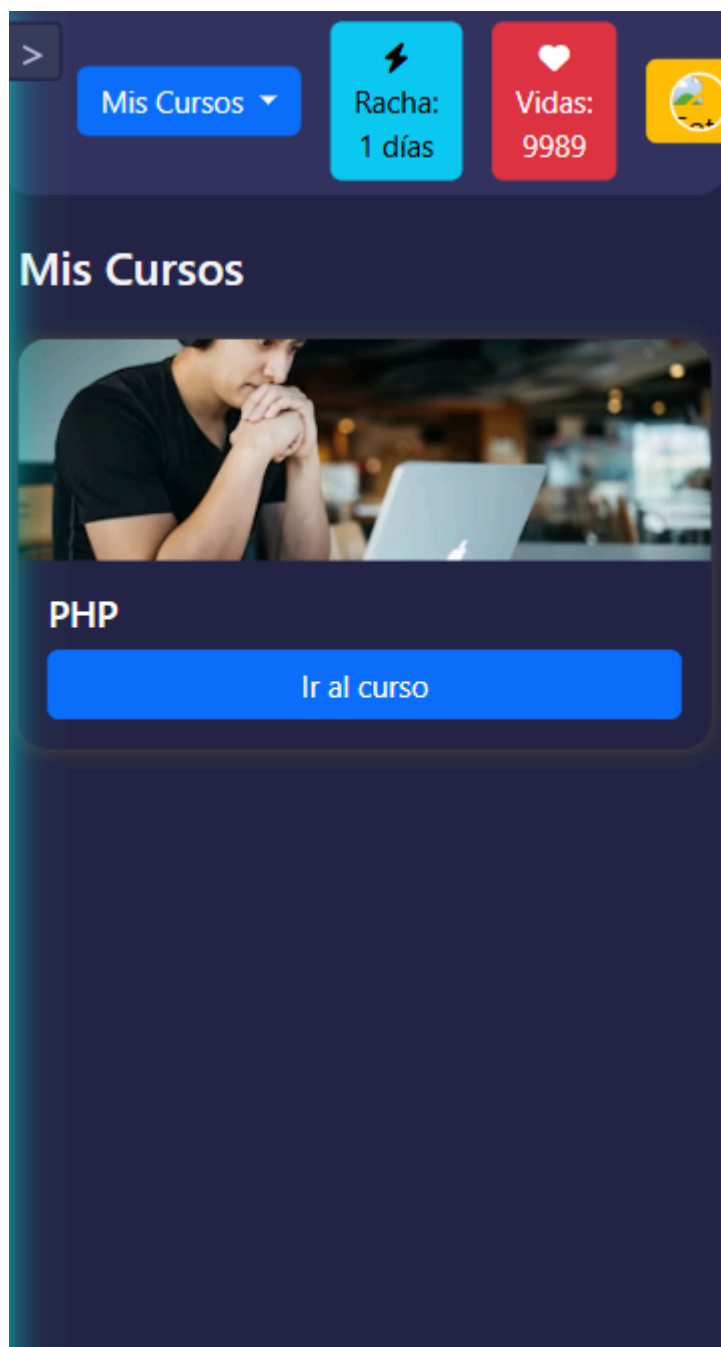
Confirmar Contraseña:

[Registrarse](#)

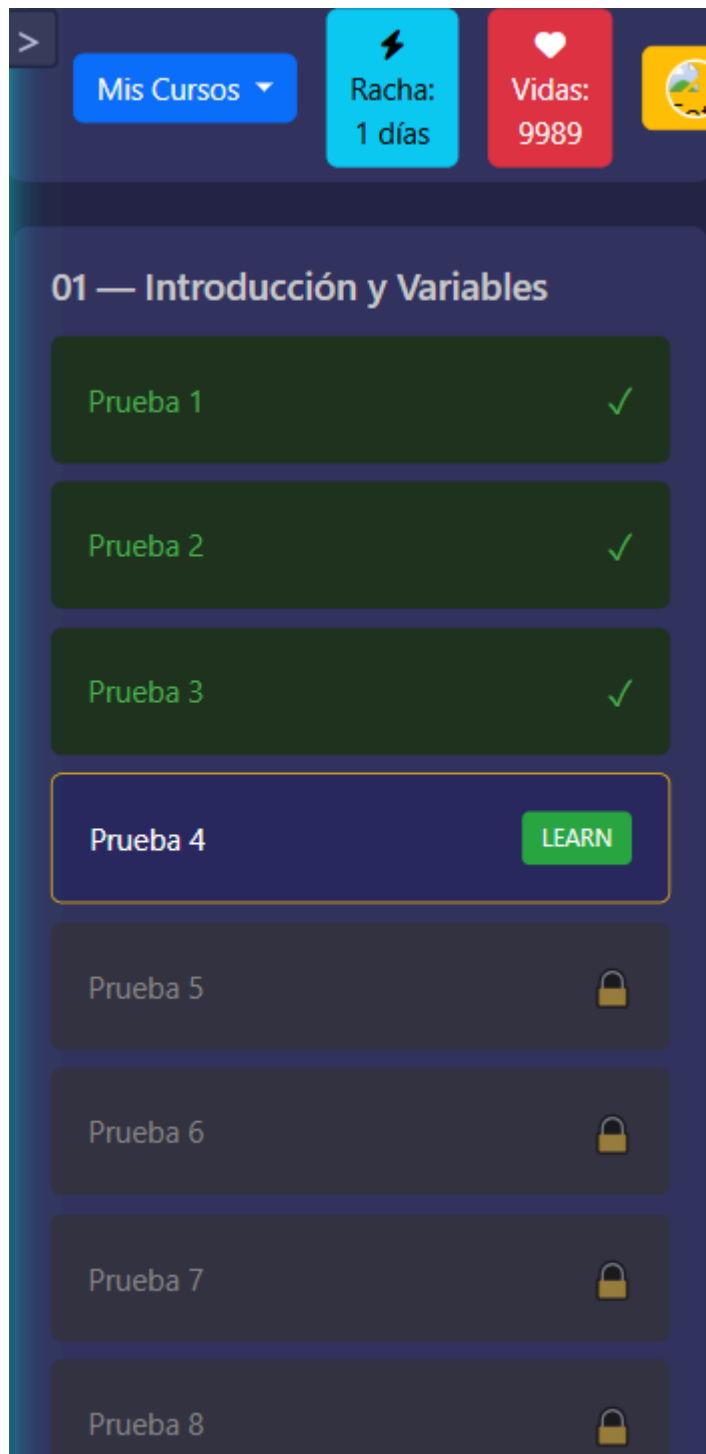
[¿Ya tienes cuenta? Inicia sesión](#)

vista usuarios:

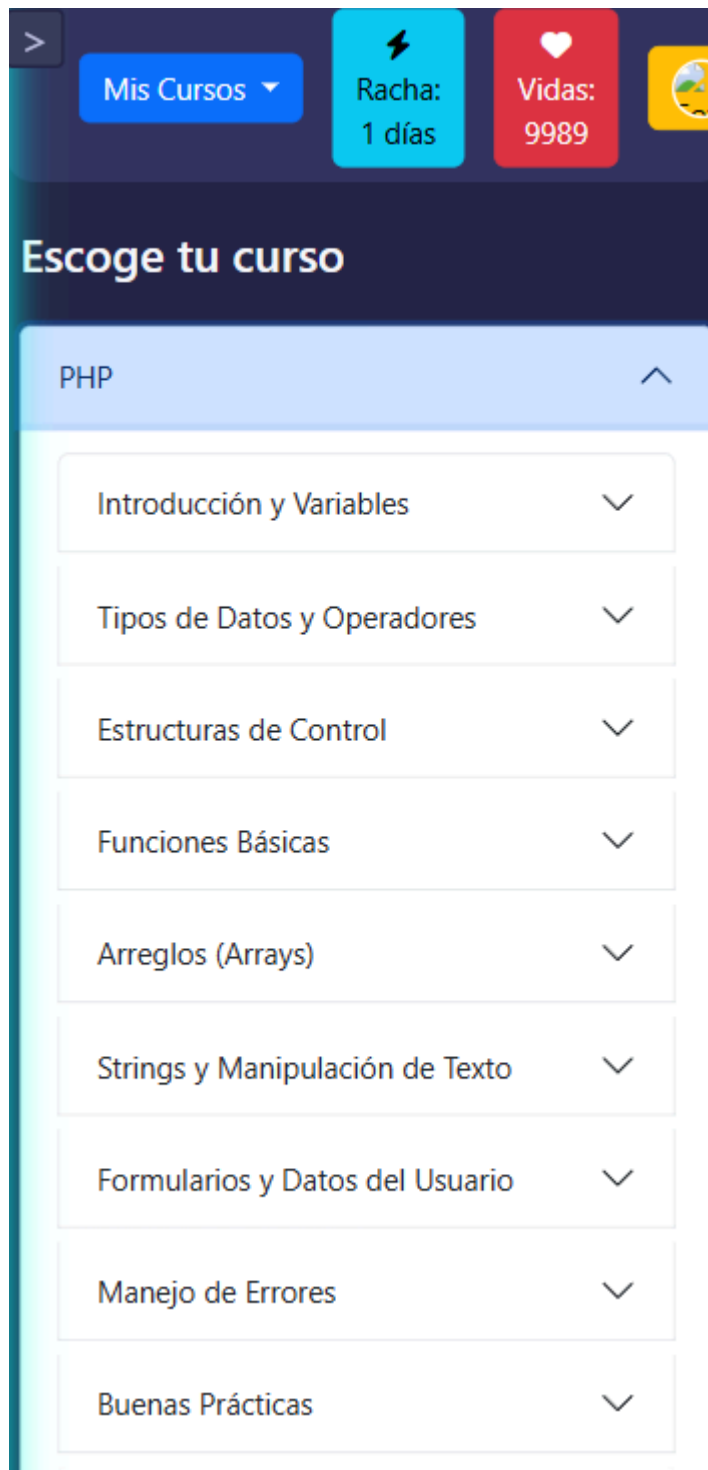




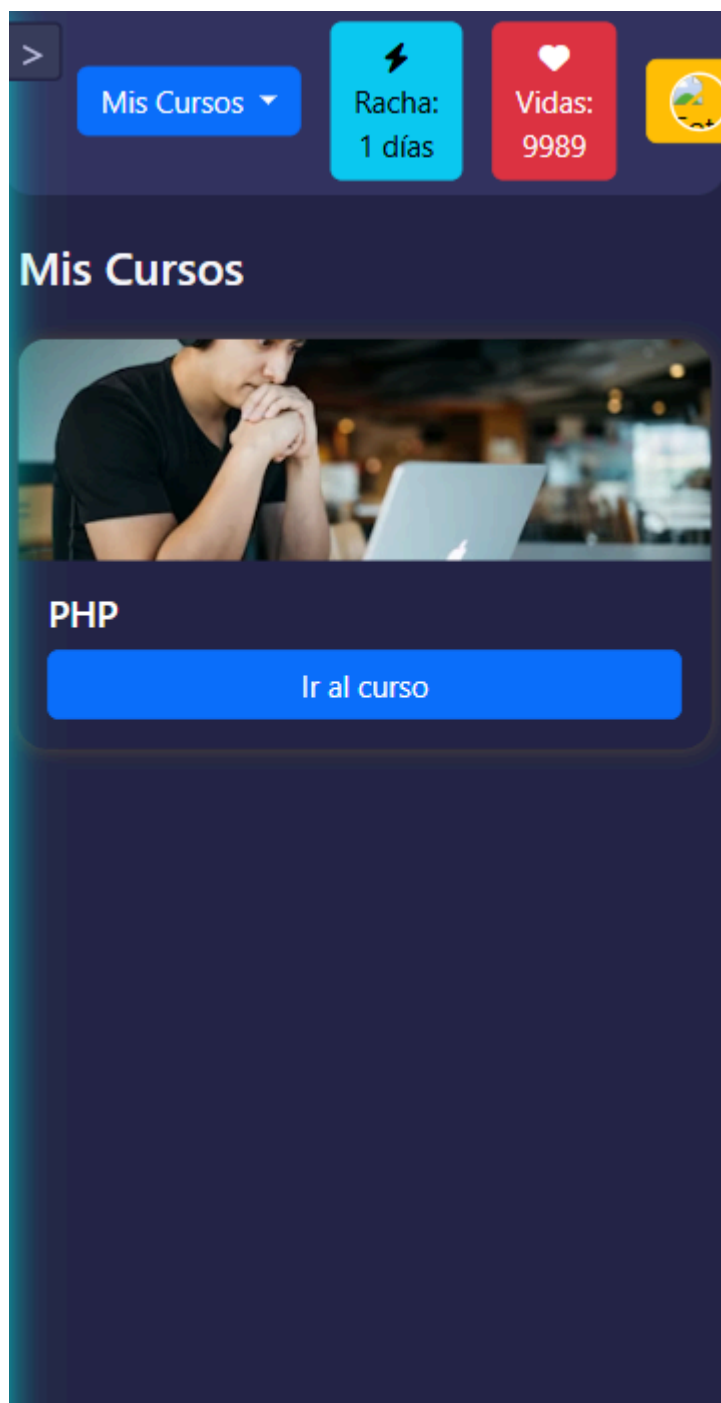
usuario camino



catálogo de cursos



mis cursos:



catalogo cursos



The screenshot shows a mobile application interface for a course catalog. At the top, there is a navigation bar with a blue button labeled 'Mis Cursos' and a dropdown arrow. To its right are two status boxes: a blue one with a lightning bolt icon showing 'Racha: 1 días' and a red one with a heart icon showing 'Vidas: 9989'. A yellow profile icon is on the far right. Below the navigation bar is a section titled 'Catalogo Cursos' in white text on a dark background. This section features a large image of a person working on a laptop. Below the image, the title 'PHP' is displayed in bold. The main text describes the course as a welcome to the PHP course by ByteQuest, highlighting its popularity in web development and the topics it covers, from basic syntax to database integration. At the bottom of this section are two large buttons: a blue one labeled 'Ir al curso' and a green one labeled 'siguiendo'. The bottom of the screen shows the top of the same person's head from the image above.

>

Mis Cursos ▾

Racha:
1 días

Vidas:
9989

Catalogo Cursos

PHP

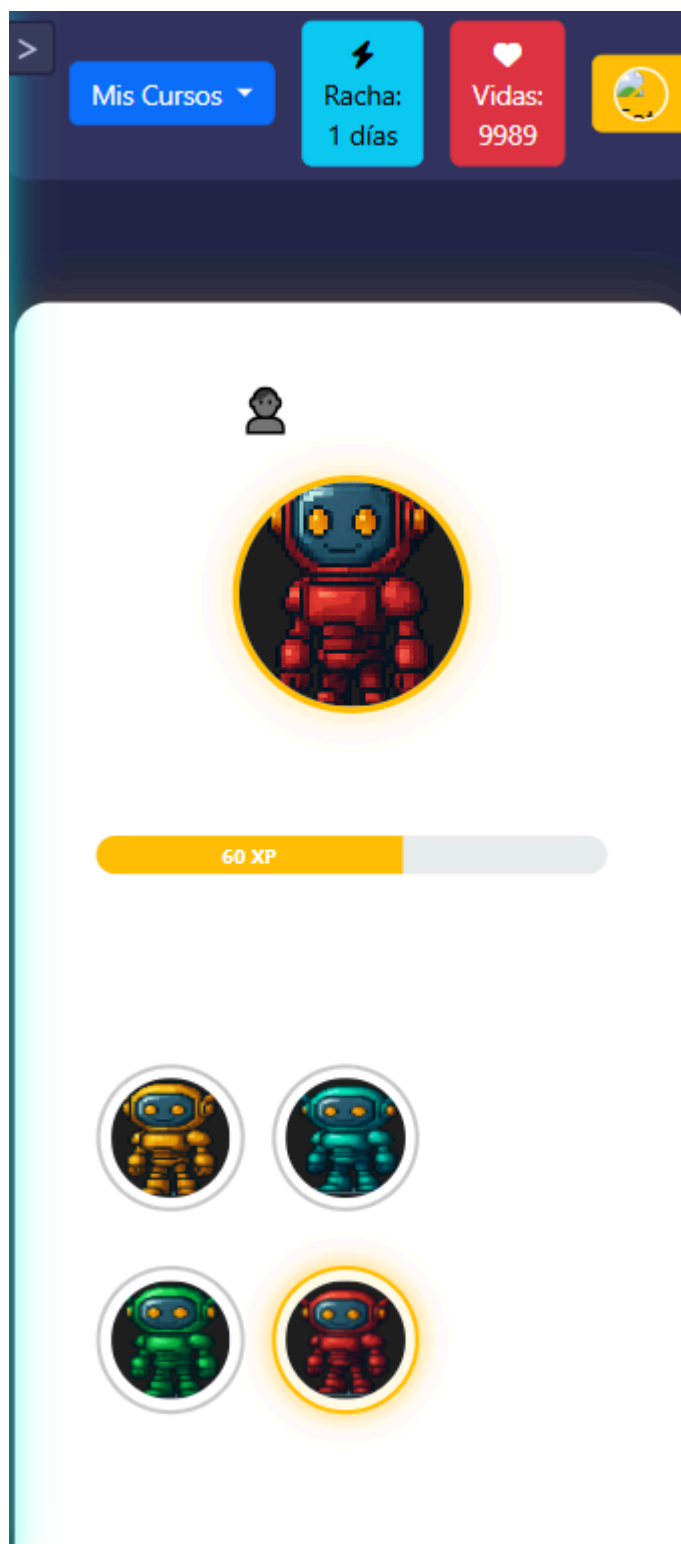
¡Bienvenido al curso de PHP de ByteQuest! PHP es uno de los lenguajes de programación más utilizados en el desarrollo web. Gracias a su facilidad de uso y gran compatibilidad con bases de datos, PHP impulsa millones de sitios y plataformas dinámicas alrededor del mundo. En este curso aprenderás desde lo más básico: entenderás la sintaxis, variables, operadores, estructuras de control, funciones y cómo PHP se integra con HTML para crear páginas dinámicas. También verás cómo trabajar con formularios, sesiones y bases de datos MySQL para construir aplicaciones web completas.

Ir al curso

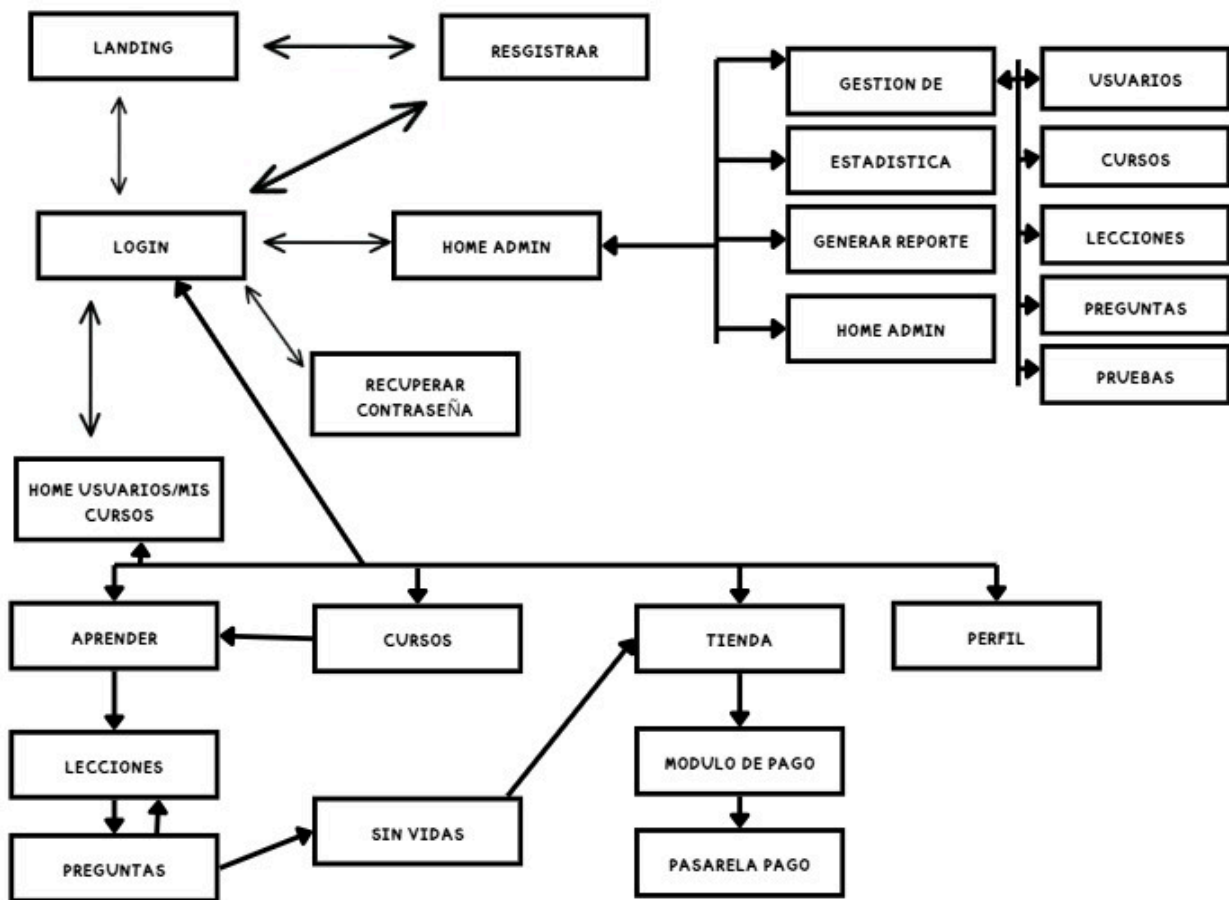
siguiendo

tienda





9.5. MAPA DE NAVEGACIÓN DE LA APLICACIÓN.



9.6. DETERMINAR TIPOS DE BASES DE DATOS.

En el desarrollo del proyecto **ByteQuest**, la base de datos constituye un elemento central para el almacenamiento, organización y gestión de la información de los usuarios, cursos, lecciones y transacciones. Dada la naturaleza de la aplicación, se requiere una solución que garantice **consistencia, seguridad, escalabilidad y eficiencia** en el manejo de datos.

1. Base de Datos Relacional

Se implementará una **base de datos relacional** (MySQL o PostgreSQL), debido a que permite modelar de forma clara las relaciones entre las diferentes entidades del sistema.

En ella se gestionará:

- Información de los usuarios (datos personales, credenciales, planes de suscripción).
- Progreso académico (cursos, lecciones completadas, resultados de evaluaciones).
- Gestión de la tienda (productos, configuraciones de pago, historial de compras).

- Administración del sistema (roles, permisos y configuraciones del administrador).

2. Base de Datos No Relacional (Complementaria)

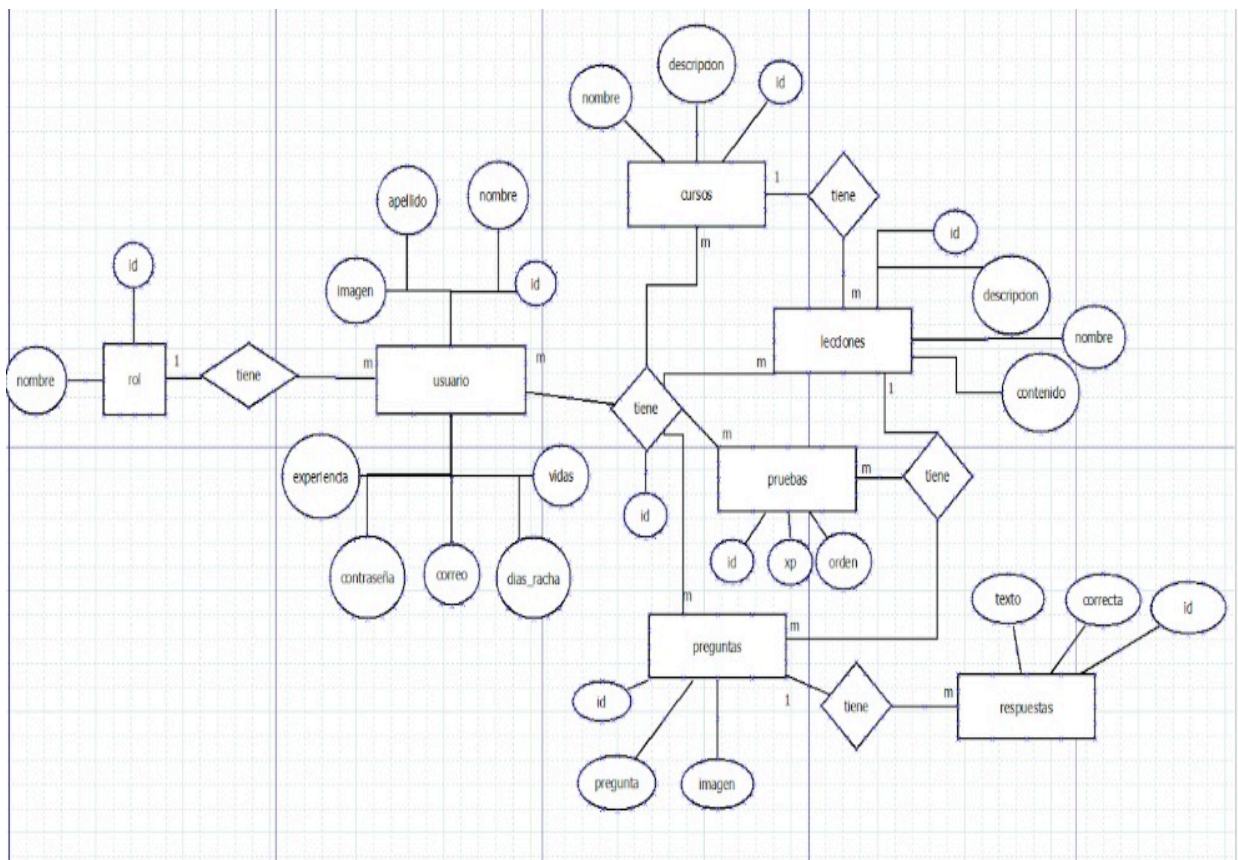
De manera complementaria, se puede integrar una **base de datos NoSQL** como **MongoDB** o **Redis**, para cubrir aspectos dinámicos del sistema que requieren un rendimiento más ágil en tiempo real.

En esta se manejan principalmente:

- Sesiones activas de usuarios.
- Rankings y estadísticas de uso en tiempo real.
- Almacenamiento de logs de actividad y notificaciones rápidas.

La inclusión de este tipo de base de datos permite mayor flexibilidad en el manejo de información no estructurada y mejora la experiencia de los usuarios al ofrecer respuestas más inmediatas en procesos críticos.

9.7. MODELO DE DATOS DIAGRAMA ER



9.8. DICCIONARIO DE DATOS.

Tabla_1_Usuario						
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción	
Id	BIGINT	20	No	PK	Identificador único del usuario.	
Nombre	VARCHAR	150	No		Nombre completo del usuario.	
Correo	VARCHAR	150	No	UQ	Correo electrónico único para login.	
Password	VARCHAR	255	No		Contraseña encriptada.	
Vidas	INT	11	Sí		Número de vidas disponibles (gamificación).	
Racha	INT	11	Sí		Puntuación acumulada.	
created_at	TIMESTAMP	—	Sí		Fecha de creación.	
updated_at	TIMESTAMP	—	Sí		Última actualización.	

Tabla_2_rols		
Campo	Tipo de Dato	Descripción
id	INT (PK)	Identificador único del rol
nombre	VARCHAR	Nombre del rol (Admin, Estudiante, Profesor, etc.)
descripcion	TEXT	Detalle del rol
created_at	TIMESTAMP	Fecha de creación
updated_at	TIMESTAMP	Última actualización

Tabla_3_cursos						
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción	
id	BIGINT	20	No	PK	Identificador único del curso.	
nombre	VARCHAR	255	No		Nombre del curso.	
descripcion	TEXT	—	Sí		Breve descripción del curso.	
slug	VARCHAR	255	No	UQ	Identificador amigable en URL.	
created_at	TIMESTAMP	—	Sí		Fecha de creación.	
updated_at	TIMESTAMP	—	Sí		Última actualización.	

A	B	C	D	E	F
Tabla_4_Lecciones					
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción
id	BIGINT	20	No	PK	Identificador único de la lección.
curso_id	BIGINT	20	No	FK	Relación con el curso al que pertenece.
titulo	VARCHAR	255	No		Título de la lección.
tipo	ENUM('texto','video')	—	No		Tipo de contenido de la lección.
contenido	TEXT	—	Sí		Texto o ruta al archivo de video.
orden	INT	11	Sí		Posición dentro del curso.
created_at	TIMESTAMP	—	Sí		Fecha de creación.
updated_at	TIMESTAMP	—	Sí		Última actualización.

Tabla_5_Preguntas					
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción
id	BIGINT	20	No	PK	Identificador único de la pregunta.
leccion_id	BIGINT	20	No	FK	Relación con la lección.
enunciado	TEXT	—	No		Texto de la pregunta.
tipo	ENUM('unica','multiple')	—	No		Tipo de respuesta permitida.
created_at	TIMESTAMP	—	Sí		Fecha de creación.
updated_at	TIMESTAMP	—	Sí		Última actualización.

Tabla_6_respuestas					
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción
id	BIGINT	20	No	PK	Identificador único de la respuesta.
pregunta_id	BIGINT	20	No	FK	Relación con la pregunta.
texto	TEXT	—	No		Texto de la respuesta.
es_correcta	BOOLEAN	—	No		Indica si la respuesta es correcta.
created_at	TIMESTAMP	—	Sí		Fecha de creación.
updated_at	TIMESTAMP	—	Sí		Última actualización.

Tabla_7_Resultado					
Campo	Tipo de dato	Longitud	Nulo	Clave	Descripción
id	BIGINT	20	No	PK	Identificador único del resultado.
user_id	BIGINT	20	No	FK	Usuario al que pertenece el resultado.
leccion_id	BIGINT	20	No	FK	Lección a la que corresponde.
correctas	INT	11	Sí		Número de respuestas correctas.
totales	INT	11	Sí		Número total de preguntas.
puntuacion	INT	11	Sí		Puntaje obtenido.
created_at	TIMESTAMP	—	Sí		Fecha del registro.

9.9. POLÍTICAS DE SEGURIDAD DE LOS DATOS.

En el proyecto, se aplican rigurosamente las siguientes políticas para garantizar la seguridad de los datos manejados:

1. **Control de acceso:** El sistema utiliza roles y permisos definidos en Laravel para restringir el acceso a los datos según el perfil del usuario. La autenticación se realiza mediante contraseñas seguras y, cuando es necesario, se implementa autenticación multifactor para mayor protección. Las sesiones de usuario expiran automáticamente tras un período de inactividad para evitar accesos no autorizados.
2. **Confidencialidad:** Todos los datos sensibles, como las contraseñas, se almacenan cifrados utilizando algoritmos robustos (bcrypt). La comunicación entre el cliente y el servidor se realiza exclusivamente a través de HTTPS, garantizando que la información transmitida esté protegida contra interceptaciones.
3. **Integridad:** El sistema valida y sanitiza todas las entradas de datos para prevenir modificaciones no autorizadas o inyecciones maliciosas. Además, se realizan copias de seguridad periódicas de la base de datos MySQL para asegurar la recuperación de la información en caso de fallos o incidentes.
4. **Disponibilidad:** La infraestructura del proyecto está diseñada para mantener la disponibilidad continua del sistema, incluyendo planes de recuperación ante desastres y monitoreo constante para detectar y resolver posibles fallos o ataques que puedan afectar el servicio.
5. **Auditoría y monitoreo:** Se registran logs detallados de accesos y acciones relevantes dentro del sistema, almacenados de forma segura para facilitar auditorías y detectar actividades sospechosas. Estos registros se conservan durante un período definido para cumplir con políticas internas y normativas vigentes.
6. **Capacitación y concienciación:** Todo el equipo de desarrollo y administración recibe formación continua sobre buenas prácticas de seguridad, manejo adecuado de datos y cumplimiento normativo, asegurando que las políticas se apliquen correctamente.
7. **Cumplimiento legal:** El proyecto cumple con las leyes y regulaciones aplicables en materia de protección de datos personales, garantizando la privacidad y derechos de los usuarios.

10. CONSTRUCCIÓN DEL SOFTWARE

La construcción del software se llevó a cabo siguiendo un enfoque incremental e iterativo, permitiendo realizar entregas parciales y pruebas continuas para asegurar la calidad del producto. El proceso de desarrollo se estructuró en las siguientes fases:

1. Configuración del Entorno de Desarrollo

- Lenguaje de programación: PHP 8 con paradigma de Programación Orientada a Objetos.
- Framework Backend: Laravel 10, para gestionar rutas, controladores, modelos y migraciones de base de datos.
- Framework Frontend: Bootstrap 5, para la implementación de interfaces responsive y usables.
- Base de datos: MySQL, gestionando la persistencia de la información.
- Control de versiones: Git y repositorio en GitHub, para el trabajo colaborativo y control de cambios.
- Servidor de pruebas: Laravel Artisan y servidor local con XAMPP.

2. Desarrollo de Módulos Principales

- Módulo de Usuarios: Registro, inicio de sesión seguro, roles y gestión de perfiles.
- Módulo de Cursos y Lecciones: Creación, visualización y gestión de cursos con contenidos en texto y video.
- Módulo de Evaluaciones: Preguntas, respuestas y calificación automática del progreso de cada estudiante.
- Módulo de Progreso: Registro de avances en cada lección y retroalimentación.
- Módulo de Administración: Gestión de usuarios, cursos y control del sistema por parte de los administradores.

3. Prácticas de Calidad en la Construcción

10.1.1. OBJETOS DE LA BASE DE DATOS (PROCEDIMIENTOS ALMACENADOS, VISTAS, DISPARADORES)

1. Procedimientos almacenados (Stored Procedures)

Se implementan para encapsular operaciones repetitivas y mejorar la eficiencia:

- **sp_registrar_usuario:** Inserta un nuevo usuario en la tabla **usuarios** validando correo único.
- **sp_registrar_progreso:** Actualiza el avance del estudiante en la tabla **progreso_preguntas**.
- **sp_resultado_prueba:** Calcula el puntaje total obtenido en una prueba, sumando las respuestas correctas.

```
DELIMITER //
CREATE PROCEDURE sp_registrar_usuario (
    IN p_nombre VARCHAR(100),
    IN p_email VARCHAR(150),
    IN p_password VARCHAR(255),
    IN p_rol_id INT
)
BEGIN
    INSERT INTO usuarios (nombre, email, password, rol_id)
    VALUES (p_nombre, p_email, p_password, p_rol_id);
END //
DELIMITER ;
```

2. Vistas (Views)

Facilitan consultas frecuentes y reportes:

- **vw_progreso_estudiantes:** Muestra el avance de cada estudiante por curso.
- **vw_resultados_pruebas:** Devuelve el listado de usuarios con sus calificaciones en pruebas.
- **vw_cursos_activos:** Filtra los cursos disponibles y visibles para estudiantes.

```
CREATE VIEW vw_progreso_estudiantes AS  
SELECT u.nombre, c.nombre AS curso, p.porcentaje_avance  
FROM usuarios u  
JOIN curso_usuario cu ON u.id = cu.usuario_id  
JOIN cursos c ON cu.curso_id = c.id  
JOIN progreso_preguntas p ON u.id = p.usuario_id;
```

3. Disparadores (Triggers)

Automatizan validaciones y actualizaciones en la base de datos:

- **trg_registrar_fecha_usuario:** Al insertar un usuario, se guarda automáticamente la fecha de creación.
- **trg_actualizar_progreso:** Cada vez que un estudiante responde una pregunta, se actualiza su progreso.
- **trg_control Eliminacion:** Impide eliminar un curso si aún tiene estudiantes inscritos.

3. Disparadores (Triggers)

Automatizan validaciones y actualizaciones en la base de datos:

- **trg_registrar_fecha_usuario:** Al insertar un usuario, se guarda automáticamente la fecha de creación.
- **trg_actualizar_progreso:** Cada vez que un estudiante responde una pregunta, se actualiza su progreso.
- **trg_control Eliminacion:** Impide eliminar un curso si aún tiene estudiantes inscritos.

10.1.2. ESQUEMAS DE SEGURIDAD DE LOS DATOS.

El sistema implementa un conjunto de esquemas de seguridad que garantizan la protección de la información en los diferentes niveles (aplicación, base de datos y comunicación):

1. Seguridad en la Base de Datos

- **Usuarios y Roles en la BD:**
 - Se definen usuarios con privilegios diferenciados en MySQL (administrador, aplicación, solo lectura).
 - Se restringen permisos de *INSERT*, *UPDATE* o *DELETE* únicamente a los procesos autorizados.
- **Integridad referencial:**
 - Uso de claves primarias y foráneas para garantizar consistencia de la información.
- **Disparadores de auditoría:**
 - Los triggers registran acciones críticas (inserciones, modificaciones y eliminaciones).

2. Seguridad en la Aplicación

- **Autenticación y Autorización:**
 - Implementación de *middleware* en Laravel para restringir accesos según el rol del usuario.
 - Control de sesiones mediante tokens únicos.
- **Cifrado de contraseñas:**
 - Uso del algoritmo **bcrypt** para almacenar contraseñas de manera irreversible.
- **Validación de datos:**
 - Todas las entradas de formularios son validadas en el backend antes de ser insertadas en la base de datos.

3. Seguridad en la Comunicación

- **Protocolos seguros:**
 - Todo el tráfico entre cliente y servidor se realiza mediante **HTTPS/TLS**.
- **Protección contra ataques comunes:**
 - Filtros para prevenir **SQL Injection** (mediante consultas preparadas en Laravel).
 - Protección contra **XSS (Cross Site Scripting)** con escape de datos en Blade.
 - Prevención de **CSRF (Cross-Site Request Forgery)** con tokens de seguridad generados por Laravel.

4. Respaldo y Recuperación

- **Copias de seguridad automáticas** de la base de datos de manera periódica.
- **Estrategias de recuperación** para restaurar datos en caso de pérdida o daño.

10.2. CODIFICACIÓN DEL SOFTWARE DE ACUERDO CON EL DISEÑO ESTABLECIDO.

El proceso de codificación se desarrolló siguiendo los lineamientos planteados en la fase de diseño, aplicando principios de **programación orientada a objetos (POO)**, el uso de **frameworks modernos** y las convenciones de codificación recomendadas por la comunidad de desarrollo.

1. Estructuración del Código

- Se utilizó el framework **Laravel 10 (PHP)** para implementar el backend, organizando el proyecto en **Modelos, Vistas y Controladores (MVC)**.
- El frontend se desarrolló con **Blade Templates** y **Bootstrap 5**, garantizando interfaces amigables y responsivas.
- Se definieron **rutas web y API** para gestionar la navegación y comunicación entre módulos.

- La base de datos se construyó mediante **migraciones en Laravel**, garantizando portabilidad y consistencia del esquema.

Modelo de usuario

```
app > Models > Usuario.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Foundation\Auth\User as Authenticatable;
6  use Illuminate\Notifications\Notifiable;
7  use Illuminate\Database\Eloquent\Factories\HasFactory;
8
9  class Usuario extends Authenticatable
10 {
11     use HasFactory, Notifiable;
12
13     protected $table = 'usuarios';
14
15     protected $fillable = [
16         'nombre',
17         'apellido',
18         'email',
19         'password',
20         'vidas',
21         'experiencia',
22         'role_id',
23         'imagen',
24     ];
```

Migración de tabla cursos

```

database > migrations > 2025_06_13_000003_create_cursos_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('cursos', function (Blueprint $table) {
15             $table->id(); // Clave primaria obligatoria
16             $table->string('nombre')->unique();
17             $table->text('descripcion');
18             $table->timestamps();
19         });
20     }
21
22     /**
23      * Reverse the migrations.
24      */
25     public function down(): void
26     {
27         Schema::dropIfExists('cursos');
28     }
29 };
30

```

Controlador cursos

```
app > Http > Controllers > CursoController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use App\Models\Curso;
6  use Illuminate\Http\Request;
7
8  class CursoController extends Controller
9  {
10     /**
11      * Muestra todos los cursos.
12      */
13     public function index()
14     {
15         $cursos = Curso::all();
16         return view('CrudCursos.GestionarCurso', compact('cursos'));
17     }
18
19     /**
20      * Muestra el formulario para crear un curso.
21      */
22     public function create()
23     {
24         return view('CrudCursos.CrearCurso');
25     }
26
27     /**
28      * Almacena un nuevo curso.
29      */
30     public function store(Request $request)
31     {
32         $request->validate([
```

Vista blande cursos

```

resources > views > VistasEstudiante > cursos.blade.php
1  @extends('layouts.estudiante')
2  @section('title', 'Catalogo de Cursos')
3  @section('content')
4      <h2 class="mb-4 text-light">Catalogo Cursos</h2>
5      @if (session('error'))
6          <div class="alert alert-danger small">{{ session('error') }}</div>
7      @endif
8      <div class="row g-4">
9          @foreach ($cursos as $curso)
10             <div class="col-12 col-md-4">
11                 <div class="card course-card">
12                     nombre }}">
14                     <div class="card-body">
15                         <h5 class="card-title">{{ $curso->nombre }}</h5>
16                         <p class="card-text">{{ $curso->descripcion }}</p>
17                         <a href="{{ route('usuarios.caminoCurso', ['curso_id' => $curso->id]) }}"
18                             class="btn btn-primary w-100">Ir al curso</a>
19                         @if (in_array($curso->id, $cursosSeguidos))
20                             <a href="{{ route('usuarios.caminoCurso', ['curso_id' => $curso->id]) }}" class="
21                         @else
22                             <form method="POST" action="{{ route('usuarios.seguirCurso', ['curso_id' => $cu
23                                 class="mt-2">
24                                 @csrf
25                                 <button type="submit" class="btn btn-warning w-100">Seguir</button>
26                             </form>
27                         @endif
28                     </div>
29                 </div>
30             </div>
31         </div>
32     @endforeach

```

2. Convenciones de Programación

- Se aplicaron estándares **PSR-12** para PHP, lo que asegura legibilidad y mantenibilidad.
- El código se documentó con **comentarios estructurados** y uso de PHPDoc.
- Se siguió el principio **DRY (Don't Repeat Yourself)** para evitar duplicación de código.
- Los controladores mantienen la lógica mínima, delegando procesos complejos a **Servicios y Modelos**.

3. Implementación de Funcionalidades

- **Gestión de Usuarios y Roles:** autenticación, registro y autorización con middleware de Laravel.
- **Gestión de Cursos y Lecciones:** creación, edición y visualización de contenidos.

- **Sistema de Evaluaciones:** preguntas, respuestas y calificación automática.
- **Seguimiento de Progreso:** actualización de avances y retroalimentación al estudiante.
- **Panel de Administración:** herramientas de gestión para administradores.

4. Buenas Prácticas de Codificación

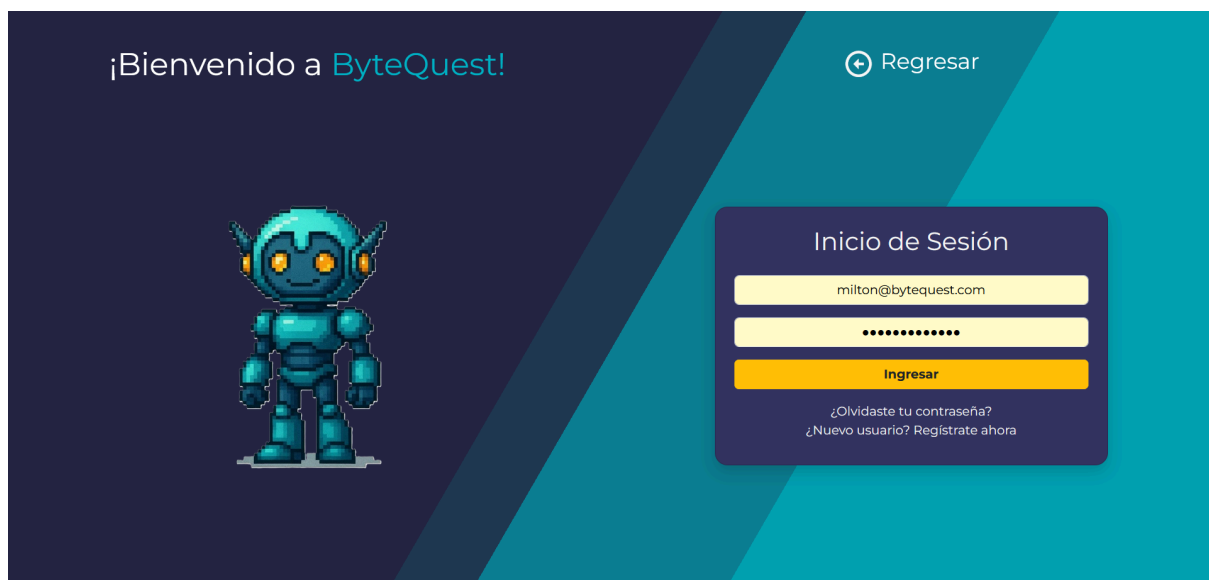
- Uso de **control de versiones (Git/GitHub)** para registrar cambios y trabajo colaborativo.
- Implementación de **pruebas unitarias y funcionales** con PHPUnit.
- Validación de formularios y datos en el backend antes de ser persistidos en la base de datos.
- Modularización del código para facilitar mantenibilidad y escalabilidad.

5. Relación con el Diseño Establecido

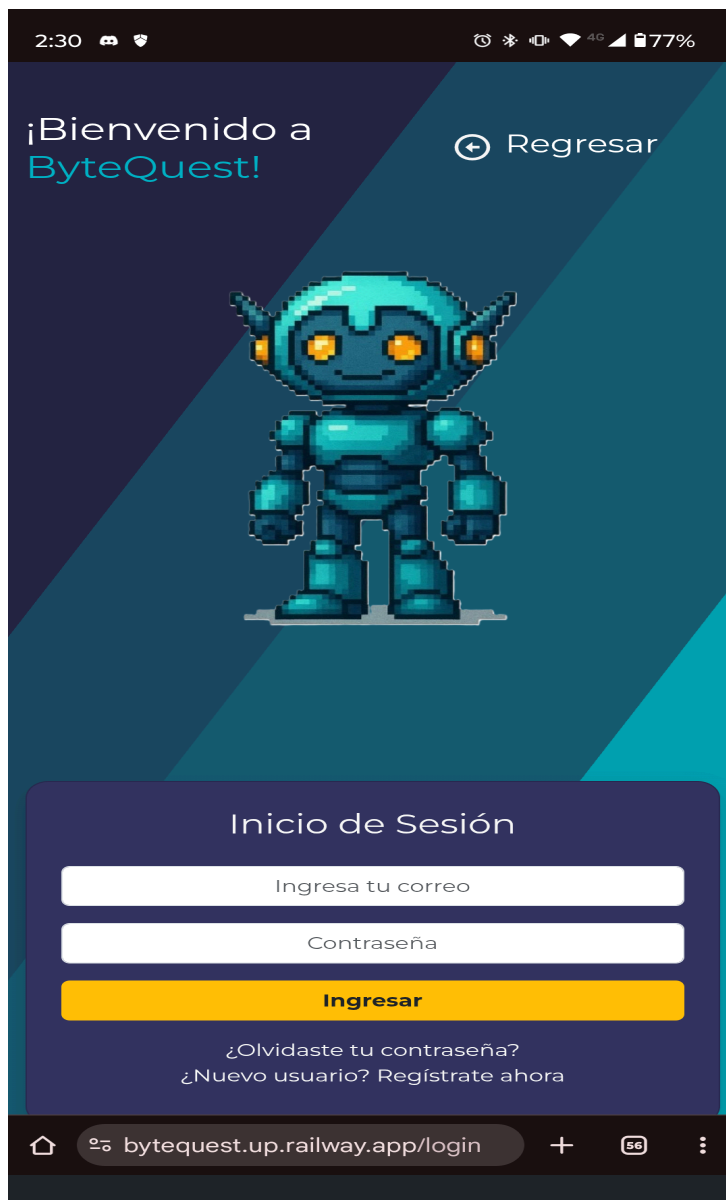
La codificación se realizó conforme a los **diagramas de clases, casos de uso, modelo entidad-relación y diagramas de despliegue** definidos en la fase de diseño. Esto aseguró coherencia entre la planeación y la implementación final del sistema.

10.2.1. FRONT END (HTML, CSS, JAVASCRIPT, RESPONSIVE)

HTML



Responsive



CSS

```
@import url("https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap");
```

```
* {
  font-family: Montserrat, Arial, sans-serif !important;
  font-weight: 300 !important;
}
```

```
h1,
```

```
h2,
```

```
h3,  
h4,  
h5,  
h6 {  
    font-weight: 300 !important;  
}
```

```
html {  
    height: 100%;  
}
```

```
body {  
    margin: 0;  
    overflow-y: auto;  
    min-width: 100vh;  
    max-height: none;  
}
```

```
.bg {  
    animation: slide 3s ease-in-out infinite alternate;  
    background-image: linear-gradient(-60deg, #00b2c3 50%, #252746 50%);  
    bottom: 0;  
    left: -50%;  
    opacity: 0.5;  
    position: fixed;  
    right: -50%;
```



```
top: 0;

z-index: -1;

}


.bg2 {

  animation-direction: alternate-reverse;

  animation-duration: 4s;

}


.bg3 {

  animation-duration: 5s;

}


.content {

  background-color: rgba(255, 255, 255, 0.8);

  border-radius: 0.25em;

  box-shadow: 0 0 0.25em rgba(0, 0, 0, 0.25);

  box-sizing: border-box;

  left: 50%;

  padding: 10vmin;

  position: fixed;

  text-align: center;

  top: 50%;

  transform: translate(-50%, -50%);

}
```

```

@keyframes slide {
  0% {
    transform: translateX(-25%);
  }
  100% {
    transform: translateX(25%);
  }
}

.circle {
  color: whitesmoke;
}

.card {
  color: whitesmoke;
  background-color: #333661;
}

.links a {
  color: whitesmoke;
}

```

10.2.2. ESTÁNDAR DE CODIFICACIÓN (DOCUMENTACIÓN DEL CÓDIGO, APLICABILIDAD DEL ESTÁNDAR DE CODIFICACIÓN).

En el proyecto se sigue un estándar riguroso de documentación del código para asegurar su mantenibilidad, comprensión y calidad. Esto incluye:

Comentarios claros y concisos:

Cada función, método y clase cuenta con comentarios que describen su propósito, parámetros, valores de retorno y cualquier comportamiento especial. Se utilizan comentarios en formato PHPDoc para facilitar la generación automática de documentación.

Nomenclatura consistente:

Se emplean nombres descriptivos y coherentes para variables, funciones, clases y archivos, siguiendo las convenciones de Laravel y PSR (PHP Standards Recommendations).

Estructura organizada:

El código se organiza en carpetas y archivos según la arquitectura MVC de Laravel, separando claramente la lógica de negocio, la presentación y el acceso a datos.

Uso de control de versiones:

Todo el código se gestiona mediante un sistema de control de versiones (Git), con mensajes de commit claros que describen los cambios realizados.

Aplicabilidad del estándar de codificación**Lenguaje y framework:**

El estándar se aplica a todo el código PHP desarrollado con Laravel, así como a los archivos JavaScript y CSS que utilizan Bootstrap para la interfaz.

Consistencia en todo el equipo:

Todos los desarrolladores del proyecto deben seguir este estándar para garantizar uniformidad y facilitar la colaboración.

Revisión de código:

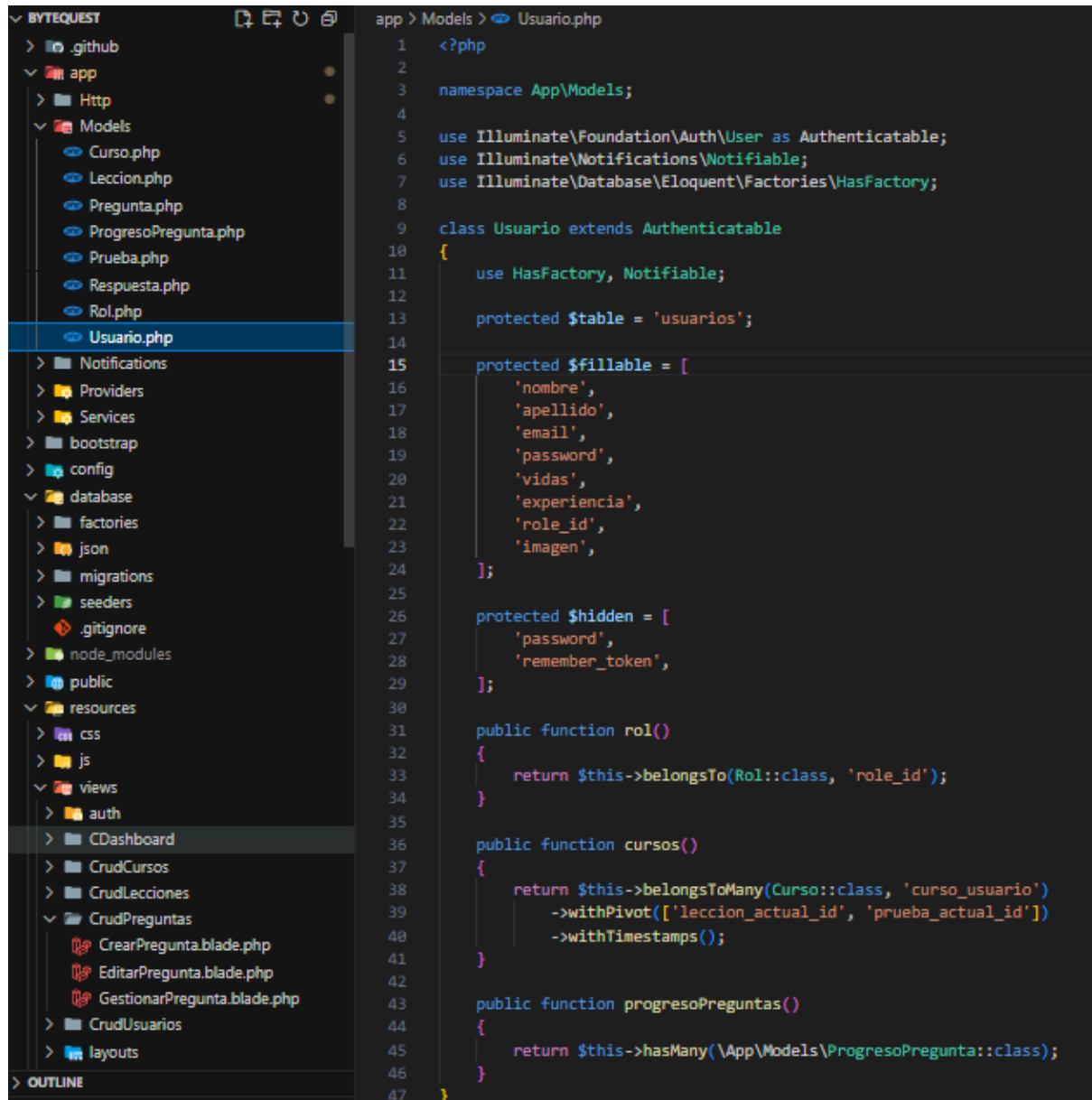
Se implementan revisiones de código (code reviews) para verificar el cumplimiento del estándar antes de integrar cambios al repositorio principal.

Automatización:

Se utilizan herramientas de análisis estático y linters (por ejemplo, PHP_CodeSniffer con reglas PSR-12) para detectar desviaciones del estándar y mejorar la calidad del código.

10.2.3. CÓDIGO FUENTE DE LOS MÓDULOS DEL SOFTWARE WEB Y MÓVIL.

1. Módulo Web – Gestión de Usuarios



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, with the 'app' directory expanded to show 'Models'. The 'Usuario.php' file is selected. The code editor shows the source code of the 'Usuario' model, which extends 'Authenticatable' and uses 'HasFactory' and 'Notifiable' traits. The code includes attributes like 'nombre', 'apellido', 'email', 'password', 'vidas', 'experiencia', 'role_id', and 'imagen'. It also includes methods for 'rol()', 'cursos()', and 'progresoPreguntas()'.

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Foundation\Auth\User as Authenticatable;
6  use Illuminate\Notifications\Notifiable;
7  use Illuminate\Database\Eloquent\Factories\HasFactory;
8
9  class Usuario extends Authenticatable
10 {
11     use HasFactory, Notifiable;
12
13     protected $table = 'usuarios';
14
15     protected $fillable = [
16         'nombre',
17         'apellido',
18         'email',
19         'password',
20         'vidas',
21         'experiencia',
22         'role_id',
23         'imagen',
24     ];
25
26     protected $hidden = [
27         'password',
28         'remember_token',
29     ];
30
31     public function rol()
32     {
33         return $this->belongsTo(Rol::class, 'role_id');
34     }
35
36     public function cursos()
37     {
38         return $this->belongsToMany(Curso::class, 'curso_usuario')
39             ->withPivot(['leccion_actual_id', 'prueba_actual_id'])
40             ->withTimestamps();
41     }
42
43     public function progresoPreguntas()
44     {
45         return $this->hasMany(App\Models\ProgresoPregunta::class);
46     }
47 }

```

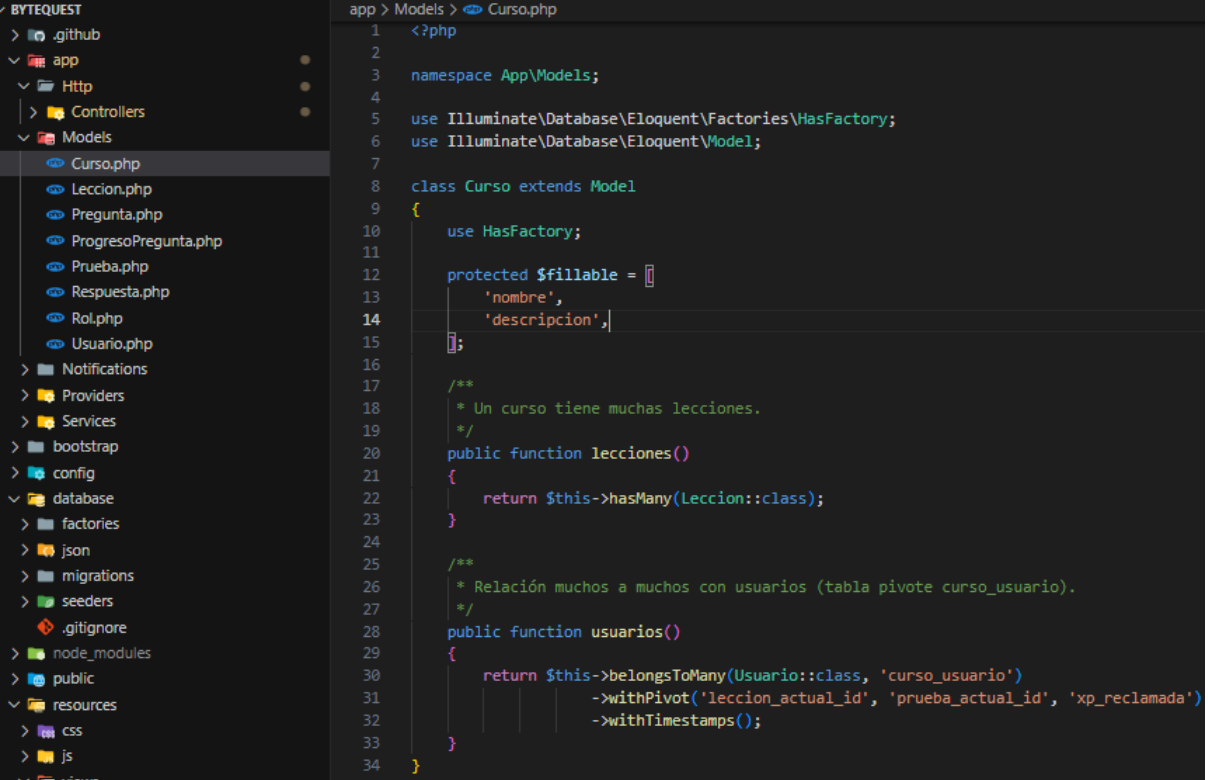
Controlador de Autenticación (app/Http/Controllers/AuthController.php)

```

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6  use Illuminate\Support\Facades\Auth;
7  use Illuminate\Support\Facades\Hash;
8  use App\Models\Usuario;
9  use Illuminate\Support\Facades\Validator;
10 use Illuminate\Support\Str;
11 use Illuminate\Support\Facades\DB;
12 use App\Notifications\ResetPasswordNotification;
13
14 class AuthController extends Controller
15 {
16     public function showLogin()
17     {
18         return view('auth.login');
19     }
20
21     public function login(Request $request)
22     {
23         $credentials = $request->validate([
24             'email' => ['required', 'email'],
25             'password' => [
26                 'required',
27                 'string',
28                 'min:8'
29                 // Al menos una mayúscula y un número
30             ],
31         ]);

```

Módulo Web – Cursos

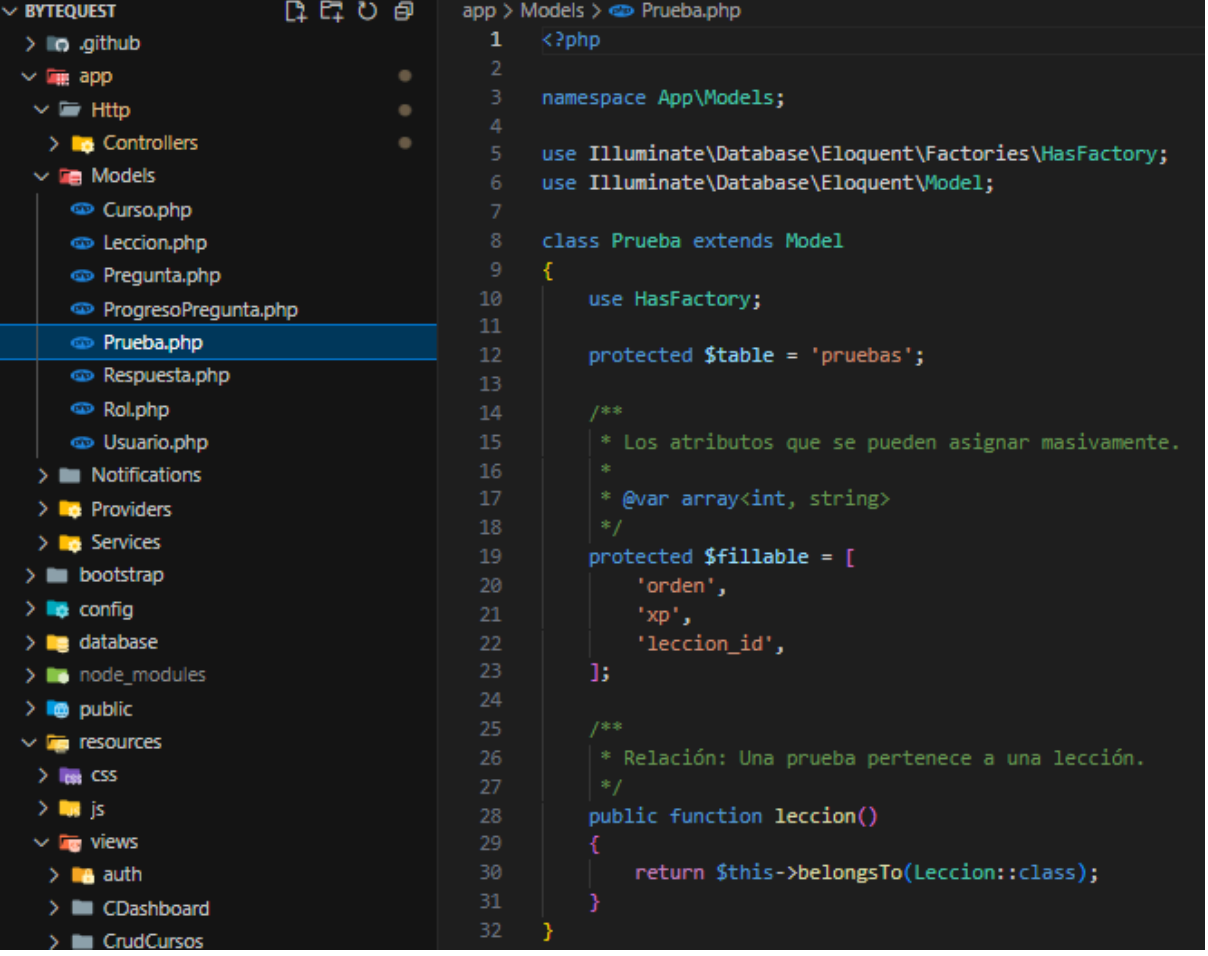


```

BYTEQUEST
> .github
> app
  > Http
  > Controllers
  > Models
    > Curso.php
    > Leccion.php
    > Pregunta.php
    > ProgresoPregunta.php
    > Prueba.php
    > Respuesta.php
    > Rol.php
    > Usuario.php
  > Notifications
  > Providers
  > Services
  > bootstrap
  > config
  > database
  > factories
  > json
  > migrations
  > seeders
  > .gitignore
  > node_modules
  > public
  > resources
    > css
    > js
  > views

app > Models > Curso.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class Curso extends Model
9  {
10     use HasFactory;
11
12     protected $fillable = [
13         'nombre',
14         'descripcion',
15     ];
16
17     /**
18      * Un curso tiene muchas lecciones.
19      */
20     public function lecciones()
21     {
22         return $this->hasMany(Leccion::class);
23     }
24
25     /**
26      * Relación muchos a muchos con usuarios (tabla pivote curso_usuario).
27      */
28     public function usuarios()
29     {
30         return $this->belongsToMany(Usuario::class, 'curso_usuario')
31             ->withPivot('leccion_actual_id', 'prueba_actual_id', 'xp_reclamada')
32             ->withTimestamps();
33     }
34 }
  
```

3. Módulo Web – Evaluaciones



The image shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the structure of a Laravel application, with the 'Models' directory expanded. The code editor shows the code for the 'Prueba.php' model.

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7
8  class Prueba extends Model
9  {
10     use HasFactory;
11
12     protected $table = 'pruebas';
13
14     /**
15      * Los atributos que se pueden asignar masivamente.
16      *
17      * @var array<int, string>
18      */
19     protected $fillable = [
20         'orden',
21         'xp',
22         'leccion_id',
23     ];
24
25     /**
26      * Relación: Una prueba pertenece a una lección.
27      */
28     public function leccion()
29     {
30         return $this->belongsTo(Leccion::class);
31     }
32 }

```

10.2.4. SERVICIOS WEB.

Los servicios web implementados en **ByteQuest** permiten la comunicación entre el frontend (desarrollado en **Bootstrap** y **JavaScript**) y el backend (**Laravel**) mediante una arquitectura **RESTful API**.

Características principales

- **Protocolo:** HTTP/HTTPS
- **Formato de datos:** JSON (JavaScript Object Notation)
- **Métodos soportados:**
 - **GET:** Consulta de información (usuarios, cursos, lecciones, preguntas, progreso).
 - **POST:** Registro de datos (usuarios, respuestas, progreso de aprendizaje).
 - **PUT/PATCH:** Actualización de registros (perfil de usuario, edición de cursos).

- **DELETE:** Eliminación de registros (por ejemplo, cursos o respuestas en pruebas).

Endpoints principales

- **Autenticación y usuarios:**
 - POST /api/login → Inicio de sesión.
 - POST /api/register → Registro de usuario.
 - GET /api/user/{id} → Consultar información de usuario.
- **Cursos y lecciones:**
 - GET /api/cursos → Listado de cursos.
 - GET /api/curso/{id}/lecciones → Lecciones de un curso específico.
 - POST /api/curso → Crear un curso (rol administrador).
- **Pruebas y preguntas:**
 - GET /api/pruebas/{id} → Consultar prueba.
 - GET /api/preguntas/{id} → Consultar preguntas.
 - POST /api/respuesta → Enviar respuesta de usuario.
- **Progreso:**
 - GET /api/progreso/{usuario_id} → Consultar progreso del estudiante.
 - POST /api/progreso → Registrar avance.

Seguridad en servicios web

- **Autenticación por tokens JWT (JSON Web Token).**
- **Roles y permisos** para controlar acceso a endpoints (administrador, estudiante, docente).

- **Validación de datos** en todas las solicitudes para evitar inyecciones y accesos indebidos.
- **CORS habilitado** para controlar los orígenes permitidos.

10.2.5. CONTROL DE VERSIONES

Para el desarrollo del software **ByteQuest** se implementó un sistema de **control de versiones** con el fin de garantizar la trazabilidad, integridad y organización del código fuente durante todo el ciclo de vida del proyecto.

Herramienta utilizada

- **Git** como sistema de control de versiones distribuido.
- **GitHub** como repositorio remoto para almacenar y gestionar el código en la nube.

Estrategia de ramificación (branching)

- **Rama principal (main/master):** contiene la versión estable y lista para despliegue.
- **Rama de desarrollo (develop):** integra las nuevas funcionalidades antes de pasar a producción.
- **Ramas de funcionalidad (feature):** creadas para el desarrollo de nuevas características específicas.
- **Ramas de corrección (hotfix):** usadas para resolver errores críticos detectados en producción.
- **Ramas de prueba (test):** dedicadas a pruebas unitarias y de integración antes de integrar al desarrollo.

10.2.6. PRUEBAS DEL SOFTWARE

10.2.6.1. PLANEACIÓN Y DISEÑO DE LA PRUEBAS A NIVEL UNITARIO(MÓDULOS), CON BASE EN LO REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES.

10.2.6.2. EJECUCIÓN DE LAS PRUEBAS, INFORME DE HALLAZGOS

1.RF1-01

Selenium IDE - bytequest*

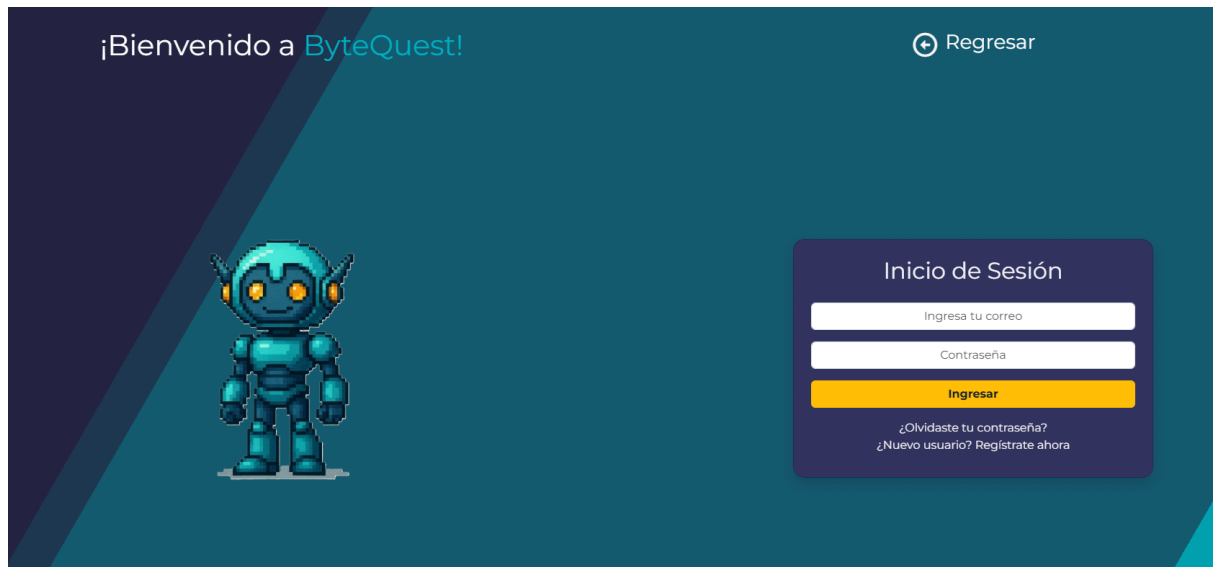
Project: bytequest*

Tests +

Search tests...

http://127.0.0.1:8000

Command	Target	Value
1 open	/	
2 set window size	1936x1048	
3 click	linkText=Empezar	
4 click	id=nombre	
5 type	id=nombre	Luis
6 click	id=apellido	
7 type	id=apellido	Diaz
8 click	id=email	
9 type	id=email	luisdiaz@gmail.com
10 click	id=password	
11 type	id=password	Prueba123!
12 type	id=password_confirmation	Prueba123!
13 send keys	id=password_confirmation	\$(KEY_ENTER)



2. RF1-02

Selenium IDE - bytequest*

Project: bytequest*

Tests +

Search tests...

http://127.0.0.1:8000

Command	Target	Value
1 open	/	
2 set window size	1936x1048	
3 click	linkText=Empezar	
4 click	id=nombre	
5 type	id=nombre	Luis
6 click	id=apellido	
7 type	id=apellido	Diaz
8 click	id=email	
9 type	id=email	luisdiaz@gmail.com
10 click	id=password	
11 type	id=password	prueba123
12 type	id=password_confirmation	prueba123
13 send keys	id=password_confirmation	\$(KEY_ENTER)

Command: send keys

Target: id=password_confirmation

Value: \$(KEY_ENTER)

Description:

Log Reference

 Regresar

- Este correo ya está registrado.
- La contraseña debe tener entre 8 y 15 caracteres, incluir mayúsculas, minúsculas, números y un carácter especial.

Registrarse

Nombre:

Apellido:

Correo:

Contraseña:

Confirmar Contraseña:

¿Ya tienes cuenta? Inicia sesión

Selenium IDE - bytequest*

Project: bytequest*

Tests +

Search tests...

http://127.0.0.1:8000

	Command	Target	Value
1.	open	/login	
2.	set window size	1936x1048	
3.	click	name=email	
4.	type	name=email	luisdaz@gmail.com
5.	click	name=password	
6.	type	name=password	Prueba123!
7.	send keys	name=password	\$(KEY_ENTER)

ByteQuest

Mis Cursos

Racha: 0 días

Vidas: 5

Luis

Aprender

Mis Cursos

Cursos

Tienda

Perfil

Salir

Mis Cursos

Aún no has seguido ningún curso.

Ver Cursos Disponibles

4. RF1-04

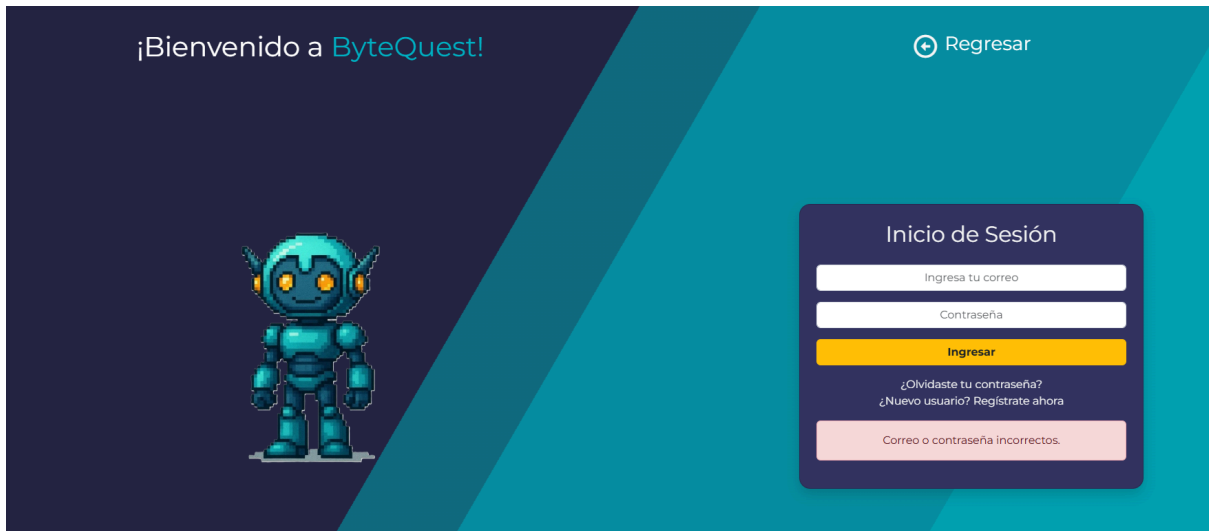
Project: bytequest*

Tests +

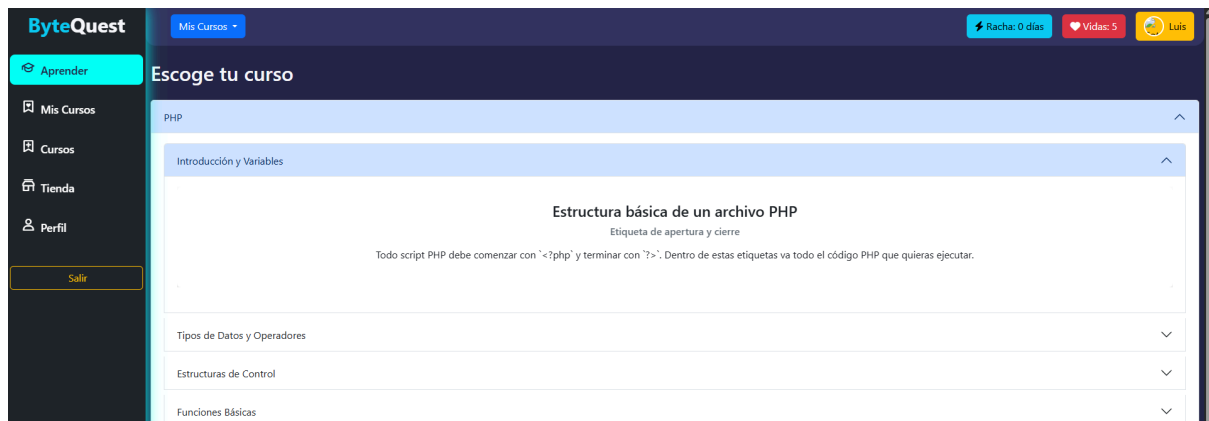
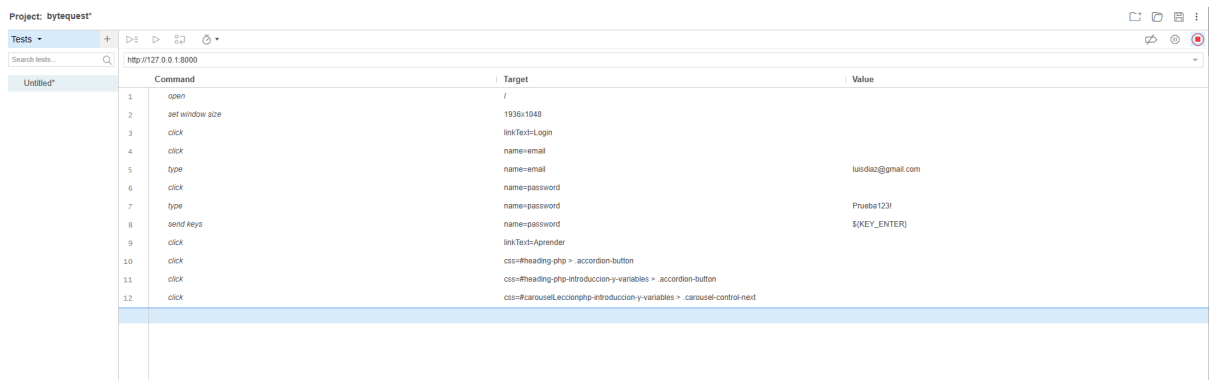
Search tests...

http://127.0.0.1:8000

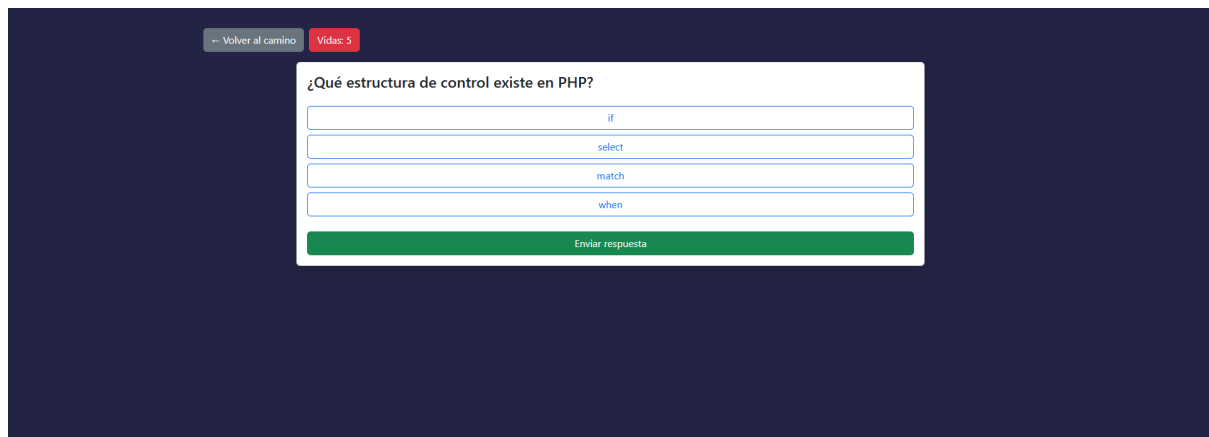
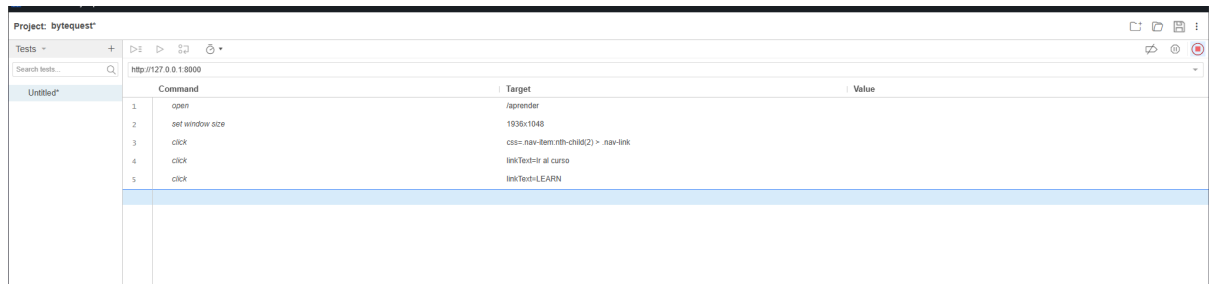
	Command	Target	Value
1.	open	/login	
2.	set window size	1936x1048	
3.	click	css= btn-outline-warning	
4.	click	name=email	
5.	type	name=email	luisdaz@gmail.com
6.	click	name=password	
7.	type	name=password	prueba123!
8.	click	css= btn	



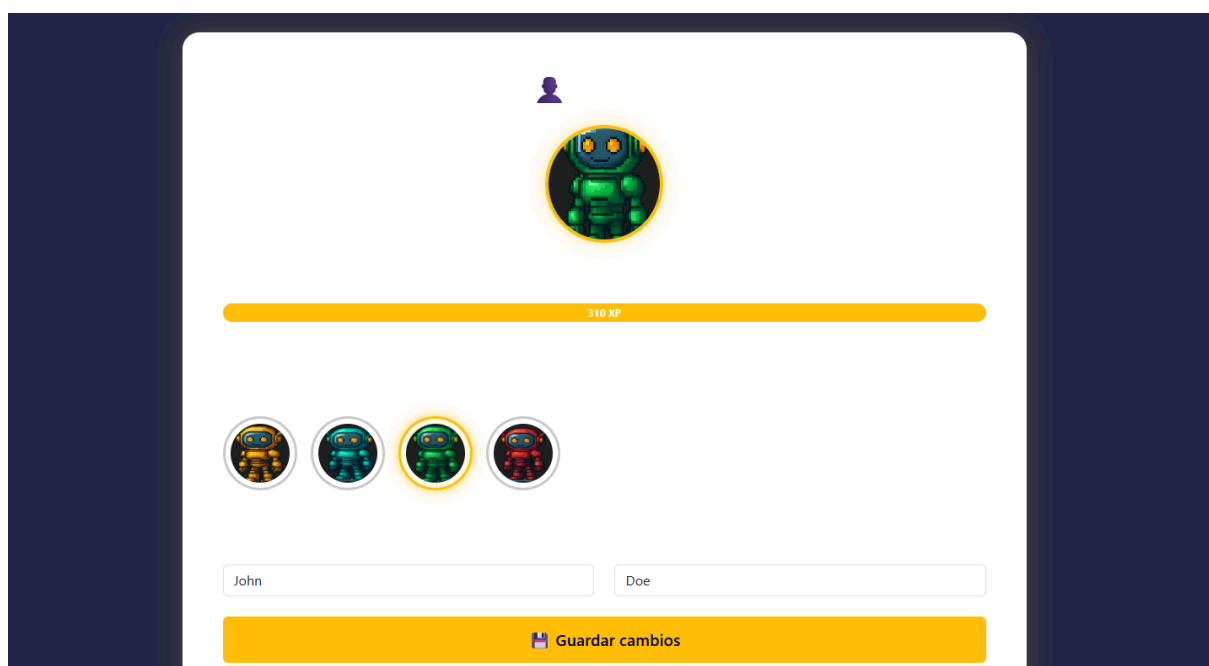
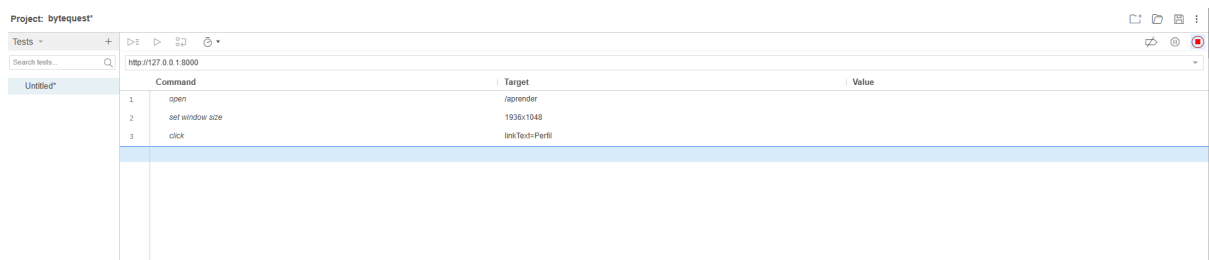
5. RF2-01



6.RF2-02 - 7. RF3-01



8. RF3-02 9.RF4-01 10.RF4-02 11.RF5-01



12. RF5-02 13. RF6-01

Project: bytequest

Tests +

Search tests...

http://127.0.0.1:8000

Command	Target	Value
1. open	/	
2. set window size	1936x1048	
3. click	linkText=Login	
4. click	name=email	
5. type	name=email	admin@bytequest.com
6. type	name=password	12345678
7. send keys	name=password	{KEY_ENTER}
8. mouse over	css= row:nth-child(1) > button-card:nth-child(1) .card-title	
9. mouse out	css= row:nth-child(1) > button-card:nth-child(1) .card-title	
10. click	css= button-card:nth-child(3) .card-title	
11. click	linkText=Añadir lección	
12. click	id=curso_id	
13. select	id=curso_id	label=PHP
14. click	id=nombre	
15. type	id=nombre	Leccion de prueba
16. type	id=descripcion	Prueba
17. click	css= btn-success	
18. click	css=#heading-php > accordion-button	

9	Buenas Prácticas	Aprende prácticas recomendadas para escribir código limpio, organizado y fácil de mantener.	
10	Repaso y Nivel de Dominio	Repasa todos los conceptos clave a través de preguntas mezcladas y mini retos para afianzar tus conocimientos.	
31	Leccion de prueba	Prueba	

14. RF6-02

Project: bytequest

Tests +

Search tests...

http://127.0.0.1:8000

Command	Target	Value
1. open	/views/dashboard	
2. set window size	1936x1048	
3. click	css= button-card:nth-child(3) .card-title	
4. click	css=#heading-php > accordion-button	
5. choose ok on next confirmation		
6. click	css=tr:nth-child(11) form > btn	
7. assert confirmation	¿Estás seguro de eliminar esta lección?	
8. webdriver choose ok on visible confirmation		
9. click	css=#heading-php > accordion-button	

15. RF7-01 16.RF7-02

PHP			
#	Nombre	Descripción	Acciones
1	Introducción y Variables	Aprende qué es PHP, cómo funciona, y domina la declaración y uso de variables para almacenar y manipular datos básicos.	
2	Tipos de Datos y Operadores	Explora los tipos de datos en PHP: strings, números, booleanos y cómo combinarlos con operadores aritméticos y lógicos.	
3	Estructuras de Control	Aprende a usar condicionales (if, else) y bucles (for, while) para controlar el flujo de tus programas PHP.	
4	Funciones Básicas	Define y usa funciones para organizar tu código y reutilizar bloques lógicos de forma eficiente.	
5	Arreglos (Arrays)	Trabaja con arrays para almacenar múltiples valores y manipular listas de datos en PHP.	
6	Strings y Manipulación de Texto	Aprende a manipular cadenas de texto con funciones integradas de PHP para crear mensajes dinámicos.	
7	Formularios y Datos del Usuario	Envía y recibe datos de formularios HTML, procesa entradas del usuario de forma segura.	
8	Manejo de Errores	Identifica y corrige errores comunes, implementa manejo básico de excepciones.	
9	Buenas Prácticas	Aprende prácticas recomendadas para escribir código limpio, organizado y fácil de mantener.	
10	Repaso y Nivel de Dominio	Repasa todos los conceptos clave a través de preguntas mezcladas y mini retos para afianzar tus conocimientos.	

10.2.6.3. CORRECCIÓN DE ERRORES, Y DOCUMENTACIÓN DE LOS MISMOS.

10.2.6.4. MANEJO DE ALERTAS / EXCEPCIONES

10.2.6.5. PLANEACIÓN Y DISEÑO DE LAS PRUEBAS A NIVEL SISTEMA, CON BASE EN LO REQUERIMIENTOS

FUNCIONALES Y NO FUNCIONALES.

Requerimientos Funcionales

Caso de Prueba	Descripción	Esperado	Obtenido
RF1-01	Registro con datos válidos	Registro exitoso	
RF1-02	Registro con correo inválido	Mensaje de error	
RF1-03	Inicio de sesión correcto	Acceso al sistema	
RF1-04	Inicio con credenciales erróneas	Error de autenticación	
RF2-01	Visualizar lección en texto	Texto legible sin errores	
RF2-02	Visualizar lección en video	Video se reproduce correctamente	
RF3-01	Completar cuestionario válido	Retroalimentación inmediata	
RF3-02	Intentar enviar sin responder	Mensaje de advertencia	
RF4-01	Mostrar puntaje final	Puntaje correcto en pantalla	
RF4-02	Mostrar retroalimentación	Comentarios correctos por pregunta	
RF5-01	Ver progreso acumulado	Barra de progreso actualizada	
RF5-02	Historial de actividades	Lista de actividades completadas	
RF6-01	Crear nueva lección (admin)	Lección almacenada en BD	
RF6-02	Eliminar cuestionario (admin)	Eliminación exitosa	

Requerimientos No Funcionales

Caso de Prueba	Descripción	Esperado	Obtenido
RNF1-01	Evaluar usabilidad con usuarios	Interfaz intuitiva	
RNF2-01	Medir tiempo de carga de login	¡2 segundos	
RNF2-02	Medir carga de cuestionario	¡2 segundos	
RNF3-01	Prueba en Chrome	Correcta visualización	
RNF3-02	Prueba en Firefox	Correcta visualización	
RNF3-03	Prueba en Edge	Correcta visualización	
RNF4-01	Caída del servidor BD simulada	Sistema se recupera	
RNF5-01	Envío de datos personales	Cifrado HTTPS	
RNF6-01	100 usuarios simultáneos	Sin degradación grave	

10.2.6.7. CORRECCIÓN DE ERRORES, Y DOCUMENTACIÓN DE LOS MISMOS.

Proceso de corrección de errores

Detección y reporte:

Los errores o fallos en el sistema son detectados mediante pruebas automatizadas, monitoreo en producción, o reportes de usuarios. Cada error se registra en un sistema de seguimiento de incidencias (por ejemplo, Jira, GitHub) con detalles claros sobre su naturaleza, pasos para reproducirlo y prioridad.

Asignación y análisis:

El equipo de desarrollo asigna el error a un responsable, quien analiza la causa raíz y evalúa el impacto en el sistema.

Corrección:

Se desarrolla la solución siguiendo los estándares de codificación establecidos, asegurando que la corrección no introduzca nuevos errores. Se realizan pruebas unitarias y de integración para validar la corrección.

Revisión y aprobación:

La corrección es revisada mediante code review para garantizar calidad y cumplimiento de estándares antes de ser integrada al código base.

Despliegue:

La solución se despliega en el entorno correspondiente (pruebas o producción) siguiendo los procedimientos establecidos para minimizar riesgos.

Documentación de errores y correcciones**Registro detallado:**

Cada error corregido se documenta en el sistema de seguimiento con la descripción del problema, análisis realizado, solución implementada y resultados de las pruebas.

Historial de cambios:

En el repositorio de código, los commits relacionados con la corrección incluyen mensajes claros que referencian el error corregido y resumen la solución aplicada.

Lecciones aprendidas:

Cuando procede, se documentan recomendaciones o mejoras para evitar la recurrencia del error, contribuyendo a la mejora continua del proyecto.

11. PLAN DE DESPLIEGUE (HOSTING, DOMINIO, SUBDOMINIO, COSTOS, HERRAMIENTAS, PLATAFORMA, SEGURIDAD)

PLAN DE DESPLIEGUE: BYTEQUEST

1. HOSTING

Plataforma: Railway

Tipo: PaaS (Platform as a Service)

URL Actual: `bytequest.railway.app`

Ventajas:

Despliegues automáticos desde GitHub

Escalado automático

SSL/TLS gratuito

Variables de entorno gestionadas

2. DOMINIO Y SUBDOMINIO

Dominio Principal: (Por definir)

Subdominio Actual: `bytequest.railway.up.app`

Configuración DNS:

Registro CNAME apuntando a `railway.app`

3. COSTOS

Concepto	Costo	Periodicidad
Railway (Free Tier)	\$0 USD	Mensual
Dominio Personalizado	~\$12 USD	Anual

Base de Datos (PostgreSQL Starter) \$7 USD Mensual

4. HERRAMIENTAS Y PLATAFORMA

Stack Tecnológico:

PHP 8.2+

Laravel 10+

PostgreSQL

Nginx (proporcionado por Railway)

Herramientas:

GitHub para control de versiones

Railway CLI para gestión

Composer para dependencias

5. SEGURIDAD

SSL: Certificado automático proporcionado por Railway

Variables de Entorno: Gestionadas en dashboard de Railway

Claves de Aplicación: Laravel APP_KEY en variables de entorno

Headers de Seguridad: Configurados en Laravel

Backups: Backup automático de base de datos (depende del plan)

6. PROCESO DE DESPLIEGUE

Push a rama main en GitHub → Despliegue automático

Railway ejecuta:

```
composer install
```

```
php artisan migrate --force (si está configurado)
```

```
php artisan optimize
```

Variables de entorno inyectadas automáticamente

7. CONFIGURACIÓN ACTUAL

Runtime: PHP 8.2

Web Server: Nginx

Build Command: `composer install --no-dev --optimize-autoloader`

Start Command: `php artisan serve --port=$PORT`

Health Check Path: /

8. PRÓXIMOS PASOS

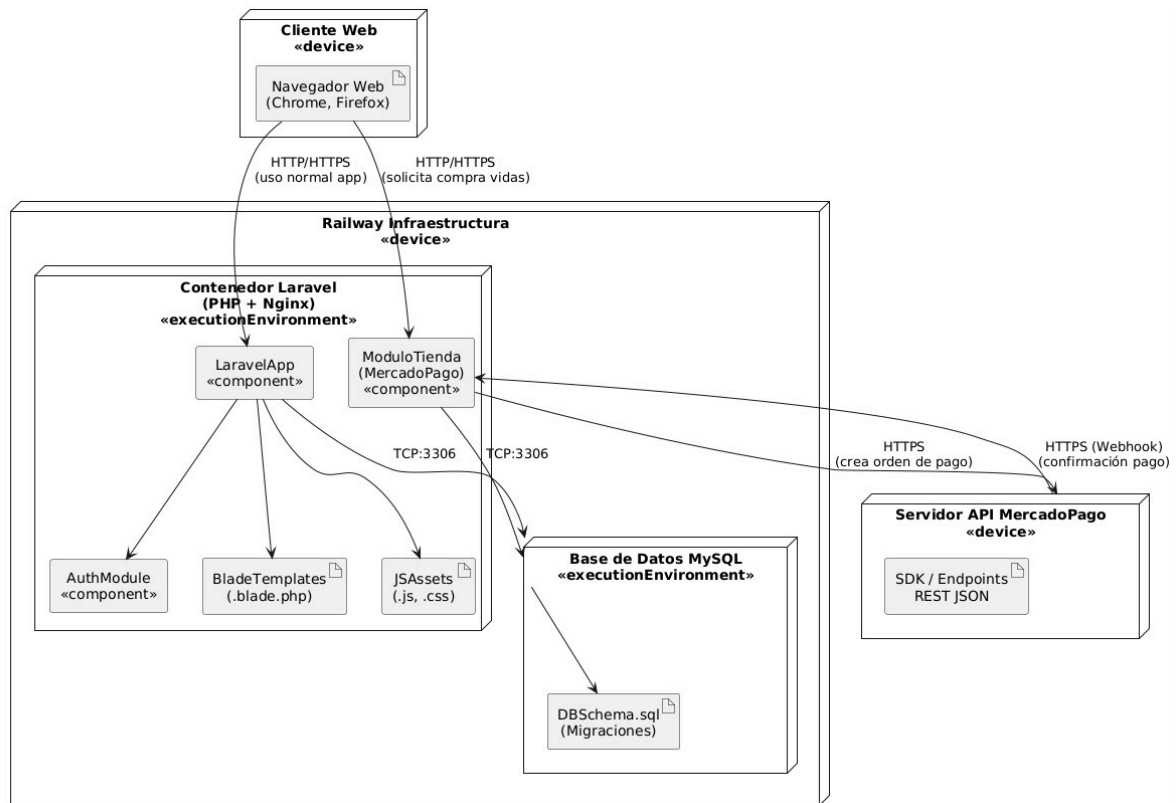
Adquirir dominio personalizado

Configurar DNS para dominio principal

Migrar a plan pago si se requiere más recursos

Configurar backup automático de base de datos

12. IMPLANTACIÓN DEL SOFTWARE.



12.1. PLAN DE IMPLANTACIÓN (PLATAFORMAS TECNOLÓGICAS)

12.2. PLAN DE CAPACITACIÓN DE USUARIOS DEL SISTEMA.

[Plan de capacitacion](#) [Capcitacion usuario](#)

12.3. MANUAL DEL USUARIO (PDF, MÓDULO AYUDA)

[Manual de usuario](#)

12.4. COPIAS DE SEGURIDAD DE DATOS Y RESPALDOS

Con el fin de garantizar la disponibilidad, integridad y continuidad del servicio de ByteQuest, se estableció un plan de copias de seguridad y respaldos que permite recuperar la información en caso de fallas, pérdida o ataques de seguridad.

Objetivos

- Proteger la base de datos y los archivos del sistema ante fallas de hardware, software o ciberataques.
- Asegurar la continuidad del servicio reduciendo el tiempo de inactividad.

- Facilitar la restauración rápida de datos en caso de incidentes.

Tipos de copias de seguridad

1. Copias completas: respaldo total de la base de datos y archivos del sistema. Se realizan semanalmente.
2. Copias incrementales: respaldos que solo incluyen los datos modificados desde la última copia completa. Se realizan diariamente.
3. Copias diferenciales: respaldo de los datos cambiados desde la última copia completa. Se realizan cada tres días como medida intermedia.

Frecuencia de respaldos

- Diaria: respaldo incremental de la base de datos MySQL (usuarios, cursos, lecciones, progreso, respuestas).
- Semanal: copia completa del sistema (archivos de configuración, repositorio de código, base de datos).
- Mensual: copia almacenada en un repositorio externo o servicio en la nube (Google Cloud / AWS S3).

Almacenamiento de respaldos

- Local: servidor físico de la institución con almacenamiento redundante (RAID).
- Remoto: almacenamiento en la nube para garantizar disponibilidad externa.
- Medio externo: discos duros externos o cintas magnéticas para casos de recuperación crítica.

Políticas de seguridad en respaldos

- Encriptación de copias para proteger información sensible.
- Control de accesos: solo administradores de sistemas autorizados pueden acceder a respaldos.

- Verificación periódica de la integridad de las copias.
- Política de retención:
 - Copias diarias → 7 días.
 - Copias semanales → 1 mes.
 - Copias mensuales → 6 meses.

Procedimiento de restauración

1. Identificación de la copia más reciente disponible.
2. Verificación de integridad de la copia.
3. Restauración en un entorno de pruebas antes de producción.
4. Recuperación total o parcial según la necesidad.
5. Documentación del incidente y medidas preventivas.

12.5. GARANTÍA (CONTRATOS, ACUERDOS DE NIVEL DE USUARIO)

Contrato

13. ADOPCIÓN DE BUENAS PRÁCTICAS EN EL PROCESO DE DESARROLLO DE SOFTWARE.

La calidad y sostenibilidad del software desarrollado en ByteQuest se soporta en la implementación de buenas prácticas a lo largo de todo el ciclo de vida del desarrollo. Estas garantizan confiabilidad, escalabilidad, seguridad y facilidad de mantenimiento.

13.1. Metodologías de Desarrollo

- Se adoptó la metodología ágil (Scrum) para gestionar iteraciones cortas e incrementales.
- Se prioriza la comunicación constante con los usuarios para validar requerimientos y retroalimentar el producto.

13.2. Gestión del Código y Versionamiento

- Uso de Git y GitHub para control de versiones, gestión de ramas y registro de cambios.
- Aplicación de flujos como Git Flow para diferenciar desarrollo, pruebas y producción.

13.3. Estándares de Codificación

- Uso de convenciones de codificación propias de PHP (PSR-12), JavaScript (ESLint/Prettier) y buenas prácticas en SQL.
- Documentación del código con comentarios claros y coherentes.

13.4. Seguridad de la Información

- Cifrado de contraseñas mediante bcrypt.
- Implementación de roles y permisos para garantizar la seguridad en el acceso a los datos.
- Uso de HTTPS en entornos de despliegue.

13.5. Aseguramiento de la Calidad

- Pruebas unitarias y de integración para validar funcionalidades críticas.
- Uso de entornos de staging antes del despliegue en producción.
- Documentación de errores y aplicación de testing de regresión.

13.6. Documentación y Mantenimiento

- Generación de documentación técnica y manuales de usuario.
- Diccionario de datos actualizado para la base de datos.
- Plan de copias de seguridad y recuperación ante desastres.

13.7. Mejora Continua

- Retroalimentación continua de los usuarios y equipo de desarrollo.
- Monitoreo de métricas de uso y desempeño para proponer mejoras.
- Actualización tecnológica de frameworks y librerías para mantener compatibilidad y seguridad.

14. CONCLUSIÓN

15. BIBLIOGRAFÍA y CIBERGRAFÍA

16. GLOSARIO

